操作系统

Operating Systems

刘川意 教授

哈尔滨工业大学（深圳）

2025年9月

Module 2-1：进程（Process）

1. 进程的引入
2. 进程模型
3. 进程表示
4. 进程类型和特性
5. 进程控制
6. 进程调度

进程的引入

为什么要引入进程?

回顾OS发展

(1) 手工操作 ← 一次装入并运行一个“作业”

(2) 简单批处理 ← 同时装入多个“程序”，串行执行

(3) 多道程序批处理 ← 同时装入多个程序，并发执行，仅当等待I/O时切换程序执行

(4) 分时处理 ← 同时装入多个程序，并发执行，程序切换条件：执行的时间长度(时间片)到 or I/O等待

(5) 对称多处理

进程概念 – OS发展的需要和必然

进程的引入

- 多道程序并发执行是提高系统资源利用率(特别是CPU)的有效途径
- 多道程序竞争CPU进行轮换执行是提高多用户响应速度的有效方法
- 计算机的客观实际支持多道程序并发的实现
- 但“作业”、“程序”等概念不能有效描述程序动态轮换执行的过程
- … …

需要一种统一的方法监视、管理、控制处理器中不同程序的动态执行过程，“进程”的概念被引入！

进程的引入

需要一种统一的方法监视、管理、控制处理器中不同程序的动态执行过程，“进程”的概念被引入！

如何独立地反映每个程序的执行？

- 引入关键的数据结构：进程控制块(PCB)
- 目的：使参与并发执行的程序(含数据)能独立地运行，为其配置一个专门的数据结构

程序执行的要素：代码、数据

进程映像(进程实体)的概念：

由程序段、相关数据段和PCB三部分构成进程映像

进程的引入

进程与程序的区别和联系？

从不同角度，进程与程序的关系是多样的：

- 进程是程序的一次执行过程。
- 进程是一个程序及其数据在处理机上顺序执行时所发生的活动。
- 进程是具有独立功能的程序在一个数据集合上运行的过程，它是系统进行资源分配和调度的独立单位。

程序是静态的，而进程是动态的。

进程的引入

【例题】一个进程映像是（ ）

A. 由协处理器执行的一个程序　　B. 一个独立的程序 + 数据集

C. PCB结构与程序和数据的组合　　D. 一个独立的程序

答案：C

进程映像是PCB、程序段和数据的组合。

【例题】进程与程序的根本区别是（ ）

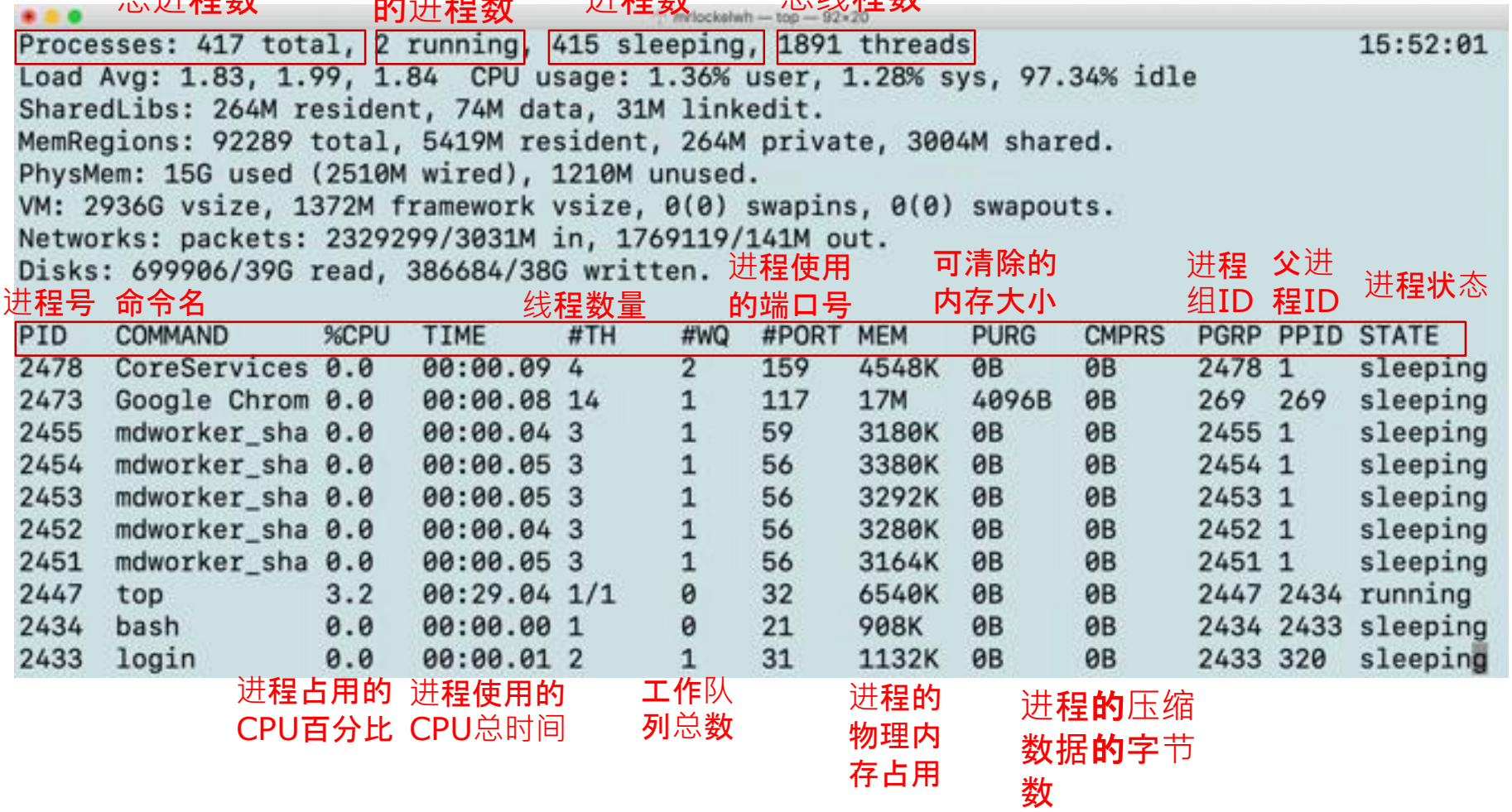
A. 静态和动态特点　　B. 是不是被调入内存

C. 是不是具有就绪、运行和等待三种状态　　D. 是不是占有处理器

答案：A

动态性是进程最重要的特性，以此来区分文件形式的静态程序。操作系统引入进程的概念，是为了从变化的角度动态地分析和研究程序的执行。

现代OS多进程实例



- 使用top命令查看Mac操作系统进程

现代OS多进程实例

使用top –o +state命令对state字段进行排序，可以实时查看到正在运行的进程：

```
Processes: 420 total, 5 running, 415 sleeping, 2142 threads                                 01
Load Avg: 4.80, 2.57, 2.07  CPU usage: 8.2% user, 2.40% sys, 89.56% idle
SharedLibs: 280M resident, 70M data, 50M linkedit.
MemRegions: 103377 total, 5784M resident, 221M private, 2779M shared.
PhysMem: 16G used (2184M wired), 366M unused.
VM: 3071G vsize, 1911M framework vsize, 31722(0) swapins, 39210(0) swapouts.
Networks: packets: 53434/27M in, 132300/28M out. Disks: 482419/8109M read, 285490/2532M written.

PID    COMMAND      %CPU  TIME     #TH    #WQ  #PORT MEM    PURG   CMPRS  PGRP   PPID   STATE
44095  vmrest       0.0   00:00.11 11/1   2    128+  9104K+ 0B     0B     766    779    running
44050  top          3.2   00:00.47 1/1    0    26    4800K+ 0B     0B     44050  33392  running
318    sysmond      0.7   00:23.31 3/1    2/1  28    7368K  0B     464K   318    1      running
245    WindowServer 2.2   03:33.70 9/1    4    3490  304M   82M+   26M    245    1      running
0      kernel_task  1.0   01:40.57 255/12 0    0     16M+   0B     0B     0      0      running
41133  Google Chrom 0.0   00:00.07 13     1    107   14M    8192B  0B     354    354    sleeping
40992  CoreServices 0.0   00:00.34 3      1    164   4368K  0B     0B     40992  1      sleeping
39982  mdworker_sha 0.0   00:00.36 3      1    59    20M    0B     0B     39982  1      sleeping
38080  FMIPClientXP 0.0   00:00.07 3      1    73    6692K  24K    0B     38080  1      sleeping
38077  siriknowledg 0.0   00:00.03 2      1    43    2788K  20K    0B     38077  1      sleeping
38063  com.apple.St 0.0   00:00.01 2      1    31    1352K  0B     0B     38063  1      sleeping
38054  AssetCacheLo 0.0   00:00.04 3      1    58    4004K  0B     0B     38054  1      sleeping
```

进程的引入

```
1.#include <stdio.h>
2.#include <stdlib.h>
3.#include "common.h"
4.
5.int main(int argc, char *argv[])
6.{
7.    if (argc != 2) {
8.    fprintf(stderr, "usage: cpu <string>\n");
9.    exit(1);
10.    }
11.    char *str = argv[1];
12.
13.    while (1) {
14.    printf("%s\n", str);
15.    Spin(1);
16.    }
17.    return 0;
18.}
```

cpu.c

进程的引入

- 示例程序会重复检查时间，直到一秒钟过去；一秒过去后，代码将打印用户传入的字符串；直到按下“Control-C”，该程序才终止
- 同时在后台运行同一个程序4次，会怎么样呢？
- 结果说明，这4个程序似乎在同时运行！
- 事实上，在硬件的帮助下，操作系统负责提供这种假象，即系统拥有非常多的虚拟CPU
- 进程就是操作系统为虚拟化CPU所做的努力之一，也是操作系统提供的核心抽象

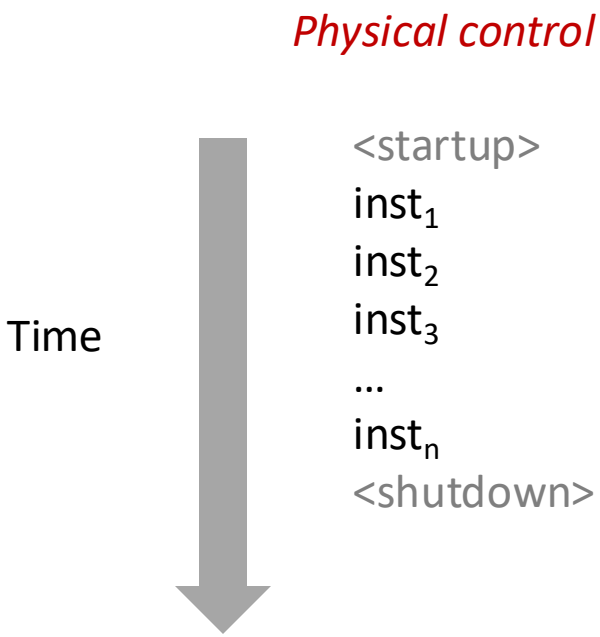
```
linux > ./cpu A & ; ./cpu B & ;
./cpu C & ; ./cpu D &
[1] 2752
[2] 2753
A
B
[3] 2754
[4] 2755
C
D
B
C
D
A
...
```

Module 2-1: 进程(Process)

1. 进程的引入
2. 进程模型
   - 2.1 异常控制流（Exceptional Control Flow）-进阶
   - 2.2 异常（Exceptions）-基础
   - 2.3 进程（Processes）-基础
3. 进程表示
4. 进程类型和特性
5. 进程控制
6. 进程调度

CPU的控制流（Control Flow）

- Processors do only one thing:
  - From startup to shutdown, a CPU simply reads and executes (interprets) a sequence of instructions, one at a time
  - This sequence is the CPU's *control flow* (控制流)



改变控制流（Altering the Control Flow）

- 目前，两种机制可以改变控制流：
  - Jumps and branches 跳转和分支
  - Call and return 调用和返回

  React to changes in ***program state（程序状态）***

- 对一个有用的系统来说是不够的:
  Difficult to react to changes in ***system state（系统状态）***
  - Data arrives from a disk or a network adapter
  - Instruction divides by zero
  - User hits Ctrl-C at the keyboard
  - System timer expires

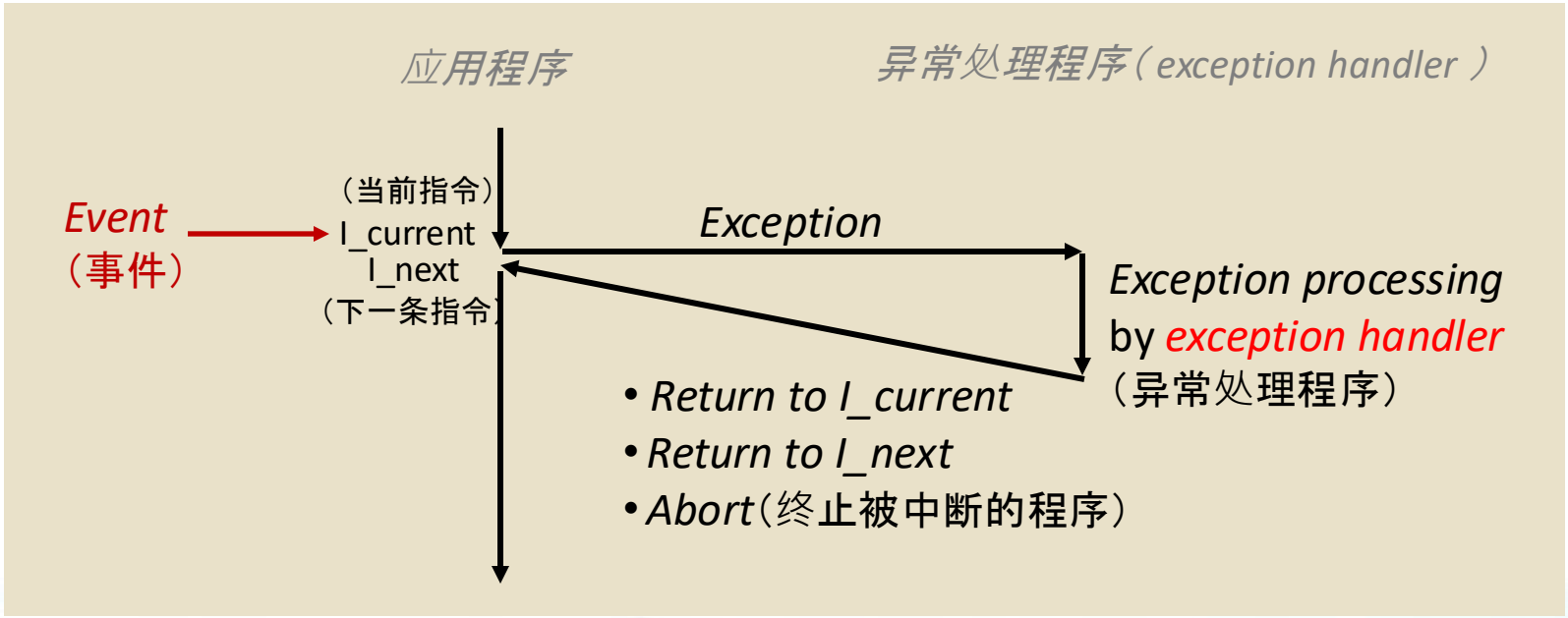
- 系统需要“异常控制流”机制

异常控制流（Exceptional Control Flow）

- Exists at all levels of a computer system
- Low level mechanisms
  - 1. 异常（**Exceptions**）
    - Change in control flow in response to a system event (i.e., change in system state)
    - Implemented using combination of hardware and OS software
- Higher level mechanisms
  - 2. 进程上下文切换（**Process context switch**）
    - Implemented by OS software and hardware timer
  - 3. 信号（**Signals**）
    - Implemented by OS software(应用层)
  - 4. 全局跳转（**Nonlocal jumps**）: `setjmp()` and `longjmp()`
    - Implemented by C runtime library
    - 详见CSAPP 3e：8.6

异常（Exceptions）

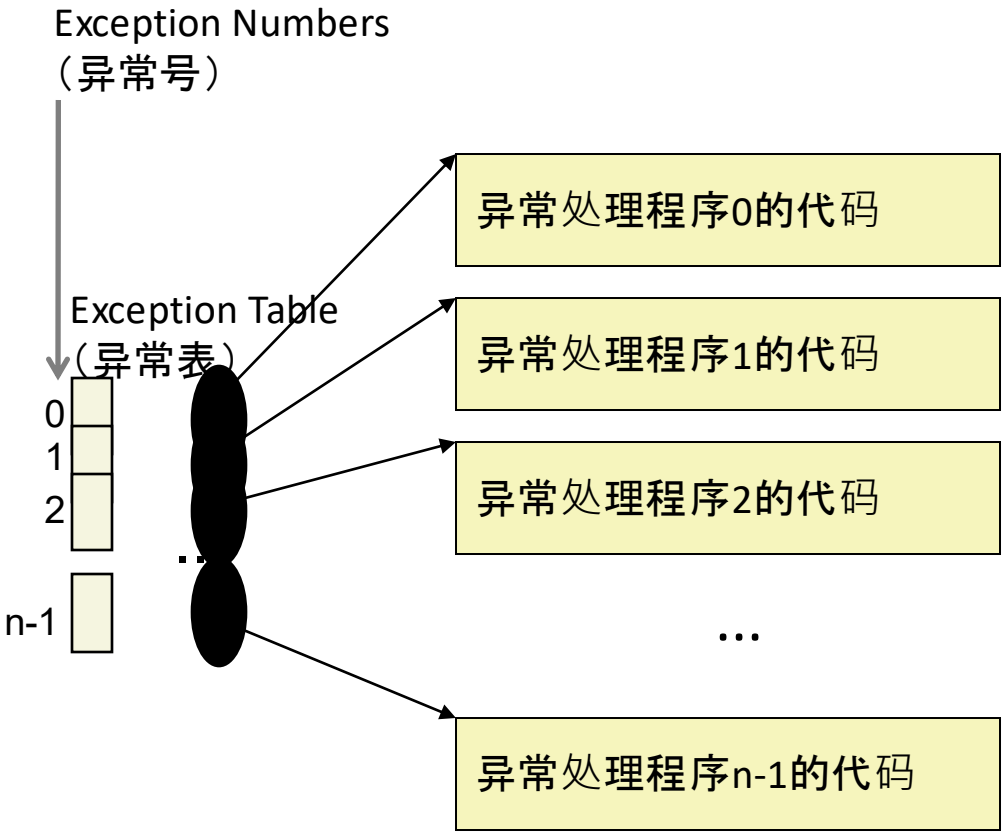
- An *exception* (异常) is a transfer of control to the OS *kernel* (内核) in response to some *event* (i.e., change in processor state)
  - Kernel is the memory-resident part of the OS (内核是OS的内存驻留部分)
  - Examples of events: Divide by 0, arithmetic overflow(算术溢出), page fault(缺页), I/O request completes, typing Ctrl-C



Module 2-1: 进程(Process)

1. 进程的引入
2. 进程模型
   - 2.1 异常控制流（Exceptional Control Flow）-进阶
   - 2.2 异常（Exceptions）-基础
   - 2.3 进程（Processes）-基础
3. 进程表示
4. 进程类型和特性
5. 进程控制
6. 进程调度

异常表（Exception Tables）



- Each type of event has a unique exception number k（每种类型的异常有唯一的异常号k）
- k = index into exception table (异常表的索引) (a.k.a. interrupt vector 中断向量)
- Handler k is called each time exception k occurs（每当异常k发生时，异常处理程序k被调用）

异常表（Exception Tables）

```
01.  .code16  .code [16|32]: 指定指令代码产生的长度  .code16会使汇编器承担目标处理器正在运行16位模式
02.  init:
03.      mov %cs, %ax                    ;将代码段寄存器的值送入AX
04.      mov %ax, %ds                    ;将数据段的地址置为代码段的地址
05.      mov %ax, %es                    ;将附加段的地址置为代码段的地址
06.      call  DispStr                  ;调用显示字符串例程
07.      jmp .                           ;无限循环，.表示当前行编译后的地址
08.      ;; 以上就是整个程序的执行过程了
09.      ;; 下面是DispStr子程序
10.  DispStr:
11.      mov $BootMessage, %ax   ;将字符串首地址传给寄存器ax
12.      mov %ax, %bp      ;CPU将用bp来寻址字符串
13.      mov $16, %cx      ;通过cx，CPU知道字符串的长度
14.      mov $0x1301, %ax ;AH=13表示13号中断，AX=01H，表示目标字符串仅仅包含字符，属性在BX中包含，移动光标
15.      mov $0x000c, %bx  ;黑底红字，BX=0CH，高亮
16.      mov $0, %dl       ;dh表示在第几行显示，dl表示第几列显示
17.      int $0x10         ;BIOS的10H中断的13号中断用于显示字符串
18.      ret
19.  BootMessage:    .asciz "Hello, OS world!" ;
20.  .fill 510-(.-init), 1, 0      ;填充剩下空间，使生成的二进制恰好为512字节
21.  .word 0xaa55         ;结束标志，如果发现扇区以0xaa55结束，则BIOS认为它是一个引导扇区
```

对应BIOS的中断向量表

详见：http://stanislavs.org/helppc/int_table.html

https://blog.csdn.net/piaopiaopiaopiaopiao/article/details/9735633

异常的类别

类别	原因	异步/同步	返回行为
中断（Interrupt）	来自I/O设备的信号	异步	总是返回到下一条指令
陷阱（Trap）	有意的异常	同步	总是返回到下一条指令
故障（Fault）	潜在的可恢复的错误	同步	可能返回到当前指令
终止（Abort）	不可恢复的错误	同步	不会返回

中断(Interrupts) / 异步异常（Asynchronous Exceptions）

- Caused by events external to the processor（由处理器外部的事件产生）
  - Indicated by setting the processor's *interrupt pin*
  - *Interrupt handler（中断处理程序）* returns to "next" instruction

- Examples:
  - Timer interrupt（时钟中断）
    - Every few ms, an external timer chip triggers an interrupt（外部定时器芯片触发中断）
    - Used by the kernel to take back control from user programs
  - I/O interrupt from external device（I/O中断）
    - Hitting Ctrl-C at the keyboard
    - Arrival of a packet from a network
    - Arrival of data from a disk

同步异常（Synchronous Exceptions）

- Caused by events that occur as a result of executing an instruction（执行当前指令的结果）
  - ***Traps（陷阱）***
    - Intentional exception（有意的异常）
    - Examples: ***system calls（系统调用）***, breakpoint traps, special instructions
    - Returns control to “next” instruction
  - ***Faults（故障）***
    - Unintentional but possibly recoverable (可恢复的)
    - Examples: page faults (recoverable), protection faults (unrecoverable), floating point exceptions
    - Either re-executes faulting (“current”) instruction or aborts（重新执行当前指令或者终止）
  - ***Aborts（终止）***
    - Unintentional and unrecoverable（无意的且不可恢复的）
    - Examples: illegal instruction, parity error, machine check
    - Aborts current program（终止当前程序）

系统调用（System Calls）

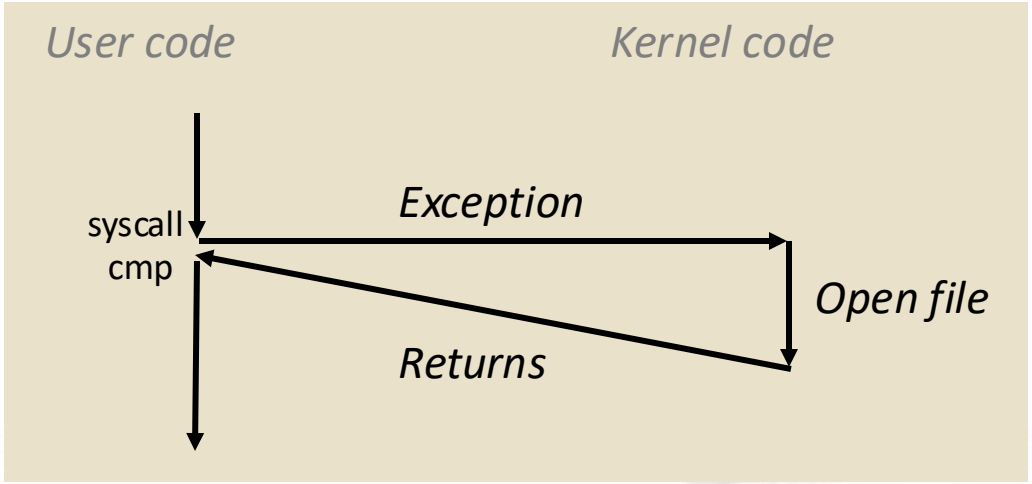
- **Each x86-64 system call has a unique ID number**
- **Examples:**

Number	Name	Description
0	read	Read file
1	write	Write file
2	open	Open file
3	close	Close file
4	stat	Get info about file
57	fork	Create process
59	execve	Execute a program
60	_exit	Terminate process
62	kill	Send signal to process

系统调用举例: Opening File

- User calls: `open(filename, options)`
- Calls `__open` function, which invokes system call instruction `syscall`
  (调用open函数, open函数调用系统调用指令syscall)

```
00000000000e5d70 <__open>:
...
e5d79:  b8 02 00 00 00     mov  $0x2,%eax  # open is syscall #2
e5d7e:  0f 05              syscall         # Return value in %rax
e5d80:  48 3d 01 f0 ff ff  cmp  $0xfffffffffffff001,%rax
...
e5dfa:  c3                 retq
```



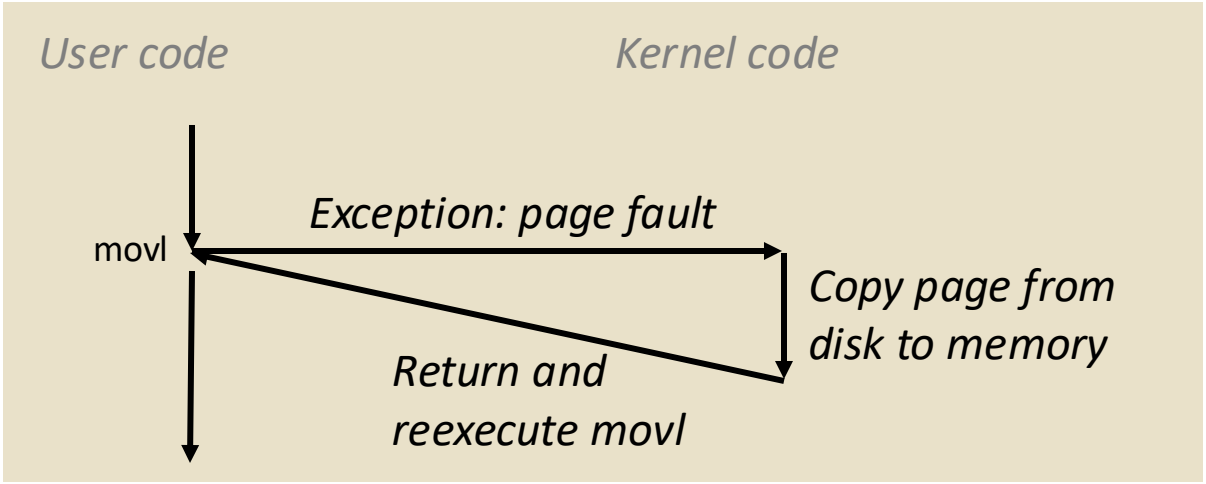
- `%rax` contains syscall number (系统调用号)
- Other arguments in `%rdi`, `%rsi`, `%rdx`, `%r10`, `%r8`, `%r9`
- Return value in `%rax`
- Negative value (负值) is an error corresponding to negative `errno`

故障（Fault）举例1: Page Fault

- User writes to memory location
- That portion (page) of user's memory is currently on disk

```
int a[1000];
main ()
{
    a[500] = 13;
}
```

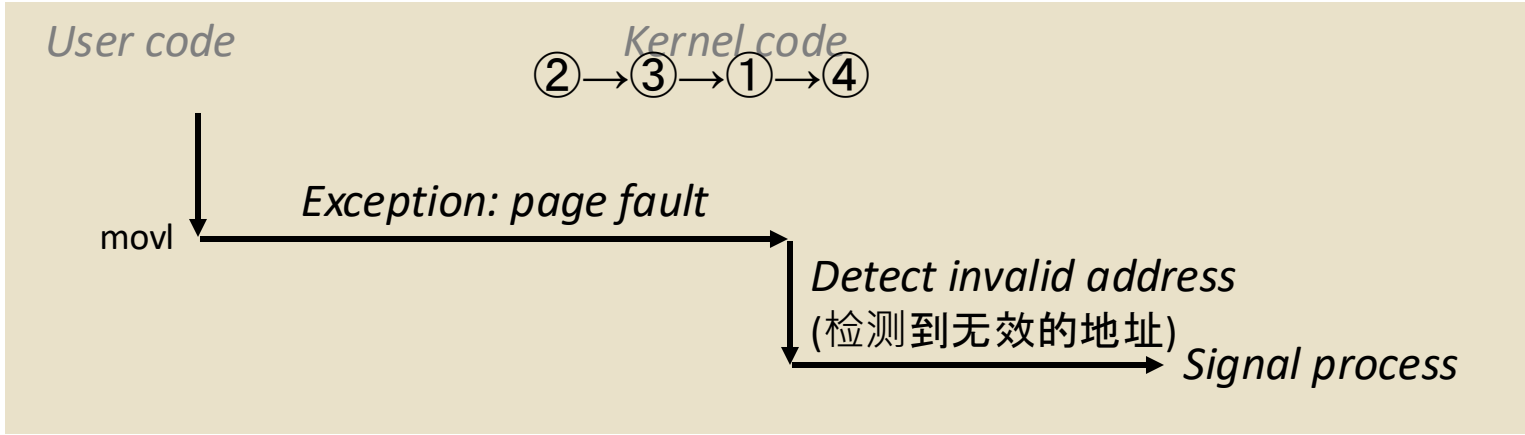
```
80483b7:    c7 05 10 9d 04 08 0d    movl   $0xd,0x8049d10
```



故障（Fault）举例2：Invalid Memory Reference

```
int a[1000];
main ()
{
    a[5000] = 13;
}
```

```
80483b7:      c7 05 60 e3 04 08 0d   movl   $0xd,0x804e360
```



- Sends `SIGSEGV` signal to user process
- User process exits with “segmentation fault”(段错误)

系统调用（System Calls）

【例题】执行系统调用的过程包括如下主要操作：
①返回用户态
②执行陷入(trap)指令
③传递系统调用参数
④执行相应的服务程序

正确的执行顺序是( )。
A. ②→③→①→④ B. ②→④→③→①
C. ③→②→④→① D. ③→④→②→①

答案：C
正在运行的进程先传递系统调用参数，然后由陷入（trap）指令负责将用户态转化为内核态，并将返回地址压入堆栈以备后用，接下来CPU执行相应的内核态服务程序，最后返回用户态。

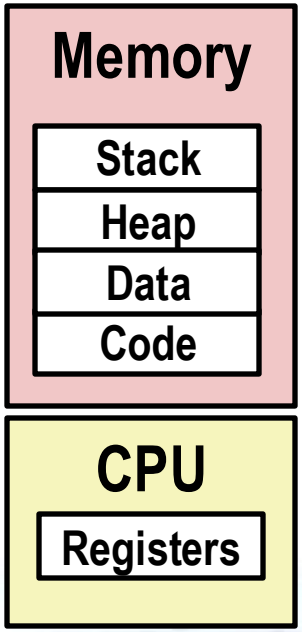
Module 2-1: 进程(Process)

1. 进程的引入
2. 进程模型
   - 2.1 异常控制流（Exceptional Control Flow）-进阶
   - 2.2 异常（Exceptions）-基础
   - 2.3 进程（Processes）-基础
3. 进程表示
4. 进程类型和特性
5. 进程控制
6. 进程调度

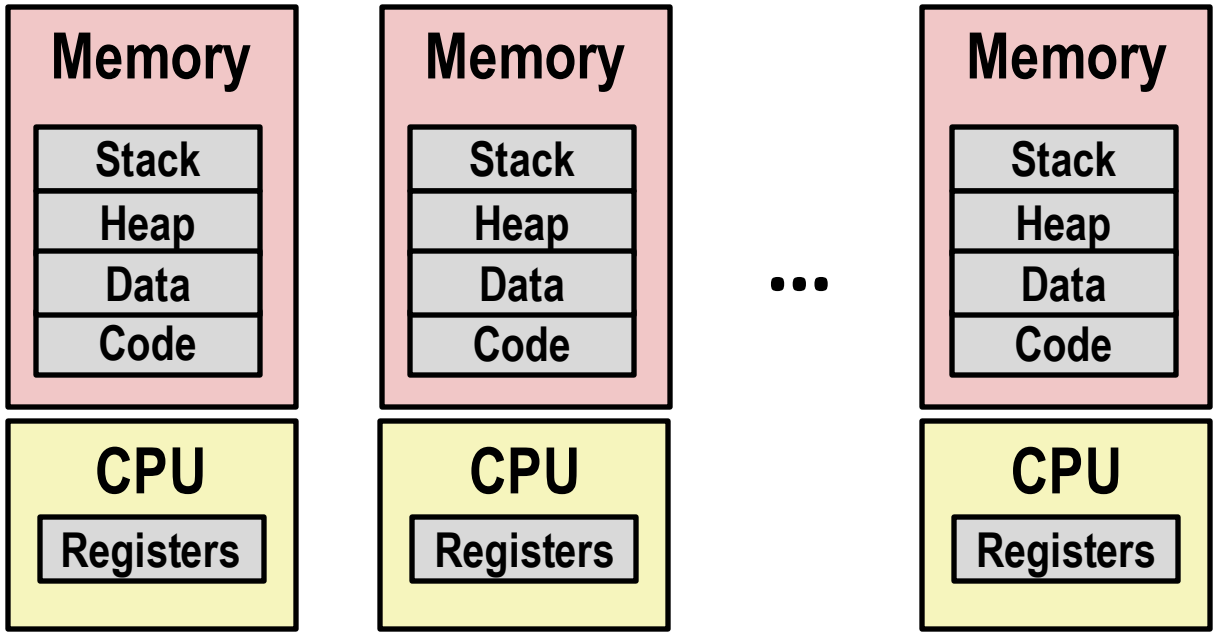
2.3 进程（Process）

- **定义: A process is an instance of a running program（进程是计算机中正在运行的程序的一个实例）**
  - One of the most profound ideas in computer science
  - Not the same as “program” or “processor”

- **进程提供给程序两种主要的抽象（abstractions）：**
  - **逻辑控制流（Logical control flow）**
    - 每个程序似乎独占CPU
    - 内核机制-----上下文切换（context switching）
  - **私有地址空间（Private address space）**
    - 每个程序似乎独占内存
    - 内核机制-----虚拟内存（virtual memory）

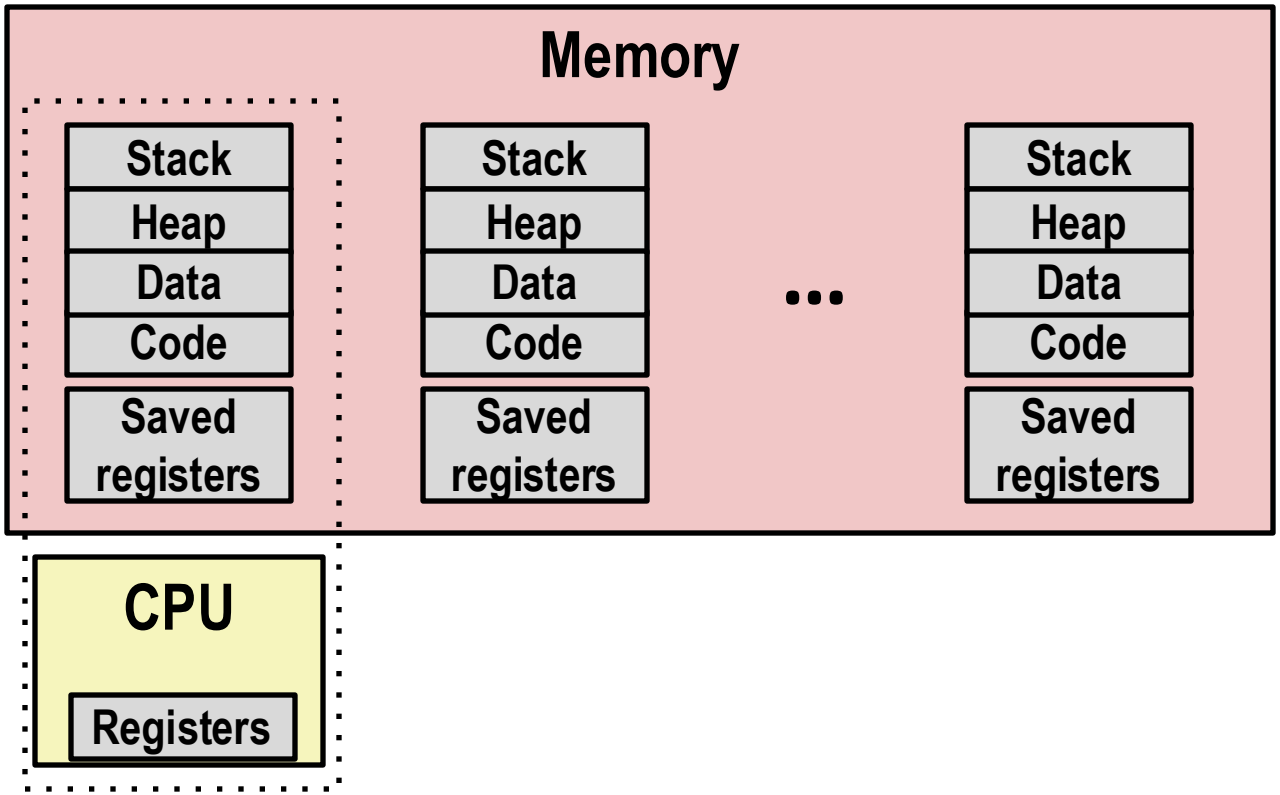


多进程（Multiprocessing）---假象



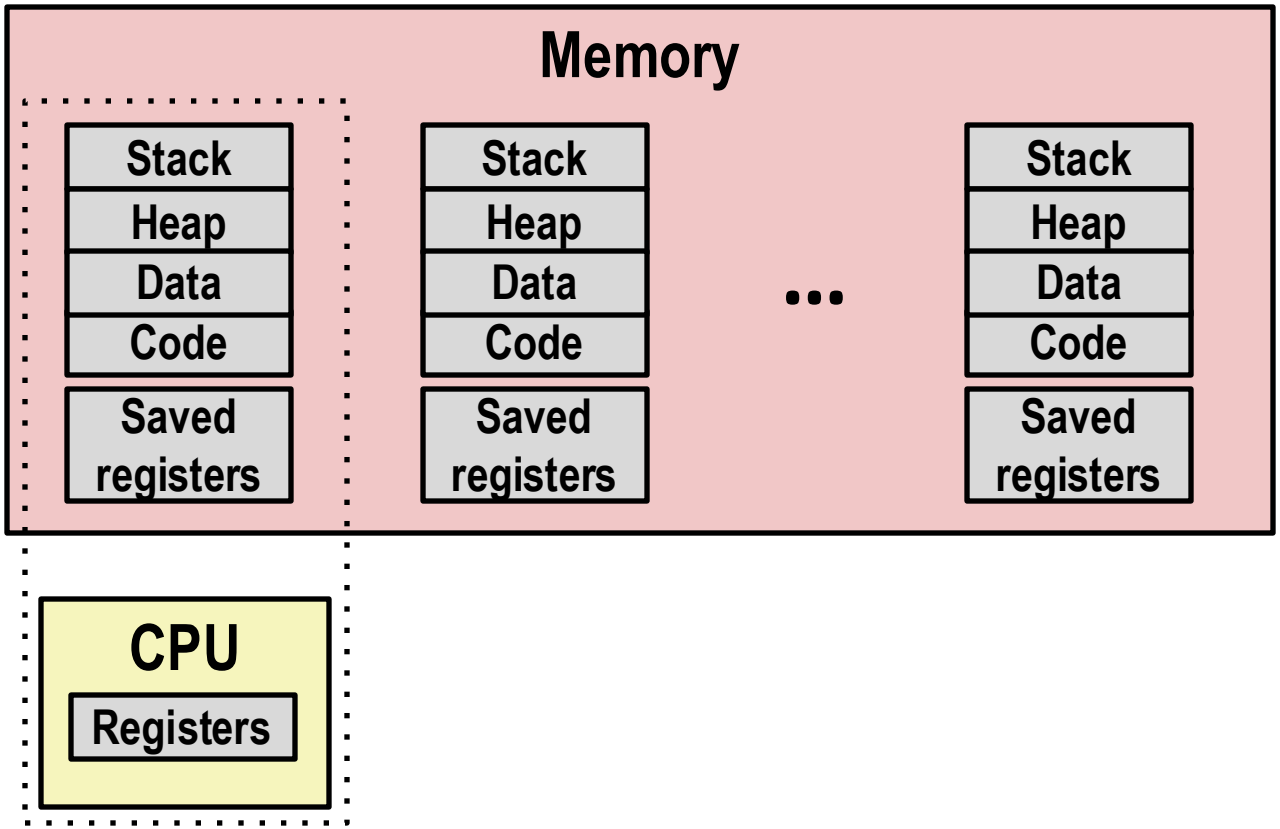
- 计算机同时运行多个进程
  - 多用户应用
    - Web browsers, email clients, editors, …
  - 后台任务
    - Monitoring network & I/O devices

多进程（Multiprocessing）---实际



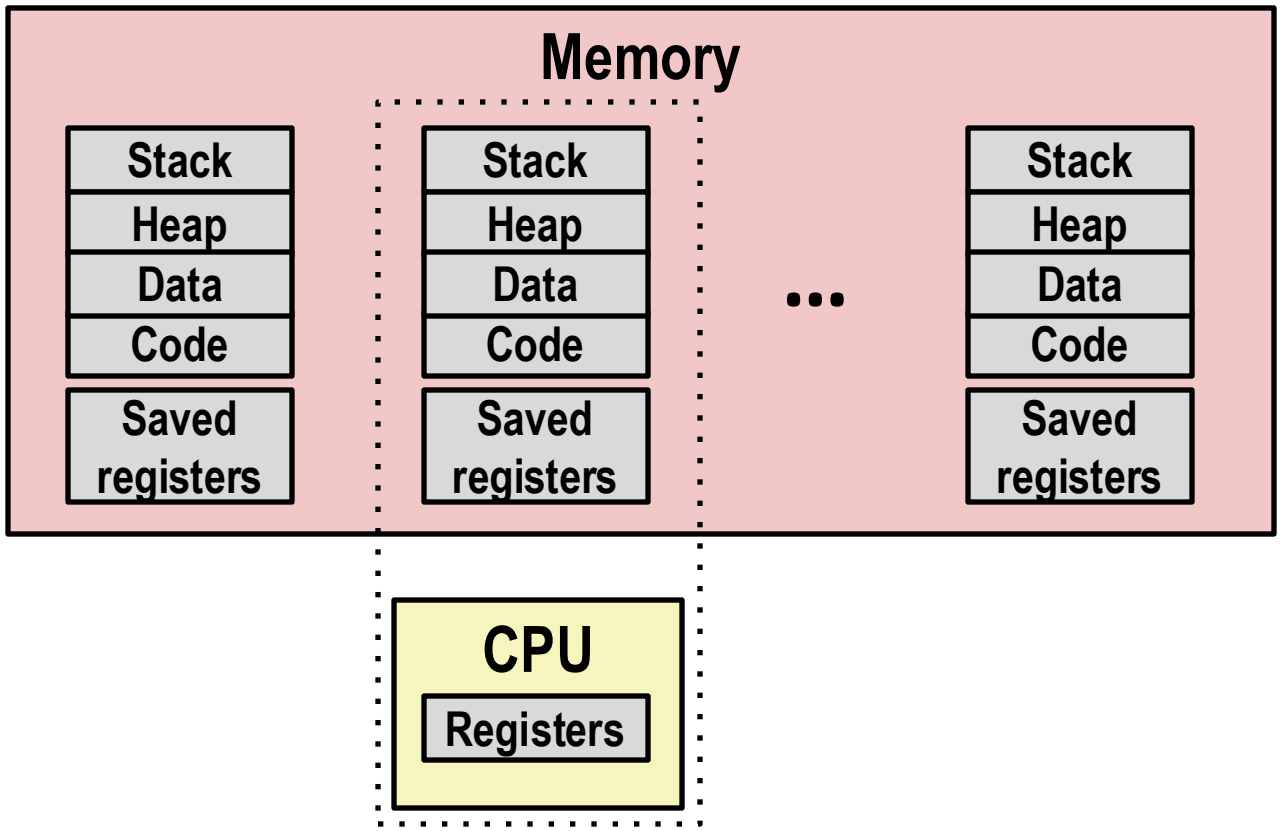
- Single processor executes multiple processes concurrently(并发)
  - Process executions interleaved(交错) (multitasking)
  - Address spaces managed by virtual memory system（虚拟内存）
  - Register(寄存器) values for nonexecuting processes saved in memory

多进程（ Multiprocessing ）---实际



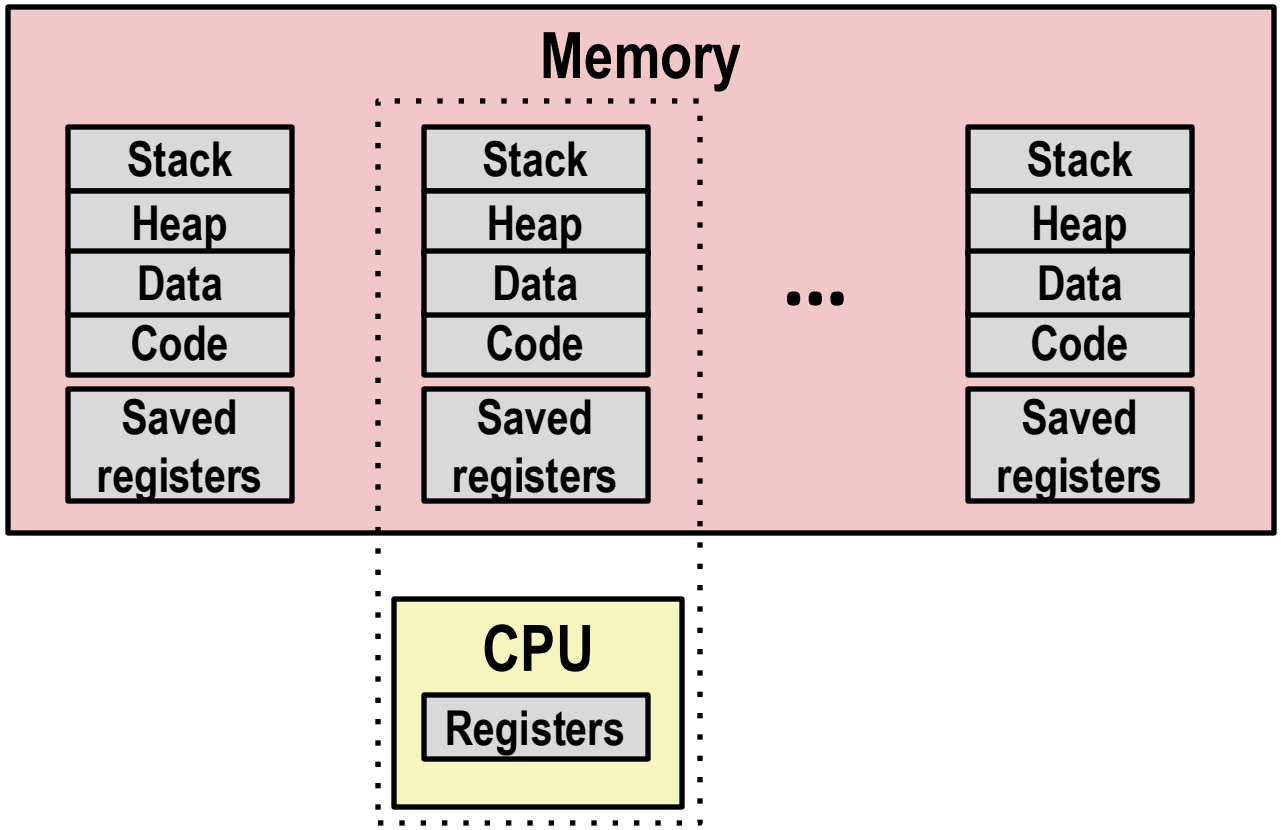
- Save current registers in memory(在内存中存储当前的寄存器值)

多进程（ Multiprocessing ）---实际



- Load saved registers and switch address space (context switch)

多进程（ Multiprocessing ）---实际



- Schedule next process for execution(调度下一个要执行的进程)

私有地址空间（Private address space）

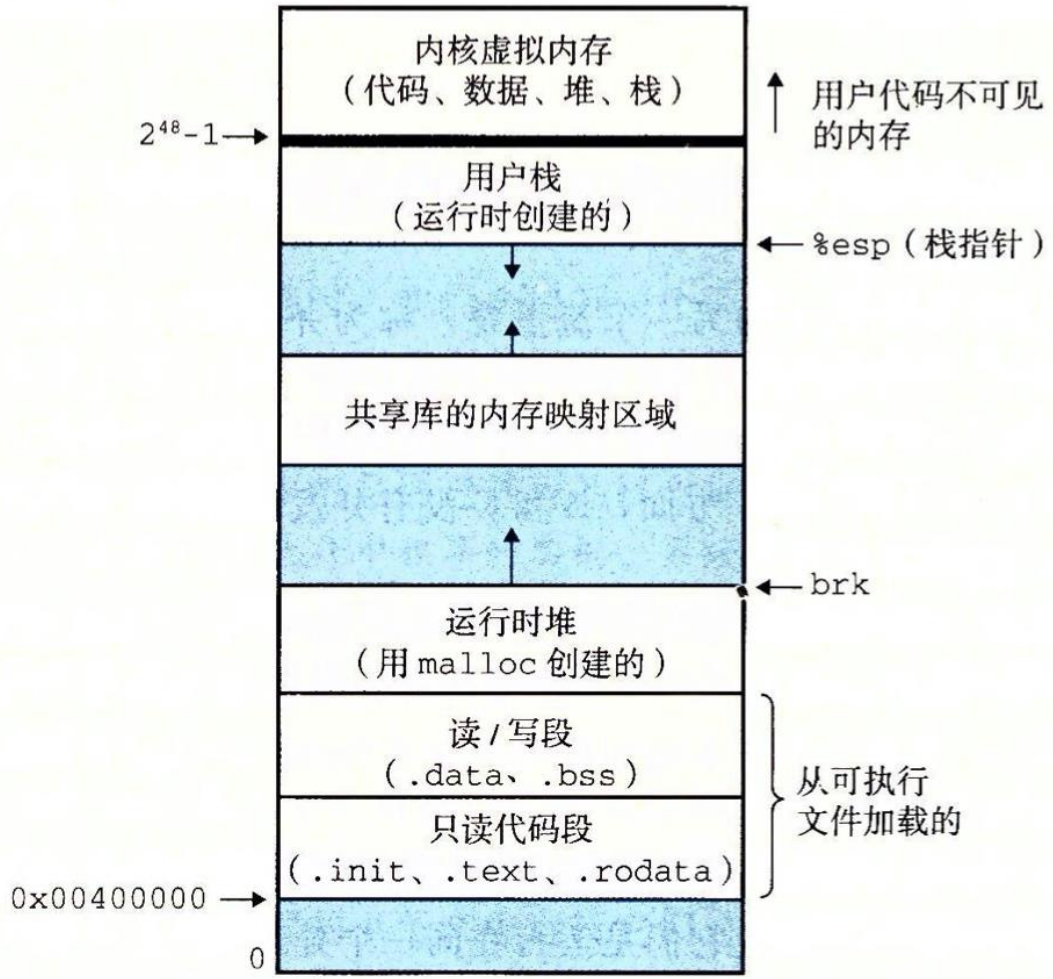
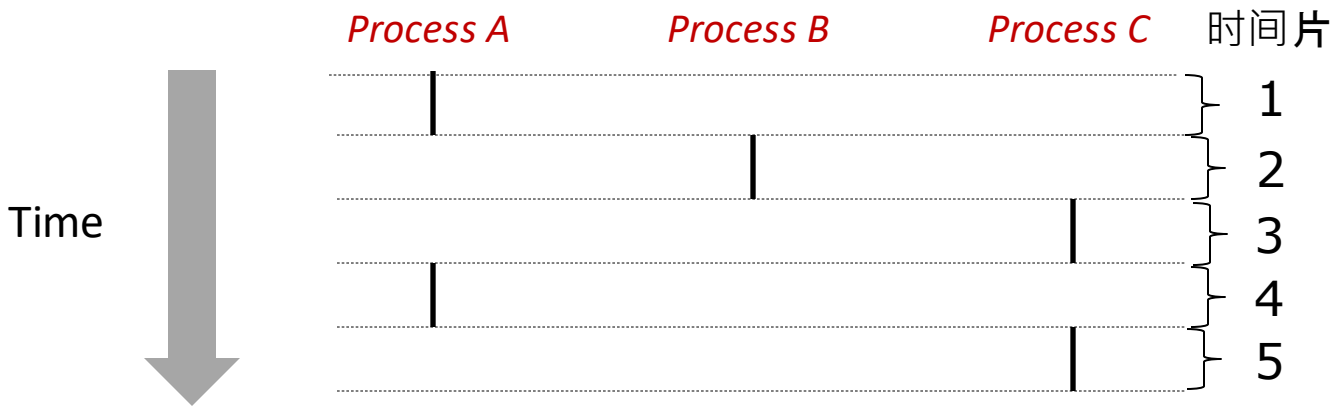


图 8-13　进程地址空间

并发进程（Concurrent Processes）

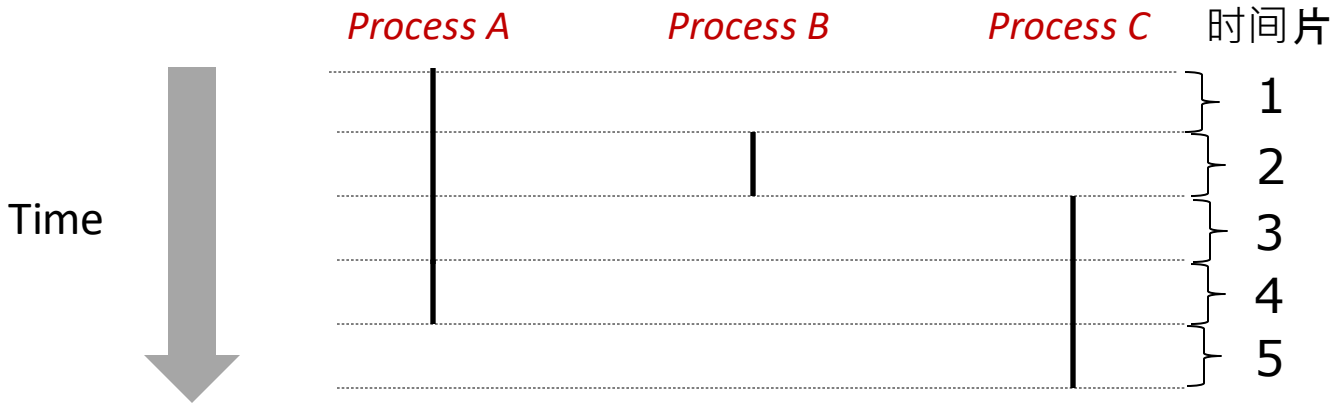
- Each process is a logical control flow(逻辑控制流)
- Two processes *run concurrently (并发)* (*are concurrent)* if their flows overlap in time
- Otherwise, they are *sequential (顺序)*
- Examples (running on single core):
  - Concurrent: A & B, A & C
  - Sequential: B & C



从用户角度看并发进程

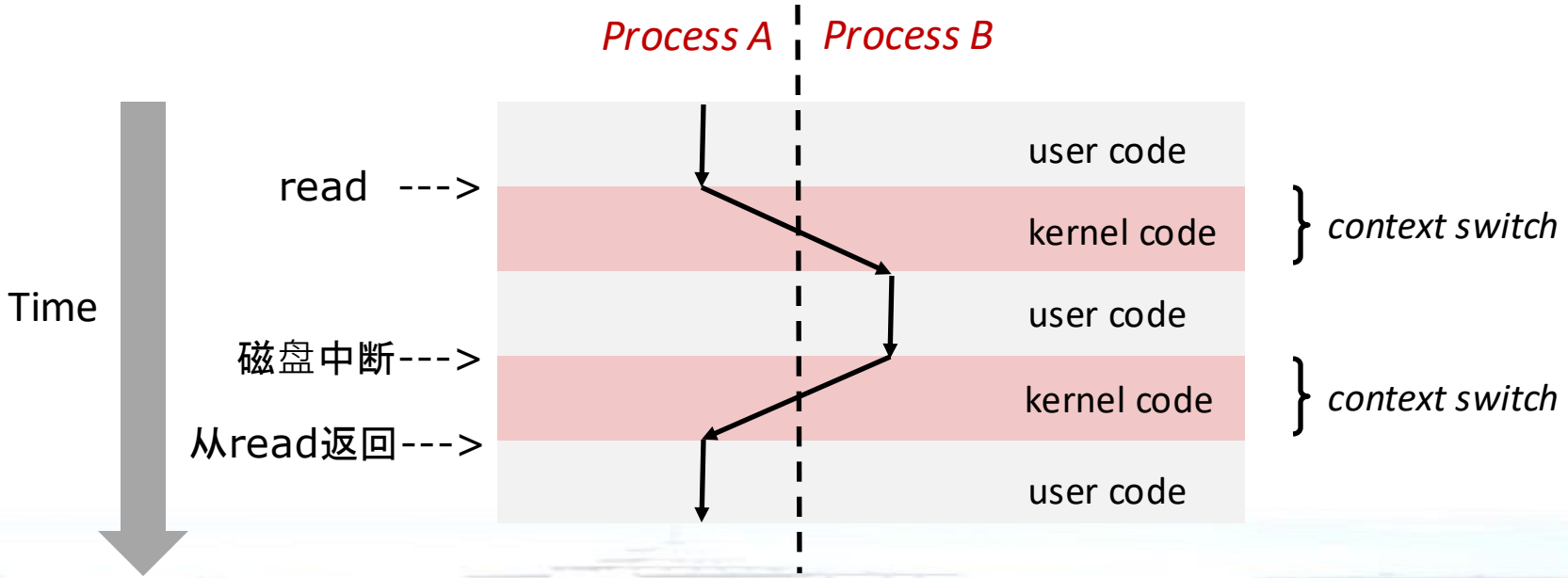
- Control flows for concurrent processes are physically disjoint（不相交的）in time

- However, we can think of concurrent processes as running in parallel（并行）with each other



上下文切换（Context Switching）

- Processes are managed by a shared chunk of memory-resident OS code(一块共享的内存驻留操作系统代码) called the *kernel（内核）*
  - Important: the kernel is not a separate process, but rather runs as part of some existing process.
- Control flow passes from one process to another via a *context switch（上下文切换）*



Summary: 进程模型

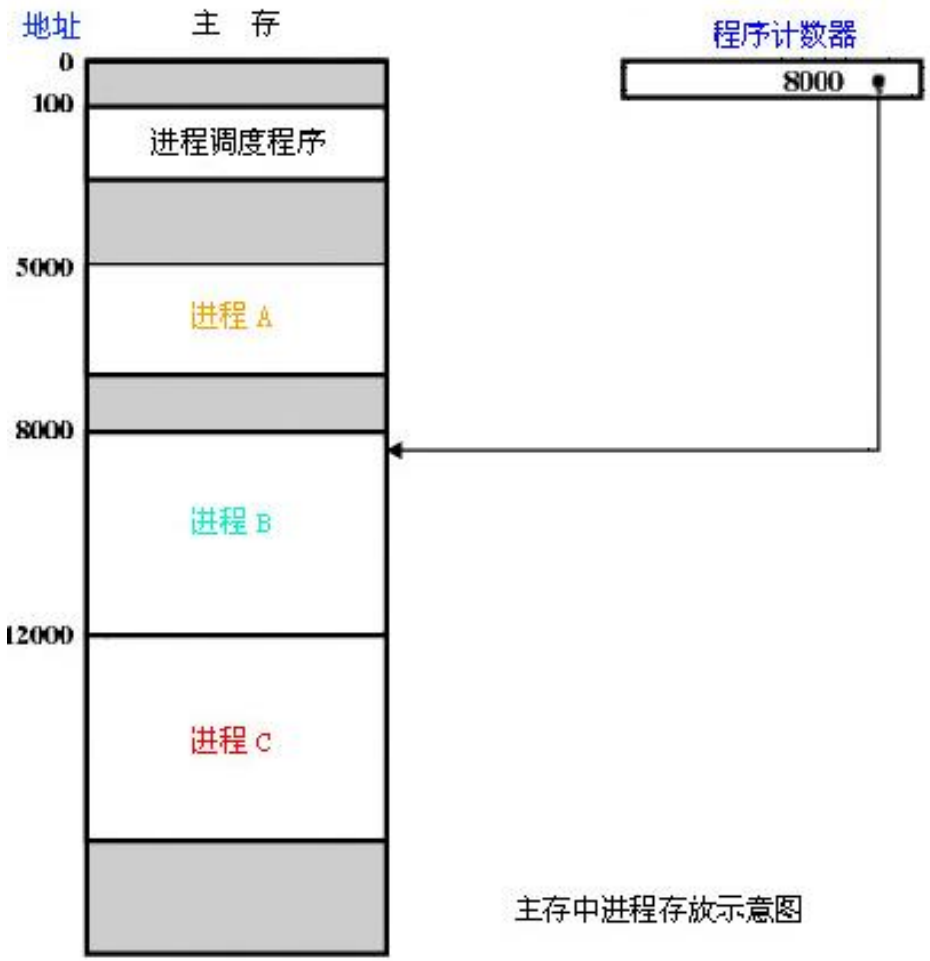
- Exceptions（异常）
  - Events that require nonstandard control flow
  - Generated externally (interrupts) or internally (traps and faults)

- Processes（进程）
  - At any given time, system has multiple active processes
  - Only one can execute at a time on a single core, though
  - Each process appears to have total control of processor + private memory space

Module 2-1：进程（Process）

1. 进程的引入
2. 进程模型
3. 进程表示
   - 3.1 多进程并发执行示例-基础
   - 3.2 进程状态-基础
   - 3.3 进程控制块-基础
   - 3.4 PCB示例-进阶
   - 3.5 Linux内核小窥-进阶
4. 进程类型和特性
5. 进程控制
6. 进程调度

一个多进程并发执行实例



主存中进程存放示意图

一个多进程并发执行实例

进程轨迹 --
进程执行的指令序列

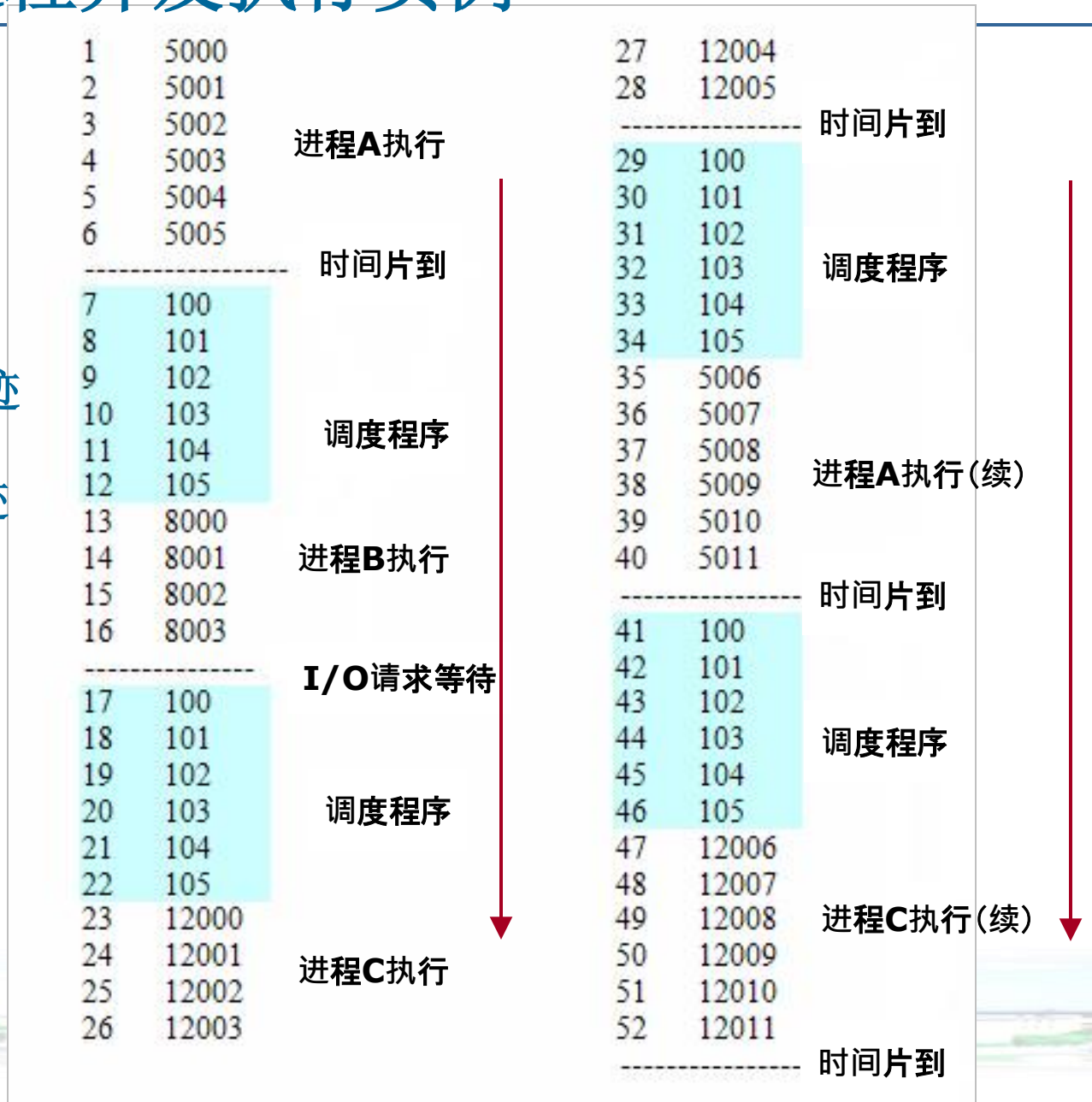
(a)进程A轨迹	(b)进程B轨迹	(c)进程C轨迹
5000	8000	12000
5001	8001	12001
5002	8002	12002
5003	8003	12003
5004	8004	12004
5005	8005	12005
5006	8006	12006
5007		12007
5008		12008
5009		12009
5010		12010
5011		12011

5000 = 进程A的程序起始地址
8000 = 进程B的程序起始地址
12000 = 进程C的程序起始地址

一个多进程并发执行实例

进程ABC组合轨迹

-- CPU调度轨迹

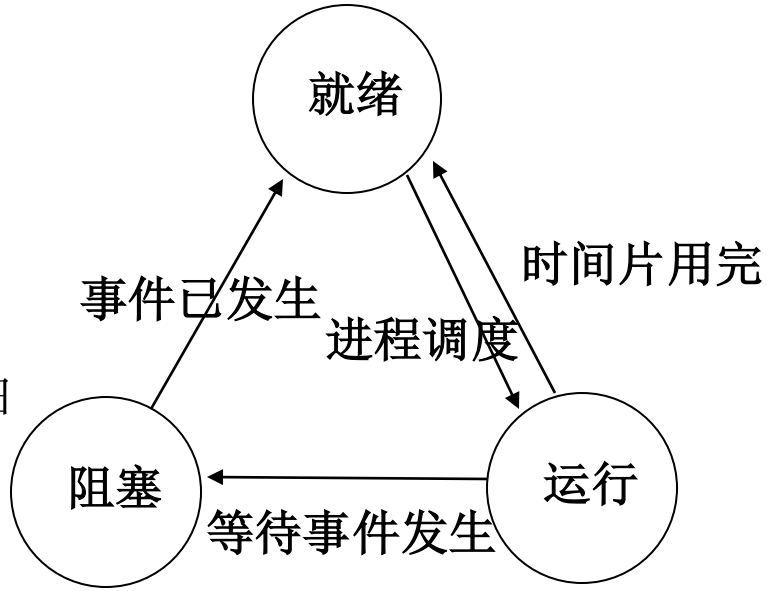


Module 2-1：进程（Process）

1. 进程的引入
2. 进程模型
3. 进程表示
   - 3.1 多进程并发执行示例-基础
   - 3.2 进程状态-基础
   - 3.3 进程控制块-基础
   - 3.4 PCB示例-进阶
   - 3.5 Linux内核小窥-进阶
4. 进程类型和特性
5. 进程控制
6. 进程调度

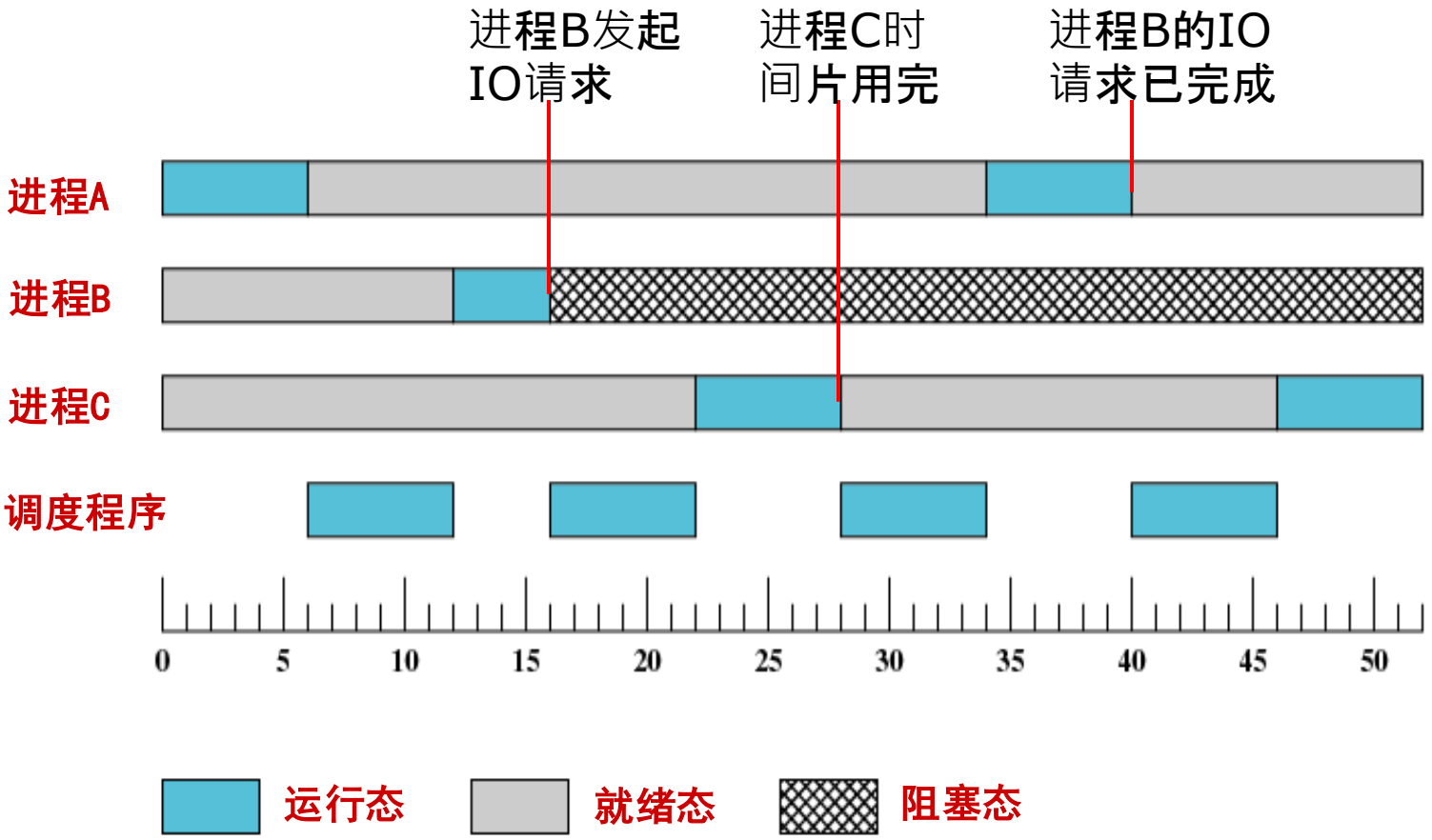
进程状态

- 进程执行时的间断性，决定了进程可能具有多种状态
- 运行中的进程至少具有以下三种基本状态
  - **就绪(ready)** – 在某时刻，进程已获得除CPU以外的所有资源，一旦分到了CPU就可以立即执行
  - **运行(running)** – 进程已经获得必要资源，并占有CPU运行
  - **阻塞(blocked)** – 正在执行的进程，由于发生某事件而暂时无法执行下去
- 进程是有生命周期的，它的状态随着自身的推进和外界条件的变化而发生变化



进程: 状态转换

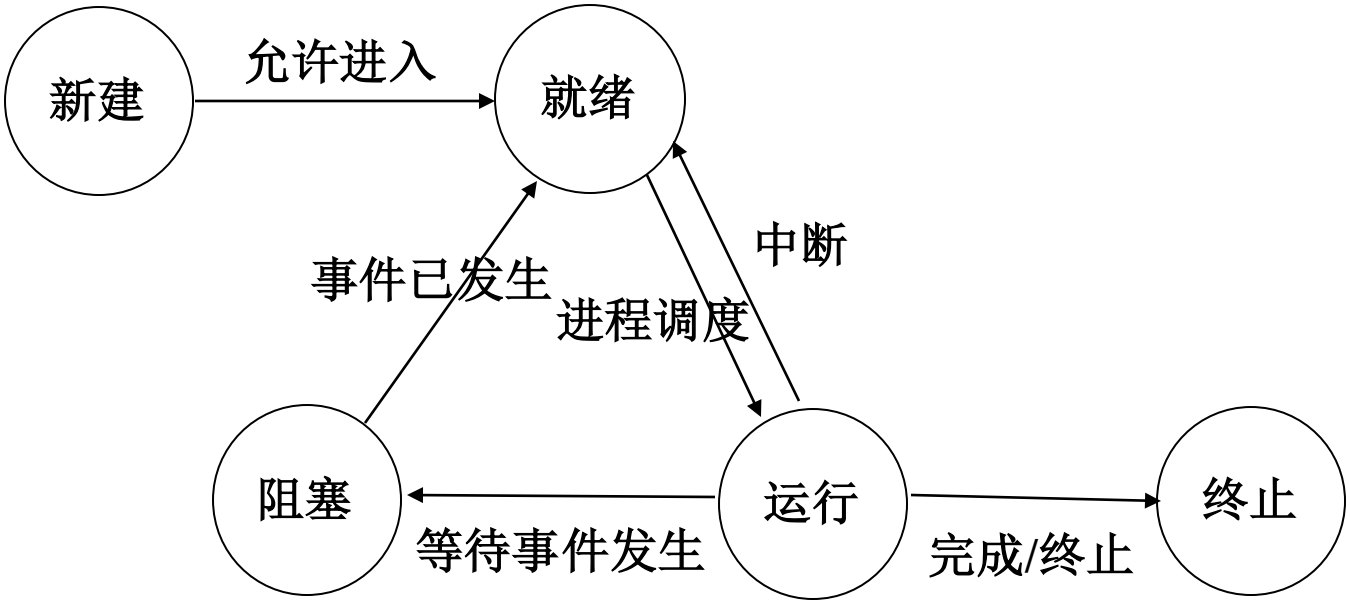
进程状态变化序列图



进程状态变化序列图举例（三状态）

进程状态

在进程的整个生命周期中，它可能处于以下五个状态：



进程状态变化图（五状态）

新建(new) – 至少建立**PCB**，但进程相关的其他内容可能未调入主存

终止(terminated) – 进程已经终止，但资源等待父进程或系统回收

进程状态

【例题】以下可能导致一个进程从运行态变为就绪态的事件是（ ）

A. 一次 I/O 操作结束

B. 运行进程需做 I/O 操作

C. 运行进程结束

D. 出现了比现在进程优先级更高的进程

答案：D

A：进程等待的I/O操作结束，进入就绪态等待唤醒，阻塞态->就绪态

B：进程需要等待I/O操作完成，运行态->阻塞态

C：运行进程结束，直接进入终止态

D：进程时间片用完或被抢占，运行态->就绪态

进程的引入

【例题】支持多道程序设计的操作系统在运行过程中，不断地选择新进程运行来实现CPU的共享，但其中（ ）不是引起操作系统选择新进程的直接原因。

A. 运行进程的时间片用完

B. 运行进程出错

C. 运行进程要等待某一事件发生

D. 有新进程被创建进入就绪态

答案：D

D选项，新进程被创建进入就绪态时，它仅被添加到就绪队列中，不会立即导致调度程序选择新进程运行。这不是直接引发调度的原因，因为只有当当前运行进程主动放弃CPU或被抢占时，调度程序才会选择新进程。

Module 2-1：进程（Process）

1. 进程的引入
2. 进程模型
3. 进程表示
   - 3.1 多进程并发执行示例-基础
   - 3.2 进程状态-基础
   - 3.3 进程控制块-基础
   - 3.4 PCB示例-进阶
   - 3.5 Linux内核小窥-进阶
4. 进程类型和特性
5. 进程控制
6. 进程调度

进程控制块 PCB

进程控制块(Process Control Block)，是一个线性的数据结构，用来描述和记录进程的动态变化信息，是进程的灵魂

- 进程控制块的作用
  - PCB 是存放进程管理和控制信息的数据结构，它是进程管理和控制的最重要的数据结构
  - 系统创建一个进程，就是由系统为某个程序（包含数据段）设置一个PCB；PCB伴随进程运行的全过程；进程执行完成时，由系统收回其PCB，该进程便消亡了
  - 通过PCB可以访问到进程的所有信息
- PCB是进程存在的唯一标志

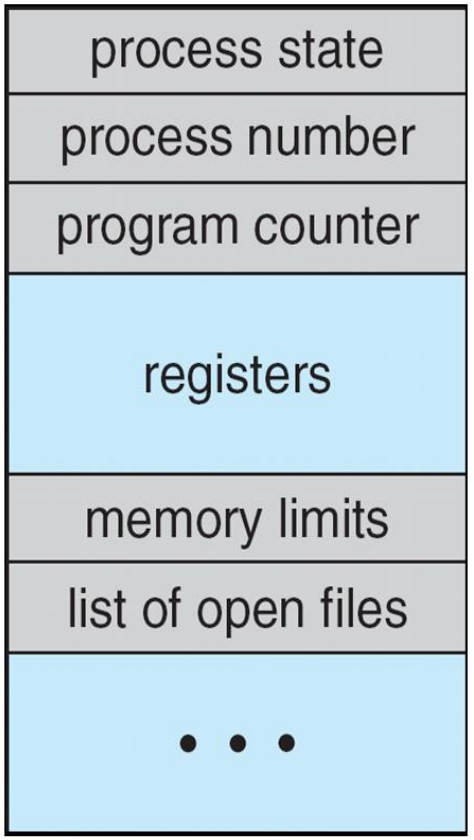
  系统将根据PCB而感知进程的存在，故PCB是进程存在的唯一标志

进程控制块 PCB

进程控制块PCB的内容：

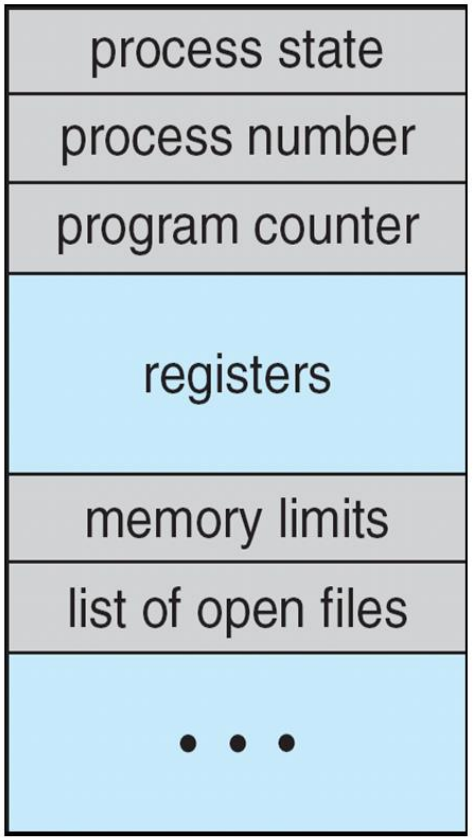
不同操作系统中，PCB的内容不尽相同，但通常包括：

- **进程标识符** 确认进程的唯一标识(Process Identifier, PID)
- **进程状态** 进程调度程序分配处理器的依据
- **程序计数器** 表示进程要执行的下个指令的地址，是被调度时PC寄存器的副本
- **CPU寄存器** 和程序计数器一起被称为执行现场，这些状态信息保证进程再次被调度时可以回到被调度前的状态
- **CPU调度信息** 包括进程优先级、调度队列的指针和其他调度参数等



进程控制块 PCB

- **内存管理信息** 维护了一个进程“独占”的内存空间，主要包括基址和界限寄存器的值、页表或段表
- **I/O状态信息** 包括分配给进程的I/O设备列表、打开文件列表等
- **家族联系** 记录父进程的PID等信息
- **记账信息** 这类信息包括CPU时间、实际使用时间、时间期限、记账数据、作业或进程数量等



简而言之，进程控制块简单地作为这些信息的仓库，这些信息随着进程的不同而不同，同时随着进程的执行而变化

进程控制块 PCB

【例题】若一个进程实体由PCB、共享正文段，数据堆段和数据栈段组成，请指出下列C语言程序中的内容及相关数据结构各位于哪一段中。

I. 全局赋值变量( )
II. 未赋值的局部变量( )
III. 函数调用实参传递值( )
IV. 用malloc()要求动态分配的存储区( )
V. 常量值(如1995、“string”)( )
VI. 进程的优先级( )

A. PCB
B. 正文段
C. 堆段
D. 栈段

答案：B、D、D、C、B、A

程序一般分为：代码段、数据段、堆段、栈段。这里正文段包括代码段和数据段。

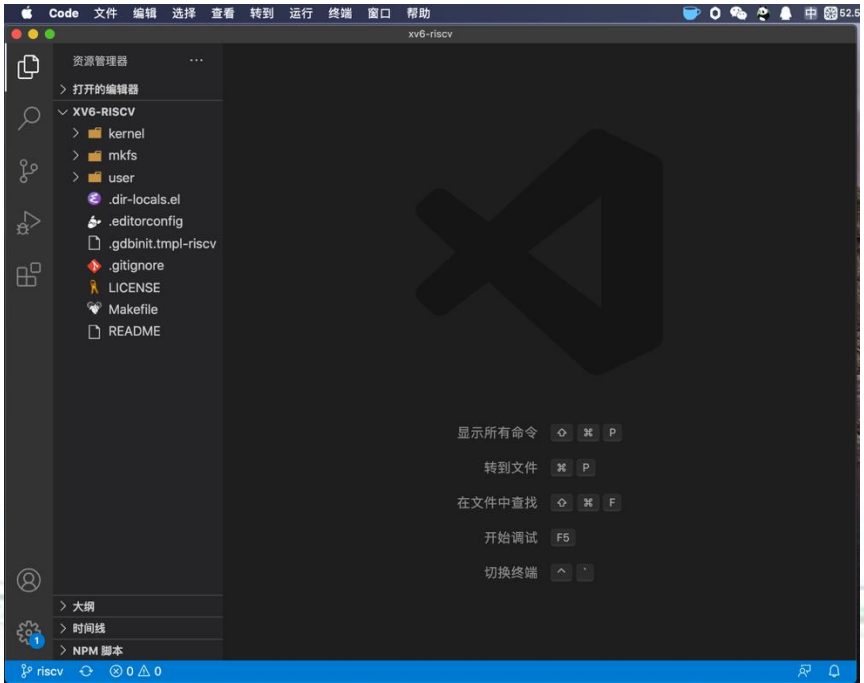
Module 2-1：进程（Process）

1. 进程的引入
2. 进程模型
3. 进程表示
   - 3.1 多进程并发执行示例-基础
   - 3.2 进程状态-基础
   - 3.3 进程控制块-基础
   - 3.4 PCB示例-进阶
   - 3.5 Linux内核小窥-进阶
4. 进程类型和特性
5. 进程控制
6. 进程调度

PCB实例——Xv6: proc

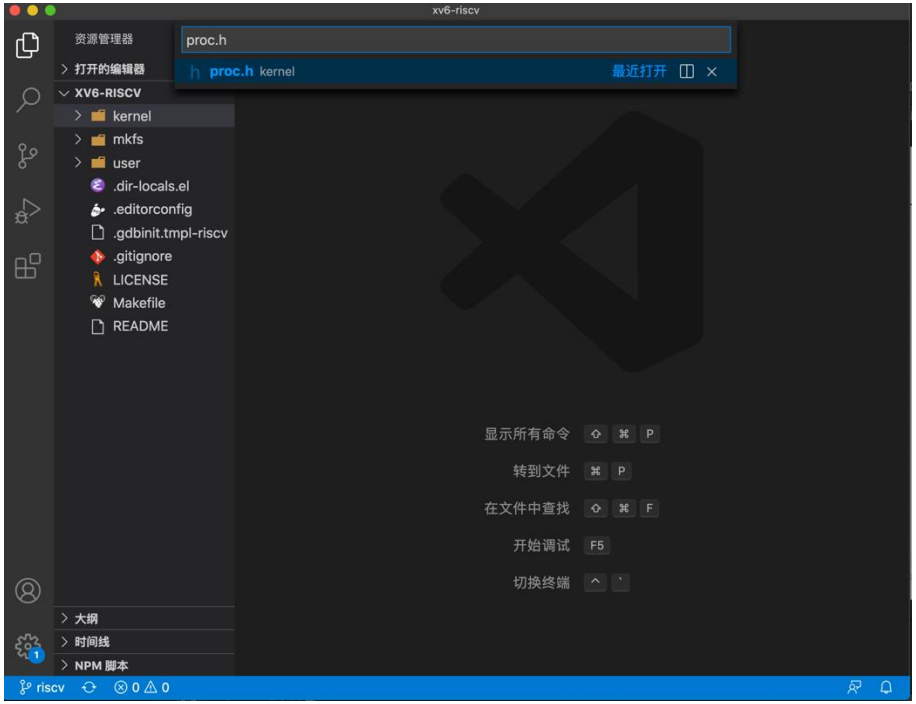
- **Xv6**
  - 2006年MIT为操作系统课程 6.828开发的一个简单、类Unix的教学操作系统
  - 2019年，移植得到xv6 RISC-V版
  - 目前，操作系统课程实验使用的就是xv6 RISC-V版
- Xv6操作系统内核中表示进程的数据结构是proc，定义在kernel/proc.h中

- 使用VSCode解析源码：
  - 下载最新版xv6源码 git clone git://github.com/mit-pdos/xv6-riscv.git
  - 使用VSCode打开，如下图：

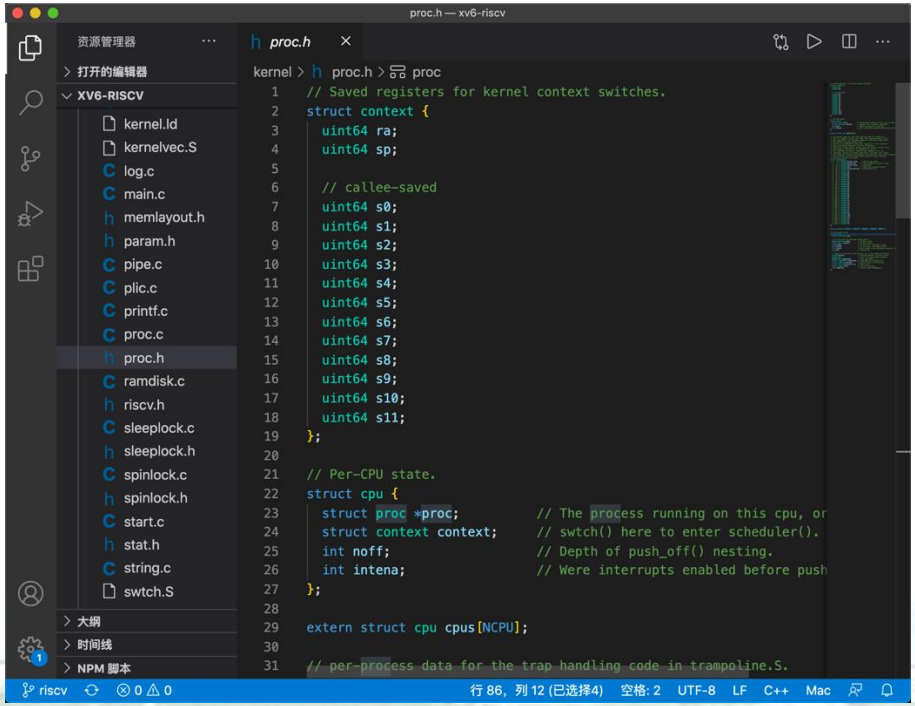


使用VSCode查看Xv6源码

使用Cmd + P (Win: Ctrl +P) 查找文件 proc.h

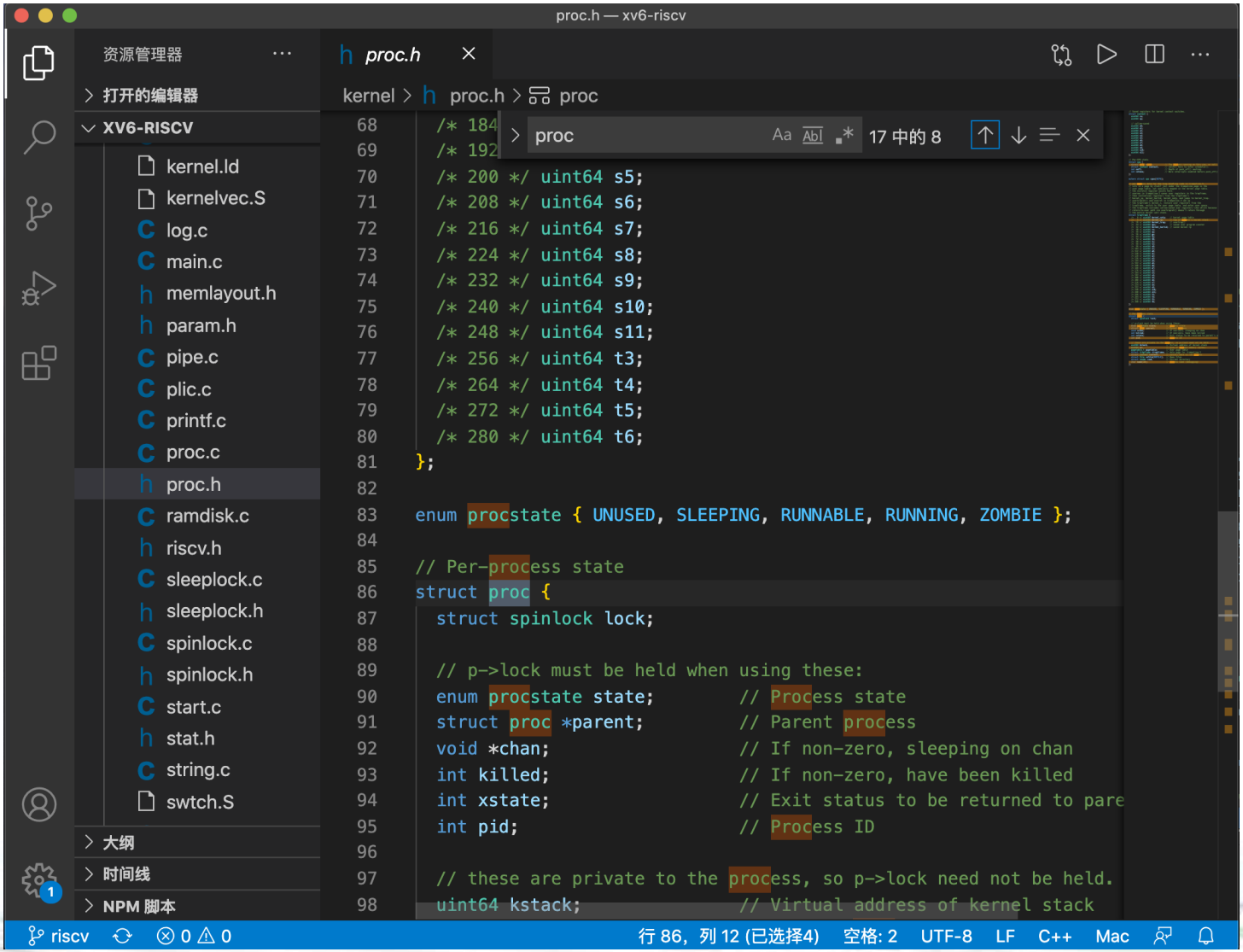


kernel/proc.h: 打开内容如下



使用VSCode查看Xv6源码

使用Ctrl+P快速定位到proc



Proc源码介绍：运行状态

proc.h

kernel > proc.h > cpu

```
79    /* 272 */ uint64 t5;
80    /* 280 */ uint64 t6;
81  };
82
83  enum procstate { UNUSED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
84
85  // Per-process state
86  struct proc {
87    struct spinlock lock;
88
89    // p->lock must be held when using these:
90    enum procstate state;        // Process state
91    struct proc *parent;         // Parent process
92    void *chan;                  // If non-zero, sleeping on chan
93    int killed;                  // If non-zero, have been killed
94    int xstate;                  // Exit status to be returned to parent's wait
95    int pid;                     // Process ID
96
97    // these are private to the process, so p->lock need not be held.
98    uint64 kstack;               // Virtual address of kernel stack
99    uint64 sz;                   // Size of process memory (bytes)
100   pagetable_t pagetable;       // User page table
101   struct trapframe *trapframe; // data page for trampoline.S
102   struct context context;      // swtch() here to run process
103   struct file *ofile[NOFILE];  // Open files
104   struct inode *cwd;           // Current directory
105   char name[16];               // Process name (debugging)
106 };
107
```

初始化状态， 阻塞态，就绪态，运行态，终止态

保存了进程所有可能的状态

Xv6内核为每一个进程都维护了许多状态，把这些状态组织在结构体proc中。在这些状态中，对一个进程来说最重要的就是页表 pagetable， 内核栈kstack和运行状态 state。我们可以使用p->xxx来执行proc结构体的某个元素

进程的运行状态，在任何时间都只能是enum procstate中的某一个状态

Proc源码介绍：页表

proc.h

kernel > proc.h > cpu

```
79    /* 272 */ uint64 t5;
80    /* 280 */ uint64 t6;
81  };
82
83  enum procstate { UNUSED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
84
85  // Per-process state
86  struct proc {
87    struct spinlock lock;
88
89    // p->lock must be held when using these:
90    enum procstate state;        // Process state
91    struct proc *parent;         // Parent process
92    void *chan;                  // If non-zero, sleeping on chan
93    int killed;                  // If non-zero, have been killed
94    int xstate;                  // Exit status to be returned to parent's wait
95    int pid;                     // Process ID
96
97    // these are private to the process, so p->lock need not be held.
98    uint64 kstack;               // Virtual address of kernel stack
99    uint64 sz;                   // Size of process memory (bytes)
100   pagetable_t pagetable;       // User page table
101   struct trapframe *trapframe; // data page for trampoline.S
102   struct context context;      // swtch() here to run process
103   struct file *ofile[NOFILE];  // Open files
104   struct inode *cwd;           // Current directory
105   char name[16];               // Process name (debugging)
106 };
107
```

Xv6为每一个进程维护了一个单独的页表，这个页表定义了该进程的地址空间。Xv6页表把一个虚拟地址（RISC-V指令所使用的地址）翻译成物理地址（CPU芯片发送到内存的地址）

进程的页表

Proc源码介绍：内核栈

proc.h

kernel > proc.h > cpu

```
79    /* 272 */ uint64 t5;
80    /* 280 */ uint64 t6;
81  };
82
83  enum procstate { UNUSED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
84
85  // Per-process state
86  struct proc {
87    struct spinlock lock;
88
89    // p->lock must be held when using these:
90    enum procstate state;        // Process state
91    struct proc *parent;         // Parent process
92    void *chan;                  // If non-zero, sleeping on chan
93    int killed;                  // If non-zero, have been killed
94    int xstate;                  // Exit status to be returned to parent's wait
95    int pid;                     // Process ID
96
97    // these are private to the process, so p->lock need not be held.
98    uint64 kstack;               // Virtual address of kernel stack
99    uint64 sz;                   // Size of process memory (bytes)
100   pagetable_t pagetable;       // User page table
101   struct trapframe *trapframe; // data page for trampoline.S
102   struct context context;      // swtch() here to run process
103   struct file *ofile[NOFILE];  // Open files
104   struct inode *cwd;           // Current directory
105   char name[16];               // Process name (debugging)
106 };
107
```

每一个进程都有两个栈：用户栈和内核栈(p->kstack)。用户栈位于用户地址空间，内核栈位于内核地址空间

当进程执行用户态指令时，仅使用用户栈，而内核栈为空。

当进程进入内核（使用系统调用或中断）时，内核代码将在进程的内核栈上执行；且当进程位于内核中时，其用户栈仍包含已保存的数据，但未被有效使用。

进程的线程在交替的使用其用户堆栈和内核堆栈。 内核堆栈是独立的（不受用户代码的干扰），因此即使进程破坏了用户堆栈，内核也可以执行

内核栈的虚拟地址

Proc源码介绍：其他字段

proc.h

kernel > proc.h > cpu

```
79    /* 272 */ uint64 t5;
80    /* 280 */ uint64 t6;
81  };
82
83  enum procstate { UNUSED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
84
85  // Per-process state
86  struct proc {
87    struct spinlock lock;
88
89    // p->lock must be held when using these:
90    enum procstate state;        // Process state
91    struct proc *parent;         // Parent process
92    void *chan;                  // If non-zero, sleeping on chan
93    int killed;                  // If non-zero, have been killed
94    int xstate;                  // Exit status to be returned to parent's wait
95    int pid;                     // Process ID
96
97    // these are private to the process, so p->lock need not be held.
98    uint64 kstack;               // Virtual address of kernel stack
99    uint64 sz;                   // Size of process memory (bytes)
100   pagetable_t pagetable;       // User page table
101   struct trapframe *trapframe; // data page for trampoline.S
102   struct context context;      // swtch() here to run process
103   struct file *ofile[NOFILE];  // Open files
104   struct inode *cwd;           // Current directory
105   char name[16];               // Process name (debugging)
106 };
107
```

进程所有的状态

进程持有的锁

保存指向父进程的指针

如果非零，则保存该进程睡眠时的队列

标识进程是否已经被杀死，非零：已被杀死

进程的退出状态

进程标识符pid：作为进程的唯一标识

进程内存的大小（单位是字节）

trapframe

进程上下文

打开文件

进程的当前工作目录

进程名，最多是15个字符

Proc源码：上下文定义

proc.h

kernel > proc.h > cpu

保存进程切换时的寄存器内容

```
1   // Saved registers for kernel context switches.
2   struct context {
3     uint64 ra;
4     uint64 sp;
5
6     // callee-saved
7     uint64 s0;
8     uint64 s1;
9     uint64 s2;
10    uint64 s3;
11    uint64 s4;
12    uint64 s5;
13    uint64 s6;
14    uint64 s7;
15    uint64 s8;
16    uint64 s9;
17    uint64 s10;
18    uint64 s11;
19  };
20
21  // Per-CPU state.
22  struct cpu {
23    struct proc *proc;          // The process running on this cpu, or null.
24    struct context context;     // swtch() here to enter scheduler().
25    int noff;                   // Depth of push_off() nesting.
26    int intena;                 // Were interrupts enabled before push_off()?
27  };
28
29  extern struct cpu cpus[NCPU];
30
31  // per-process data for the trap handling code in trampoline.S.
32  // sits in a page by itself just under the trampoline page in the
33  // user page table. not specially mapped in the kernel page table.
34  // the sscratch register points here.
35  // uservec in trampoline.S saves user registers in the trapframe,
36  // then initializes registers from the trapframe's
```

Proc源码：陷阱帧定义

proc.h

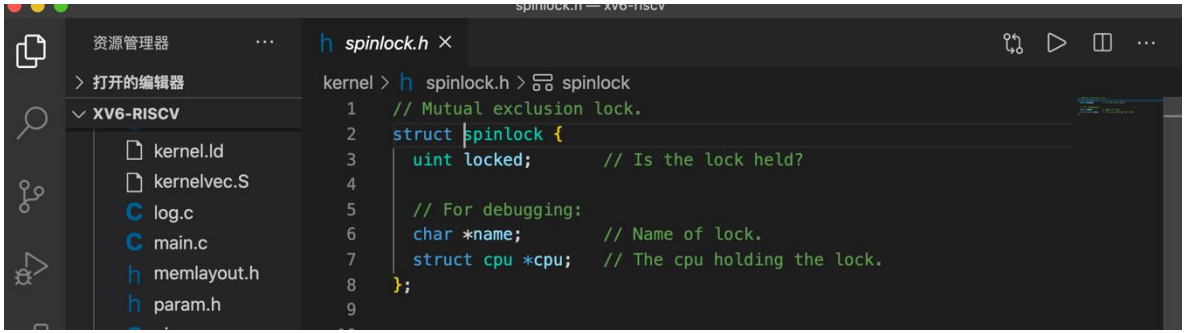
kernel > proc.h > cpu

进程当前中断的中断帧(trap frame)

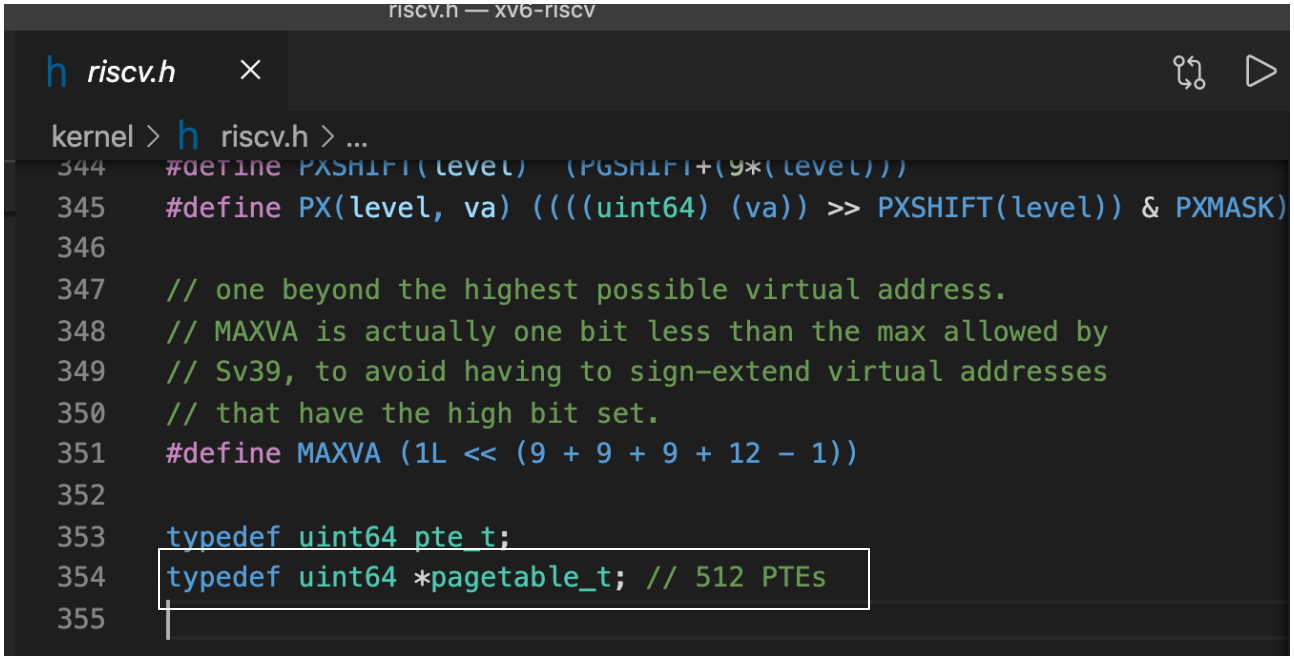
```
44  struct trapframe {
45    /*   0 */ uint64 kernel_satp;   // kernel page table
46    /*   8 */ uint64 kernel_sp;     // top of process's kernel stack
47    /*  16 */ uint64 kernel_trap;   // usertrap()
48    /*  24 */ uint64 epc;           // saved user program counter
49    /*  32 */ uint64 kernel_hartid; // saved kernel tp
50    /*  40 */ uint64 ra;
51    /*  48 */ uint64 sp;
52    /*  56 */ uint64 gp;
53    /*  64 */ uint64 tp;
54    /*  72 */ uint64 t0;
55    /*  80 */ uint64 t1;
56    /*  88 */ uint64 t2;
57    /*  96 */ uint64 s0;
58    /* 104 */ uint64 s1;
59    /* 112 */ uint64 a0;
60    /* 120 */ uint64 a1;
61    /* 128 */ uint64 a2;
62    /* 136 */ uint64 a3;
63    /* 144 */ uint64 a4;
64    /* 152 */ uint64 a5;
65    /* 160 */ uint64 a6;
66    /* 168 */ uint64 a7;
67    /* 176 */ uint64 s2;
68    /* 184 */ uint64 s3;
69    /* 192 */ uint64 s4;
70    /* 200 */ uint64 s5;
71    /* 208 */ uint64 s6;
72    /* 216 */ uint64 s7;
73    /* 224 */ uint64 s8;
74    /* 232 */ uint64 s9;
75    /* 240 */ uint64 s10;
76    /* 248 */ uint64 s11;
77    /* 256 */ uint64 t3;
```

Proc源码中涉及的其他定义

选中要查看的内容, F12快速跳转到定义处，查看定义



struct spinlock:互斥锁



pagetable_t：指向uint64的指针类型

Proc源码中涉及的其他定义

file.h — xv6-riscv

file.h

kernel > file.h > file

```
1  struct file {
2    enum { FD_NONE, FD_PIPE, FD_INODE, FD_DEVICE } type;
3    int ref; // reference count
4    char readable;
5    char writable;
6    struct pipe *pipe; // FD_PIPE
7    struct inode *ip;  // FD_INODE and FD_DEVICE
8    uint off;          // FD_INODE
9    short major;       // FD_DEVICE
10 };
11
12 #define major(dev)  ((dev) >> 16 & 0xFFFF)
13 #define minor(dev)  ((dev) & 0xFFFF)
14 #define mkdev(m,n)  ((uint)((m)<<16| (n)))
15
16 // in-memory copy of an inode
17 struct inode {
18   uint dev;           // Device number
19   uint inum;          // Inode number
20   int ref;            // Reference count
21   struct sleeplock lock; // protects everything below here
22   int valid;          // inode has been read from disk?
23
24   short type;         // copy of disk inode
25   short major;
26   short minor;
27   short nlink;
28   uint size;
29   uint addrs[NDIRECT+1];
30 };
31
```

动态查看Proc的相关信息

- 插入打印语句的关键点：
  - 一定要在init进程启动并初始化完成后，打印它的相关信息
  - Xv6的启动过程： _entry.S -> start.c -> main.c， 其中main.c调用proc.c中的userinit函数，完成init进程的创建和初始化
  - xv6在很早就已经完成了printf的初始化，可以放心的使用printf语句
  - 因此，在kernel/proc.c的userinit函数的最后，加入一些printf语句

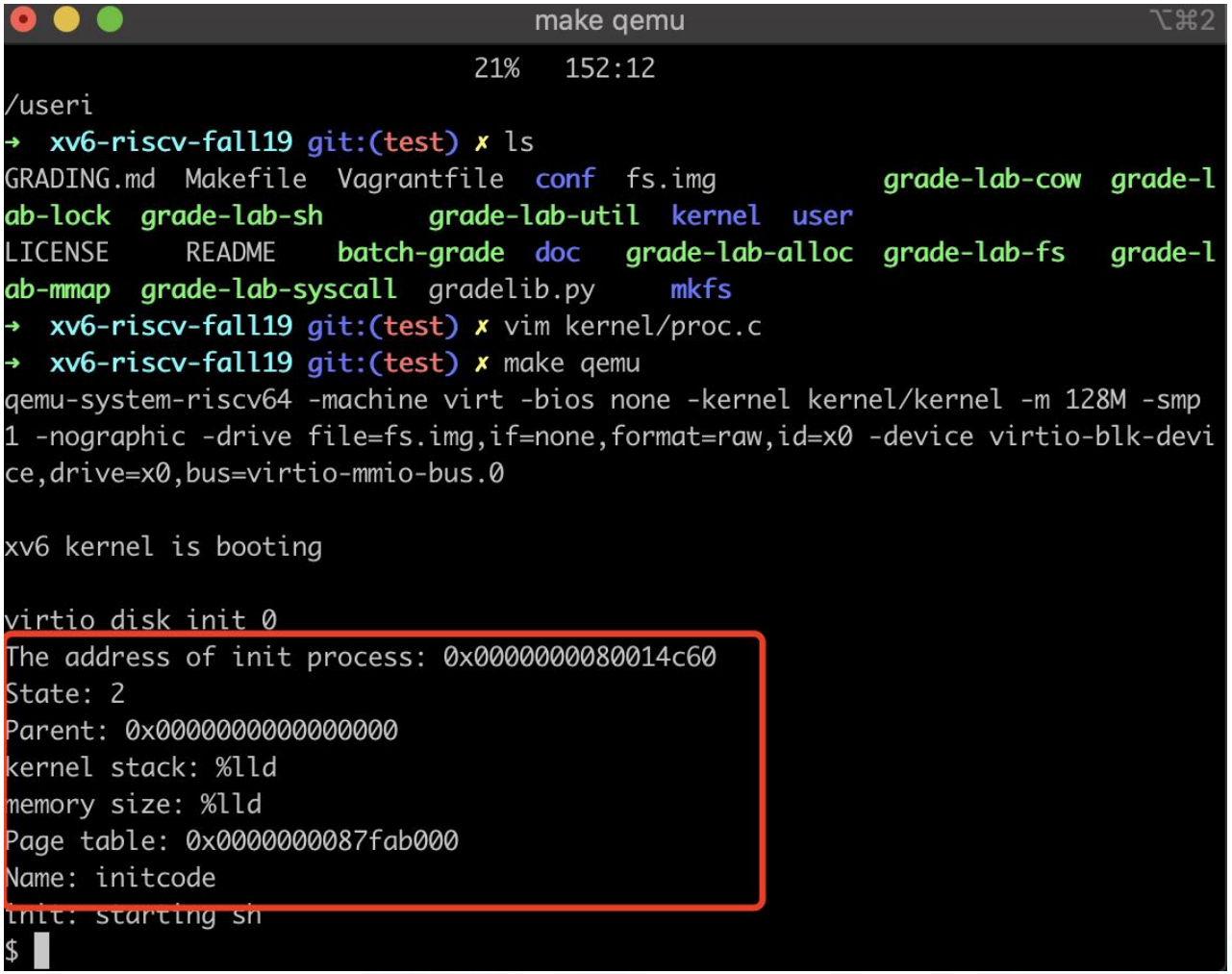
```
1 proc.c
200 userinit(void)
201 {
202   struct proc *p;
203
204   p = allocproc();
205   initproc = p;
206
207   // allocate one user page and copy init's instructions
208   // and data into it.
209   uvminit(p->pagetable, initcode, sizeof(initcode));
210   p->sz = PGSIZE;
211
212   // prepare for the very first "return" from kernel to user.
213   p->tf->epc = 0;      // user program counter
214   p->tf->sp = PGSIZE;  // user stack pointer
215
216   safestrcpy(p->name, "initcode", sizeof(p->name));
217   p->cwd = namei("/");
218
219   p->state = RUNNABLE;
220
221   release(&p->lock);
222
223   // Add some code to print the proc struct of init process
224   printf("The address of init process: %p\n", p);
225   printf("State: %d\n", p->state);
226   printf("Parent: %p\n", p->parent);
227   printf("kernel stack: %lld\n", p->kstack);
228   printf("memory size: %lld\n", p->sz);
229   printf("Page table: %p\n", p->pagetable);
230   printf("Name: %s\n", p->name);
231 }
```

动态查看Proc的相关信息

- 如何编译：
  - 安装相应的实验环境，安装xv6编译工具链，安装qemu-risc-v版本(https://pdos.csail.mit.edu/6.828/2019/tools.html）
  - 下载xv6实验源码 (git clone git://github.com/mit-pdos/xv6-riscv-fall19.git)
  - 切换到源码目录后，插入打印语句，保存修改
  - 执行 make qemu，启动xv6

动态查看Proc的相关信息

打印的proc如下：



PCB实例——Xv7: proc

xv7-kernel/src/process.rs

```
0 implementations
13  pub enum ProcessState {
14      Spawn,
15      Runnable,
16      Running,
17      Zombie,
18  }
19
20  static NEXT_PID: AtomicUsize = AtomicUsize::new(0);
21
2 implementations
22  pub struct Process {
23      pub pid: usize,
24      pub state: ProcessState,
25      pub vm: AddressSpace,
26      pub context: Context,
27      pub kstack: VirtAddr,
28      pub fds: Vec<File>,
29  }
```

进程状态：
- 派生的
- 可运行的
- 正在运行
- 僵尸

进程控制块PCB：
- pid
- state进程状态
- vm虚拟地址空间
- context上下文
- kstack栈地址
- fds文件结构

成员的定义类型定义见下一页PPT

PCB实例——Xv7: proc

xv7-kernel/src/arch/x86_64/context.rs

```
3   #[derive(Debug)]
4   #[repr(C)]
5   pub struct Context {
6       pub cr3: usize,
7       pub rsp: usize,
8       pub rflags: usize,
9       pub r15: usize,
10      pub r14: usize,
11      pub r13: usize,
12      pub r12: usize,
13      pub rbp: usize,
14      pub rbx: usize,
15  }
```

上下文结构，定义了一些寄存器的状态

- cr3，保存当前进程地址空间的物理帧
- rsp栈寄存器
- rflags标志寄存器
- r12-r15通用寄存器
- rbp基址寄存器
- rbx通用寄存器

xv7-kernel/src/arch/x86_64/paging.rs

```
5   use x86_64::structures::paging::{PageTable, PageTableFlags, PhysFrame};
6   pub use x86_64::{PhysAddr, VirtAddr};
```

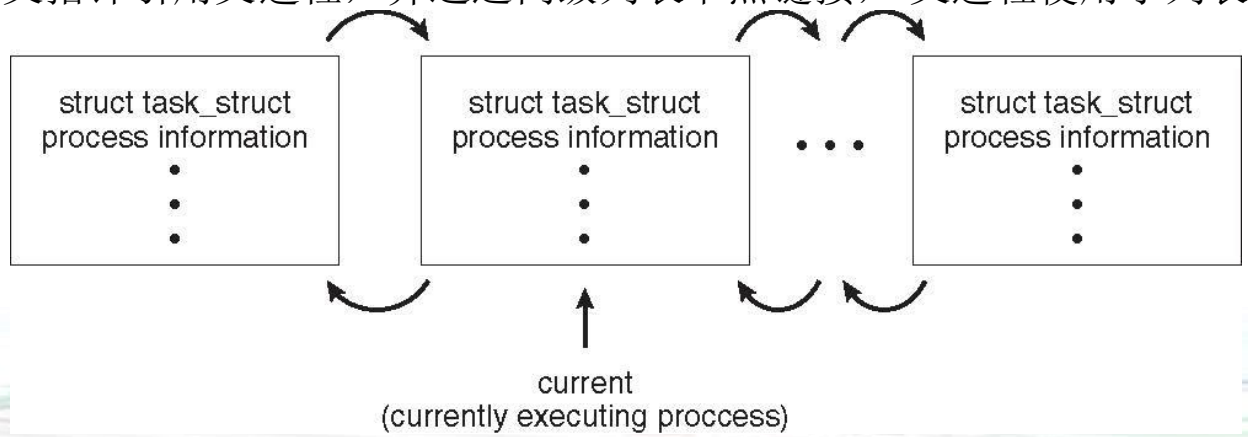
```
40  pub struct AddressSpace {
41      pub cr3: PhysFrame,
42  }
```

物理地址PhysAddr、虚拟地址VirtAddr，与CPU结构相关，引用库x86_64

地址空间结构，由cr3寄存器定义，是物理帧

PCB实例——Linux:task_struct

- Linux 内核用一个数据结构 task_struct（定义在include/linux/sched.h 中）来代表一个进程（线程）
- 对内核而言，进程或者线程都称为任务(task)
- 所有task_struct的指针构成一个task数组, 这个数组的大小默认为512,表明系统中能同时运行的进程最多为512个
- 建立新进程时, Linux为新进程分配一个task_struct结构,并将其指针保存在task数组中
- 进程中的第一个线程被称为 task group leader ， 并且所有其他线程通过thread_node链接在一起，并且包含指向它们所在进程task_struct（即task group leader 的task_struct）的指针group_leader
- 子进程通过父指针引用父进程，并通过同级列表节点链接， 父进程使用子列表头与其子进程链接



Module 2-1：进程（Process）

1. 进程的引入
2. 进程模型
3. 进程表示
   - 3.1 多进程并发执行示例-基础
   - 3.2 进程状态-基础
   - 3.3 进程控制块-基础
   - 3.4 PCB示例-进阶
   - 3.5 Linux内核小窥-进阶
4. 进程类型和特性
5. 进程控制
6. 进程调度

Linux内核小窥：task_struct 定义

```
/* linux-5.3.6/include/linux/sched.h 起始行号 631*/
struct task_struct {
1.  volatile long state;  /* -1 unrunnable, 0 runnable, >0 stopped */
2.  void *stack;
3.
4.  unsigned int flags; /* per process flags, defined below */
5.
6.  int prio, static_prio, normal_prio;
7.  unsigned int rt_priority;
8.
9.  struct mm_struct *mm, *active_mm;
10.
11.  pid_t pid;
12.
13.  struct task_struct *real_parent; /* real parent process */
14.  struct task_struct *parent; /* recipient of SIGCHLD, wait4() reports */
15.  /*
16.   * children/sibling forms the list of my natural children
17.   */
18.  struct list_head children;  /* list of my children */
19.  struct list_head sibling; /* linkage in my parent's children list */
```

Linux内核小窥：task_struct 定义

```
20.   struct task_struct *group_leader; /* threadgroup leader */
21.   cputime_t utime, stime, utimescaled, stimescaled;
22.   cputime_t gtime;
23.
24.
25.   char comm[TASK_COMM_LEN]; /* executable name excluding path
26.               - access with [gs]et_task_comm (which lock
27.                 it with task_lock())
28.               - initialized normally by setup_new_exec */
29.
30./* filesystem information */
31.   struct fs_struct *fs;
32./* open file information */
33.   struct files_struct *files;
34.   ...
35.
36.};
```

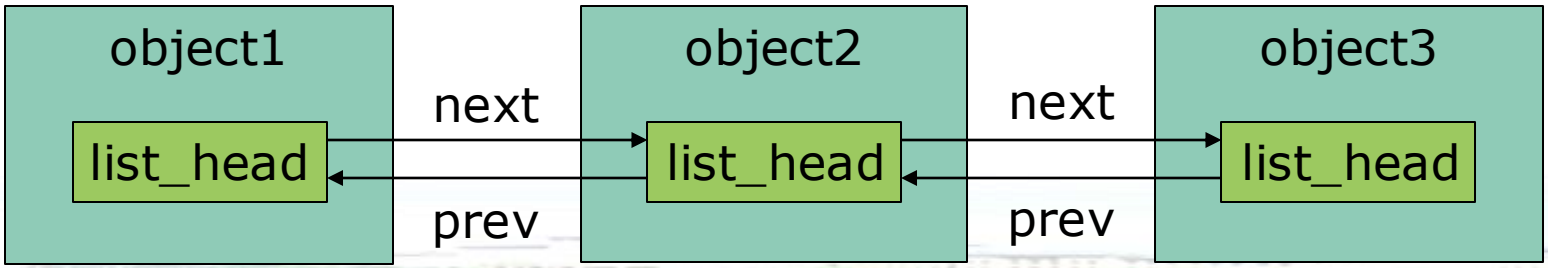
源码地址：https://github.com/torvalds/linux/blob/master/include/linux/sched.h/

Linux内核小窥: 数据结构 list_head

- Linux内核中，提供了一个用来创建双向(循环)链表的结构 list_head，其定义如下

```
/* linux-5.3.6/include/linux/list.h */
/*结构体原型*/
1.  struct list_head {
2.      struct list_head *next, *prev;
3.  };
```

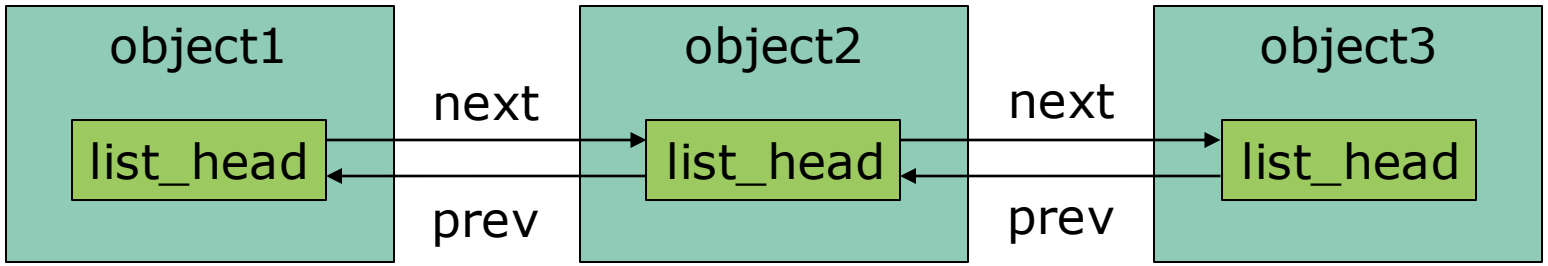
- Linux使用链表的方式与众不同，并不是通常的在数据结构中加入指针域，而是直接把结构list_head塞入数据结构中
- list_head结构体在Linux内核中被大量的引用，几乎所有内核当众需要构成链表结构的地方都用到了list_head结构体
- 为了操作方便，Linux内核还定义了一些链表操作函数，例如：list_add()可以方便的将新节点加入链表，INIT_LIST_HEAD()初始化链表



Linux内核小窥: 数据结构 list_head

```
/*通常定义双向链表的方式*/
1.  struct ListNode {
2.      int field; // 数据域
3.      struct ListNode *next, *prev;  // 指针域
4.  };
```

```
/*Linux 内核使用list_head定义双向链表的方式*/
1.  struct ListNode {
2.      int field; // 数据域
3.      struct list_head list;  // 指针域被包裹在成员变量list中
4.  };
```



Linux内核小窥: 数据结构 list_head

在Linux中可以使用list_entry（ptr, type, member）来获取整个结构体的指针

- ptr: 结构体中指向list_head类型成员的指针
- type: list_head所在结构体的类型
- member: list_head类型成员在结构体中的名字

```
/* linux-5.3.6/include/linux/list.h */
/**
 * list_entry - get the struct for this entry
 * @ptr:  the &struct list_head pointer.
 * @type: the type of the struct this is embedded in.
 * @member:       the name of the list_struct within the struct.
 */
#define list_entry(ptr, type, member) container_of(ptr, type, member)

/* linux-5.3.6/include/linux/kernel.h */
/**
 * container_of - cast a member of a structure out to the containing structure
 * @ptr:  the pointer to the member.
 * @type: the type of the container struct this is embedded in.
 * @member:       the name of the member within the struct.
 */
#define container_of(ptr, type, member) ({                                  \
        void *__mptr = (void *)(ptr);                                       \
        BUILD_BUG_ON_MSG(!__same_type(*(ptr), ((type *)0)->member) &&       \
                         !__same_type(*(ptr), void),                        \
                         "pointer type mismatch in container_of()");        \
        ((type *)(__mptr - offsetof(type, member))); })
```

list_head相关的函数


```
/* linux-5.3.6/include/linux/list.h */
/*结构体原型*/
1. struct list_head {
2.     struct list_head *next, *prev;
3. };
4.
5. #define LIST_HEAD(name)          //定义并初始化头结点head
6. #define INIT_LIST_HEAD(ptr)          //初始化头结点ptr,因此需要首先定义ptr
7. _INLINE_ void list_add(struct list_head *add, struct list_head *head)
8. //每次添加节点到head之后，始终都是添加到头结点之后
9. _INLINE_ void list_add_tail(struct list_head *add, struct list_head *head)
10.//每次添加节点都是头结点之前，由于是循环链表，就是说添加到链表尾部
11._INLINE_ void list_del(struct list_head *entry) //删除节点
12._INLINE_ void list_del_init(struct list_head *entry)
13.//删除节点，并初始化被删除的结点（也就是使被删除的结点的prev和next都指向自己）
14._INLINE_ int list_empty(struct list_head *head)//判断链表是否为空
15._INLINE_ void list_splice(struct list_head *list, struct list_head *head)
16.//通过两个链表的head，进行连接
17.#define list_entry(ptr, type, member)    //通过偏移值取type类型结构体的首地址
18.#define list_for_each(pos, head)      //遍历链表，循环内不可调用list_del()删除节点
19.#define list_for_each_safe(pos, pnext, head)  //遍历链表，可以同时有删除节点的操作
```

追踪task_struct —— Systemtap

Systemtap介绍：动态跟踪技术

什么是动态跟踪技术：

又称：动态调试，Dynamic Tracing。不影响程序的情况下，动态的观察程序运行。

常见的动态追踪技术：

- ✓ Dtrace：solaris
- ✓ SystemTap：Linux
- ✓ Perf：Linux

……

追踪task_struct —— Systemtap

- Systemtap介绍：动态跟踪技术

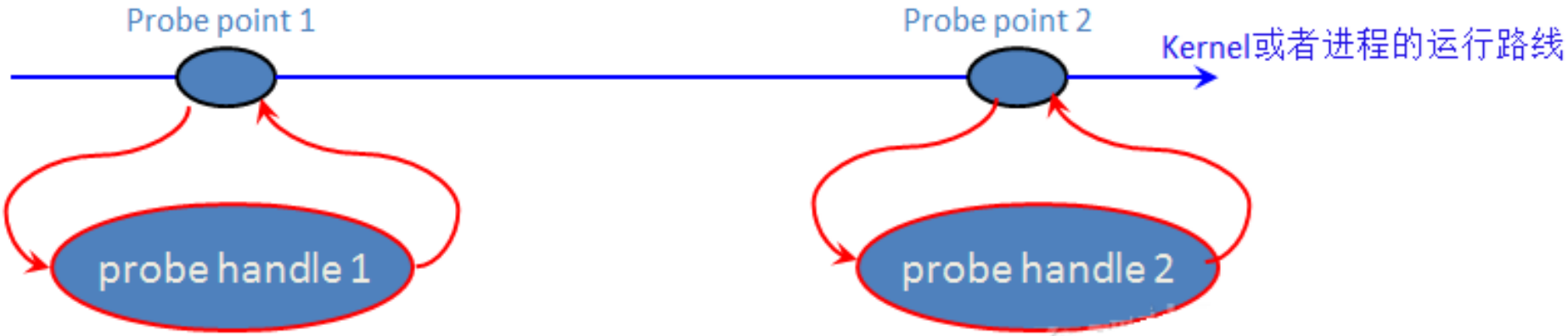
SystemTap通过systemtap script脚本语言来实现线上数据的采集和跟踪。SystemTap会根据用户写的script，使用stap工具将脚本代码转成C代码，并将其编译生成对应的内核模块，接下来将其加载到正在运行的内核中去，就可以直接从内核中提取相关数据。

SystemTap是一个Linux非常有用的调试（跟踪/探测）工具，常用于Linux 内核或者应用程序的信息采集，比如：获取一个函数里面运行时的变量、调用堆栈，甚至可以直接修改变量的值，对诊断性能或功能问题非 常有帮助。SystemTap提供非常简单的命令行接口和很简洁的脚本语 言，以及非常丰富的tapset和例子。

Systemtap 设计思想

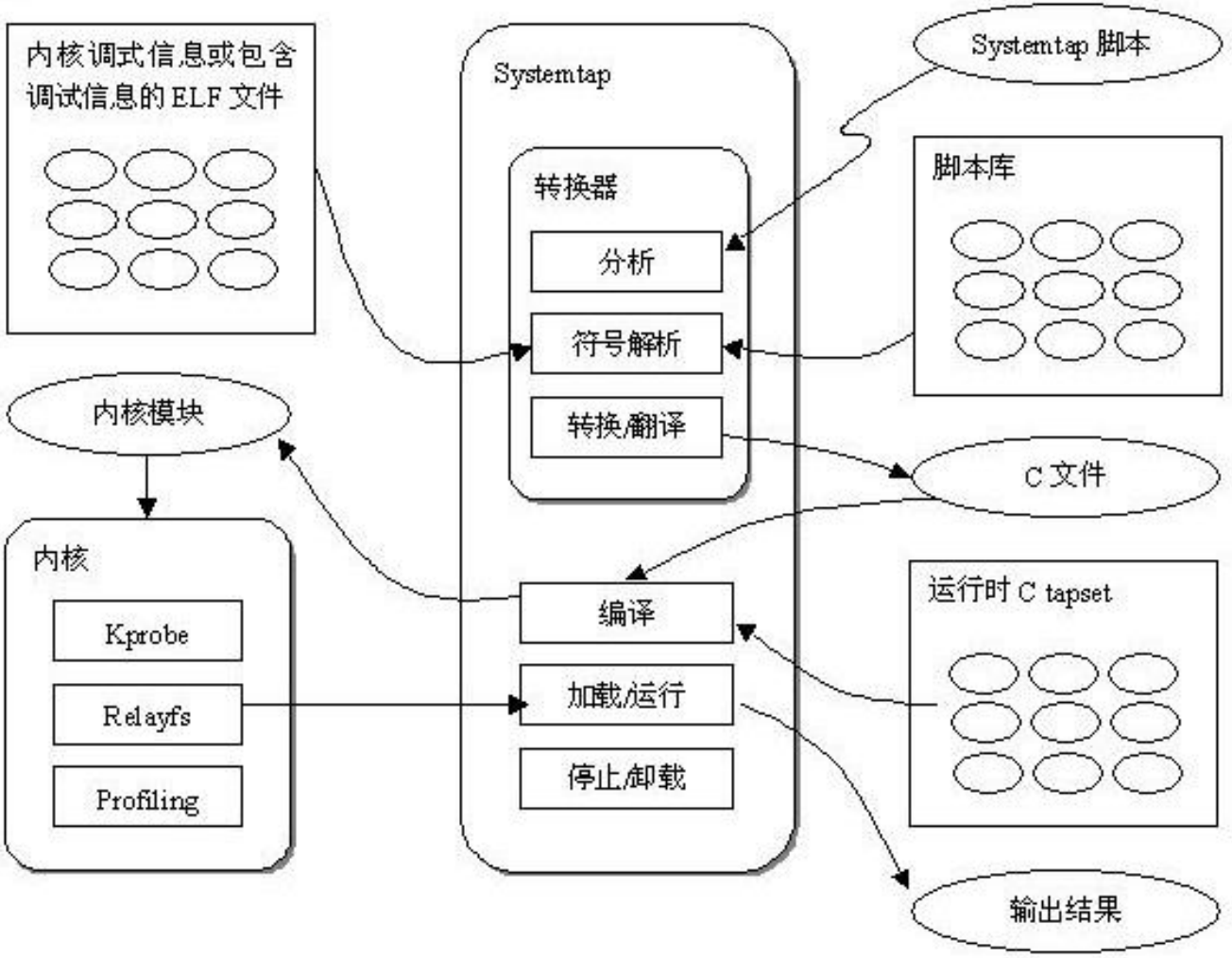
- Systemtap设计思想：**events-handlers**
  - **Probe结构：probe Event {handler}**

当运行SystemTap script的时候，SystemTap会监测对应的事件(比如函数的进入和退出、定时器超时、会话结束等)，而当事件一旦发生被捕获到了，那么对应的handler将会被作为一个子例程被内核快速执行(这个子例程通常是在当前上下文中提取感兴趣的数据，并将他们保存到内部变量中，通常还会执行打印显示操作)，接下来执行流程恢复正常。

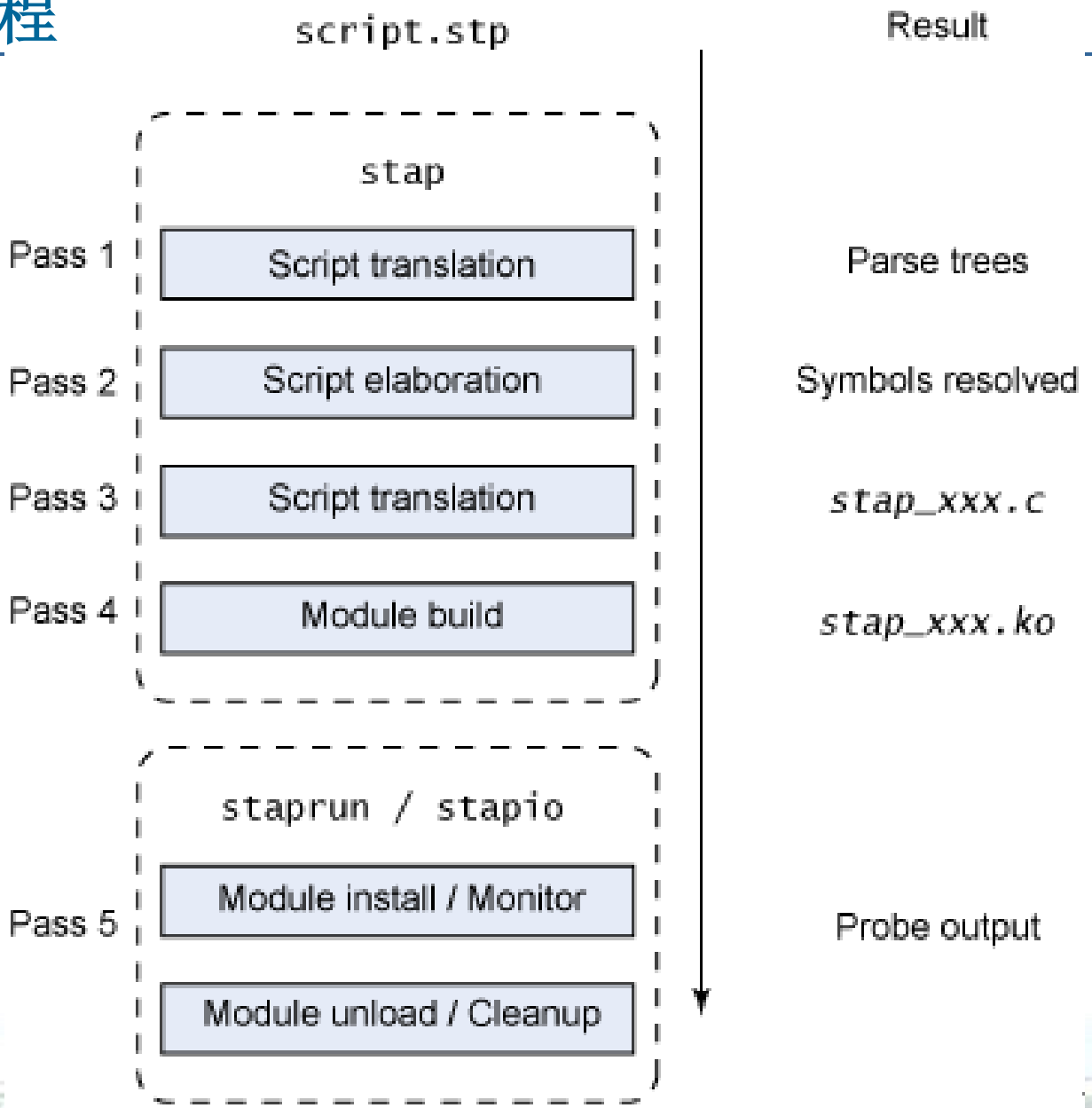


Systemtap 整体架构

SystemTap的工作机制是通过将用户编写的脚本程序转化成C代码，将C代码编译成内核模块，挂载到系统内部的方式来进行数据的获取。

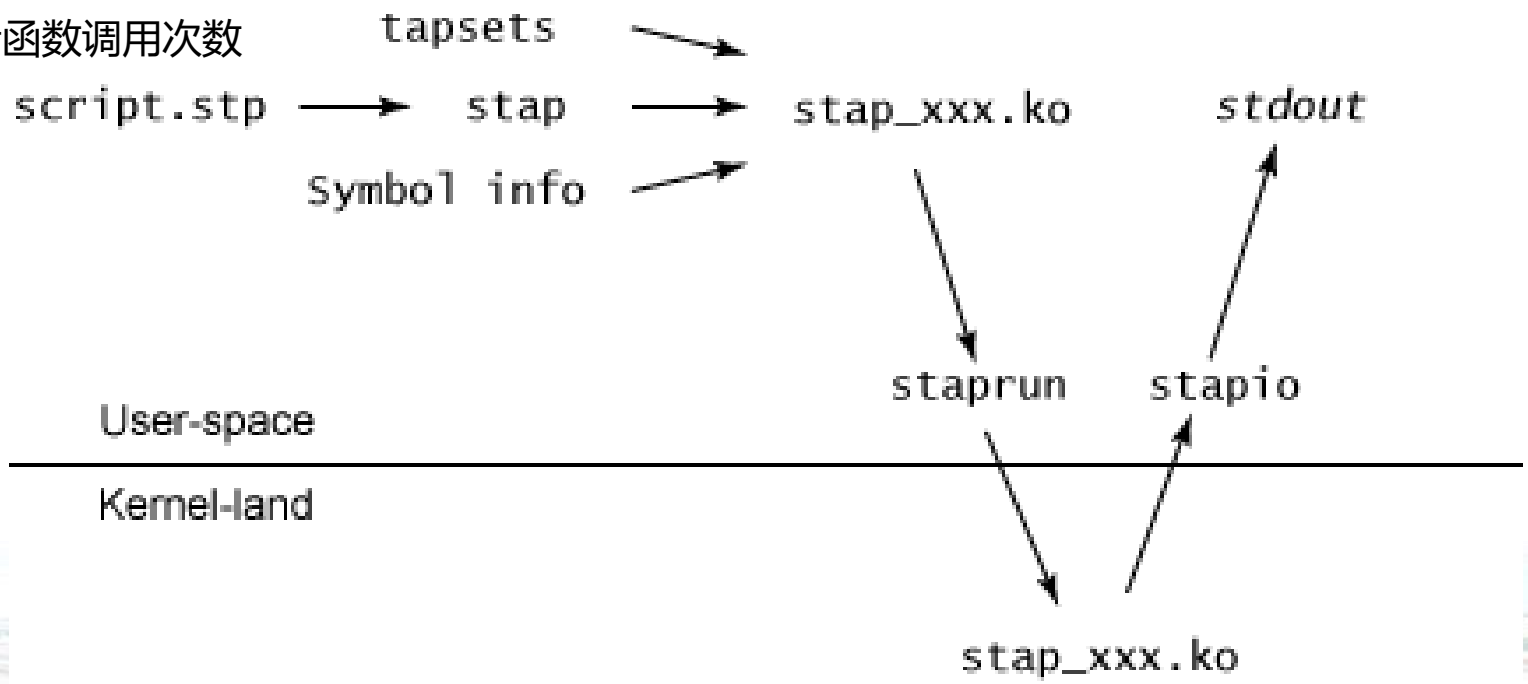


Systemtap 流程



Systemtap 使用

- 何时使用：
  - ✓ 定位（内核）函数位置
  - ✓ 查看函数调用时的调用堆栈、局部变量、参数
  - ✓ 查看函数指针变量实际指的是哪个函数
  - ✓ 查看代码的执行轨迹（哪些行被执行了）
  - ✓ 查看内核或者进程的执行流程
  - ✓ 调试内存泄漏或者内存重复释放
  - ✓ 统计函数调用次数
  - ✓ ……



SystemTap使用示例

```
[root@localhost arch_lab]# cat hello.stp
#!/usr/bin/stap
probe begin
{
        printf("Hello, World\n");exit();
}
```

/* 定义解释器，类似Shell脚本 */

/* 探针进入函数，该探针中event事件为begin，event事件触发后，执行{}中的语句，打印hello world */

```
[root@localhost arch_lab]# stap -v hello.stp
Pass 1: parsed user script and 477 library scripts using 236048virt/48956
Pass 2: analyzed script: 1 probe, 1 function, 0 embeds, 0 globals using 2
Pass 3: using cached /root/.systemtap/cache/0e/stap_0e860a510e4bc2a855631
Pass 4: using cached /root/.systemtap/cache/0e/stap_0e860a510e4bc2a855631
Pass 5: starting run.
Hello, World
Pass 5: run completed in 0usr/130sys/507real ms.
[root@localhost arch_lab]#
```

SystemTap探针类型

探针类型	说明
begin	在脚本开始时触发
end	在脚本结束时触发
kernel.function("sys_sync")	调用sys_sync时触发
kernel.function("sys_sync").call	同上
kernel.function("sys_sync").return	返回sys_sync时触发
kernel.syscall.*	进行任何系统调用时触发
kernel.function("*@kernel/fork.c:934")	到达 fork.c的第 934行时触发
module("ext3").function("ext3_file_write")	调用 ext3write函数时触发
timer.jiffies(1000)	每隔 1000个内核 jiffy触发一次
timer.ms(200).randomize(50)	每隔 200毫秒触发一次，带有线性分布的随机附加时间（-50到+50）

SystemTap 基本语句

语句	说明
if (exp) {} else {}	标准的if-then-else语句
for (exp1 ; exp2 ; exp3) {}	一个for循环
while (exp) {}	标准的while循环
do {} while (exp)	一个do-while循环
break	退出迭代
continue	继续迭代
next	从探针返回
return	从函数返回一个表达式
foreach (VAR in ARRAY) {}	迭代一个数组，将当前的键分配给VAR

91

[1] https://sourceware.org/systemtap/SystemTap_Beginners_Guide/

[2] https://sourceware.org/systemtap/langref.pdf

[3]https://sourceware.org/systemtap/examples/keyword-index.html#DISK

使用SystemTap监测进程

- SystemTap使用探针的机制来监测系统中发生的各种事件
- 通过编写脚本文件可以完成一系列对系统的追踪分析任务
- 脚本文件是由多个探针组成的，每个探针都会有相应的触发事件(可自定义选择)
- 探针的形式如下：

probe event1,event2... { Statements }

脚本运行以后，当系统中发生某个探针对应的触发事件时右面大括号中的语句序列就会被执行(类似于C语法，但有所区别，具体详见参考资料)

- 脚本中可以自定义函数，形式如下：

```
function Func:type1(task:type2) {
        //Statements
}
```

  - type1是函数返回值，type2是函数参数，Statements是函数体
  - 通过自定义函数可以完成一些定制任务

参考：

https://sourceware.org/systemtap/documentation.html
https://sourceware.org/systemtap/langref/

使用SystemTap监测进程

- 获取list_head链表的下一个节点：

```
function get_the_next_task:long(task:long) %{
    struct task_struct *p = (struct task_struct *)STAP_ARG_task;
......
    the_next_task = list_entry(p->sibling.next, struct task_struct, sibling);
......
    STAP_RETVALUE = (long)the_next_task;
%}
```

- 获取task_struct字段信息

```
the_state = @cast(task, "task_struct", "kernel<linux/sched.h>")->state;
the_flags = @cast(task, "task_struct", "kernel<linux/sched.h>")->flags;
```

- @cast()运算符告诉翻译器如何读取指针

参考：
https://sourceware.org/systemtap/documentation.html
https://sourceware.org/systemtap/langref/

使用SystemTap监测进程

```
**********************************************************
The relation is:swapper/0=>systemd=>gdm=>X
The parent is:gdm@0xffff8c22a89241c0
The parent details:
        pid:1256
        state:1
        flags:40402100
        stackAddr:0xffff8c22a8930000
        group_leader:0xffff8c22a89241c0
        parent:0xffff8c22ba630000
        fs:0xffff8c22a885a040
        mm:0xffff8c22b90bea40
The current task name is:X
The siblings:
Sibling1:gdm-session-wor
        The PID is:2180
        Parent:gdm
        Parent addr:0xffff8c22a89241c0
**********************************************************
```

进程创建关系

父进程字段信息

兄弟节点(进程)信息

```
struct task_struct {
......
    volatile long state;    /* -1 unrunnable, 0 runnable, >0 stopped */
    unsigned int flags;     /* per process flags, defined below */
    void *stack;
    struct task_struct *group_leader;/* threadgroup leader */
    struct task_struct __rcu *parent; /* recipient of SIGCHLD, wait4() reports */
    struct fs_struct *fs; /* filesystem information */
    struct mm_struct *mm, *active_mm;
......
}
```

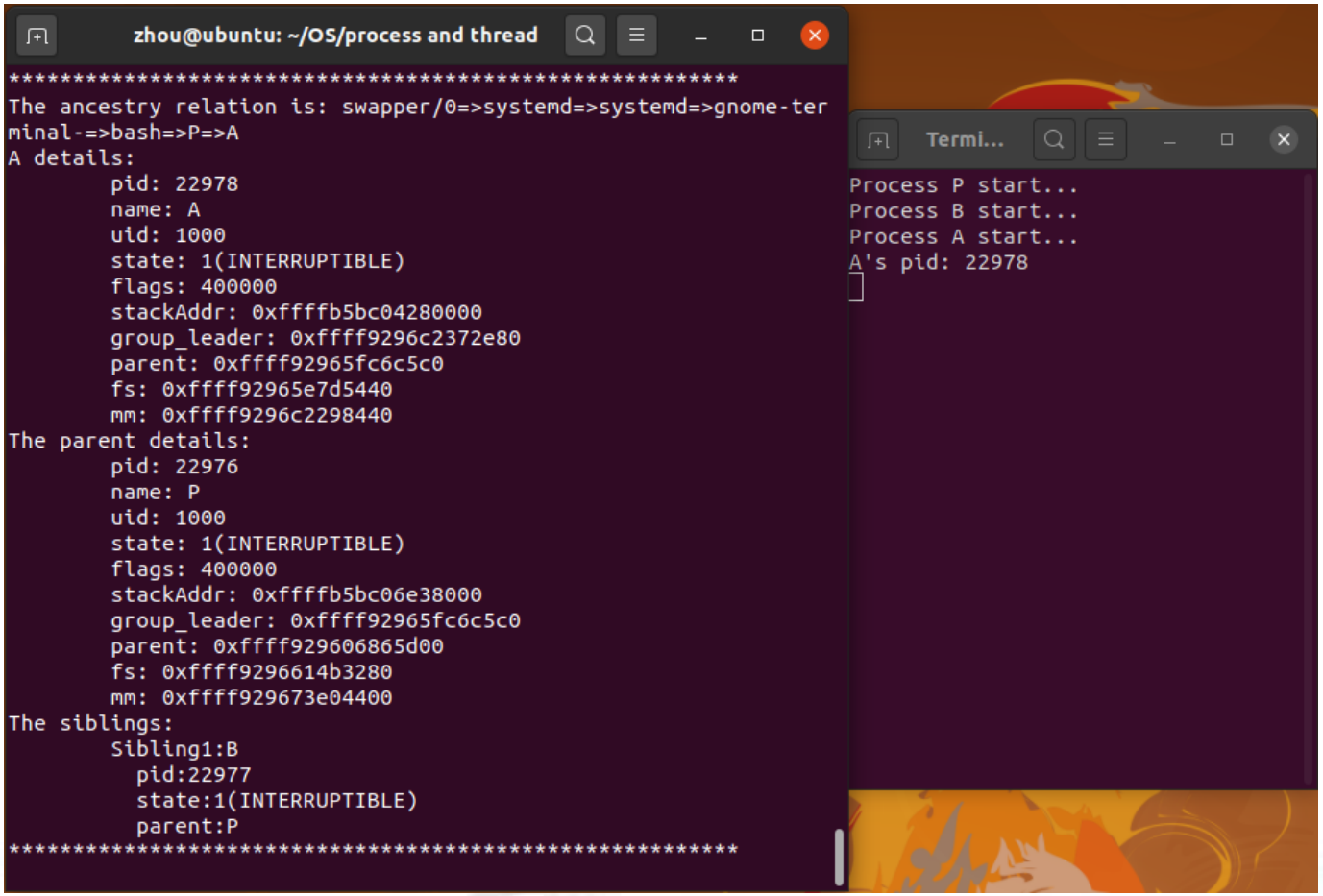
```
/*
 * Per process flags
 */
#define PF_EXITING  0x00000004  /* getting shut down */
#define PF_EXITPIDONE   0x00000008  /* pi exit done on shut down */
#define PF_VCPU     0x00000010  /* I'm a virtual CPU */
#define PF_WQ_WORKER    0x00000020  /* I'm a workqueue worker */
#define PF_FORKNOEXEC   0x00000040  /* forked but didn't exec */
#define PF_MCE_PROCESS  0x00000080      /* process policy on mce errors */
#define PF_SUPERPRIV    0x00000100  /* used super-user privileges */
#define PF_DUMPCORE 0x00000200  /* dumped core */
#define PF_SIGNALED 0x00000400  /* killed by a signal */
#define PF_MEMALLOC 0x00000800  /* Allocating memory */
#define PF_NPROC_EXCEEDED 0x00001000    /* set_user noticed that RLIMIT_NPROC was exceeded */
#define PF_USED_MATH    0x00002000  /* if unset the fpu must be initialized before use */
#define PF_USED_ASYNC   0x00004000  /* used async_schedule*(), used by module init */
#define PF_NOFREEZE 0x00008000  /* this thread should not be frozen */
#define PF_FROZEN   0x00010000  /* frozen for system suspend */
#define PF_FSTRANS  0x00020000  /* inside a filesystem transaction */
#define PF_KSWAPD   0x00040000  /* I am kswapd */
#define PF_MEMALLOC_NOIO 0x00080000 /* Allocating memory without IO involved */
#define PF_LESS_THROTTLE 0x00100000 /* Throttle me less: I clean memory */
#define PF_KTHREAD  0x00200000  /* I am a kernel thread */
#define PF_RANDOMIZE    0x00400000  /* randomize virtual address space */
#define PF_SWAPWRITE    0x00800000  /* Allowed to write to swap */
#define PF_NO_SETAFFINITY 0x04000000    /* Userland is not allowed to meddle with cpus_allowed */
#define PF_MCE_EARLY    0x08000000      /* Early kill for mce process policy */
#define PF_MEMPOLICY    0x10000000  /* Non-default NUMA mempolicy */
#define PF_MUTEX_TESTER 0x20000000  /* Thread belongs to the rt mutex tester */
#define PF_FREEZER_SKIP 0x40000000  /* Freezer should not count it as freezable */
#define PF_SUSPEND_TASK 0x80000000      /* this thread called freeze_processes and should not be frozen */
```

Practice：追踪task_struct

- SystemTap提供了一个简单的脚本语言，用于为实时运行的内核和用户空间应用程序编写检测脚本。
- 现有一个正在运行的进程A及其父进程P和兄弟进程B，请根据进程A的pid，利用SystemTap提供的脚本语言及脚本库“tapset”，完成以下任务：
  - 找到进程A的所有祖先进程，并按顺序输出其进程名；
  - 输出进程A的task_struct中的各个字段；
  - 通过A的task_struct找到其父进程P的task_struct，并输出各个字段；
  - 通过sibling链表找到兄弟进程B，并输出进程B的pid、状态及父进程名。

追踪 task_struct——SystemTap

- **Demo：**



Summary: 进程表示

- 进程的状态： 新建态、就绪态、运行态、阻塞态、终止态
- 进程控制块对进程管理和控制的作用
- PCB的内容
- Linux 中的 task_struct 结构
- 追踪task_struct——SystemTap

Module 2-1：进程（Process）

1. 进程的引入
2. 进程模型
3. 进程表示
4. 进程类型和特性
5. 进程控制
6. 进程调度

进程类型

- 分为系统进程和用户进程

Mac系统的进程类型

系统进程

用户进程

linjie — top — 80×24

```
Processes: 513 total, 2 running, 511 sleeping, 4809 threads            18:07:09
Load Avg: 2.17, 2.48, 2.68  CPU usage: 14.62% user, 11.65% sys, 73.71% idle
SharedLibs: 168M resident, 38M data, 9392K linkedit.
MemRegions: 1408184 total, 1170M resident, 64M private, 1385M shared.
PhysMem: 7580M used (1924M wired, 2946M compressor), 48M unused.
VM: 322T vsize, 4272M framework vsize, 6396101(40) swapins, 7150770(0) swapouts.
Networks: packets: 12468065/4381M in, 12149434/2671M out.
Disks: 26425794/609G read, 6847601/178G written.

PID     COMMAND          %CPU  MEM     PPID   UID   USER
41945   Google Chrome He 62.8  196M-   492    501   linjie
0       kernel_task      27.5  55M     0      0     root
149     WindowServer     26.6  1537M-  1      88    _windowserver
511     EpicGamesLaunche 13.4  467M    1      501   linjie
18168   top              8.3   6785K+  18114  0     root
10702   sysmond          7.7   2337K   1      0     root
480     youduqt          7.2   693M    1      501   linjie
11791   Activity Monitor 5.1   64M     1      501   linjie
296     TouchBarServer   4.2   32M+    1      0     root
690     Code Helper (GPU 3.3   411M-   503    501   linjie
507     Terminal         2.7   108M+   1      501   linjie
456     Microsoft PowerP 2.5   926M    1      501   linjie
518     VMware Fusion    2.0   233M    1      501   linjie
84768   vmware-vmx       2.0   2874M   1      0     root
```

进程类型

- 分为系统进程和用户进程

Ubuntu系统的进程类型

用户进程

系统进程

```
top - 07:32:35 up 0 min,  1 user,  load average: 1.29, 0.46, 0.16
Tasks: 352 total,   1 running, 351 sleeping,   0 stopped,   0 zombie
%Cpu(s):  3.0 us,  1.3 sy,  0.0 ni, 95.7 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3920.0 total,   2317.0 free,    939.1 used,    663.9 buff/cache
MiB Swap:   3920.0 total,   3920.0 free,      0.0 used.   2810.4 avail Mem

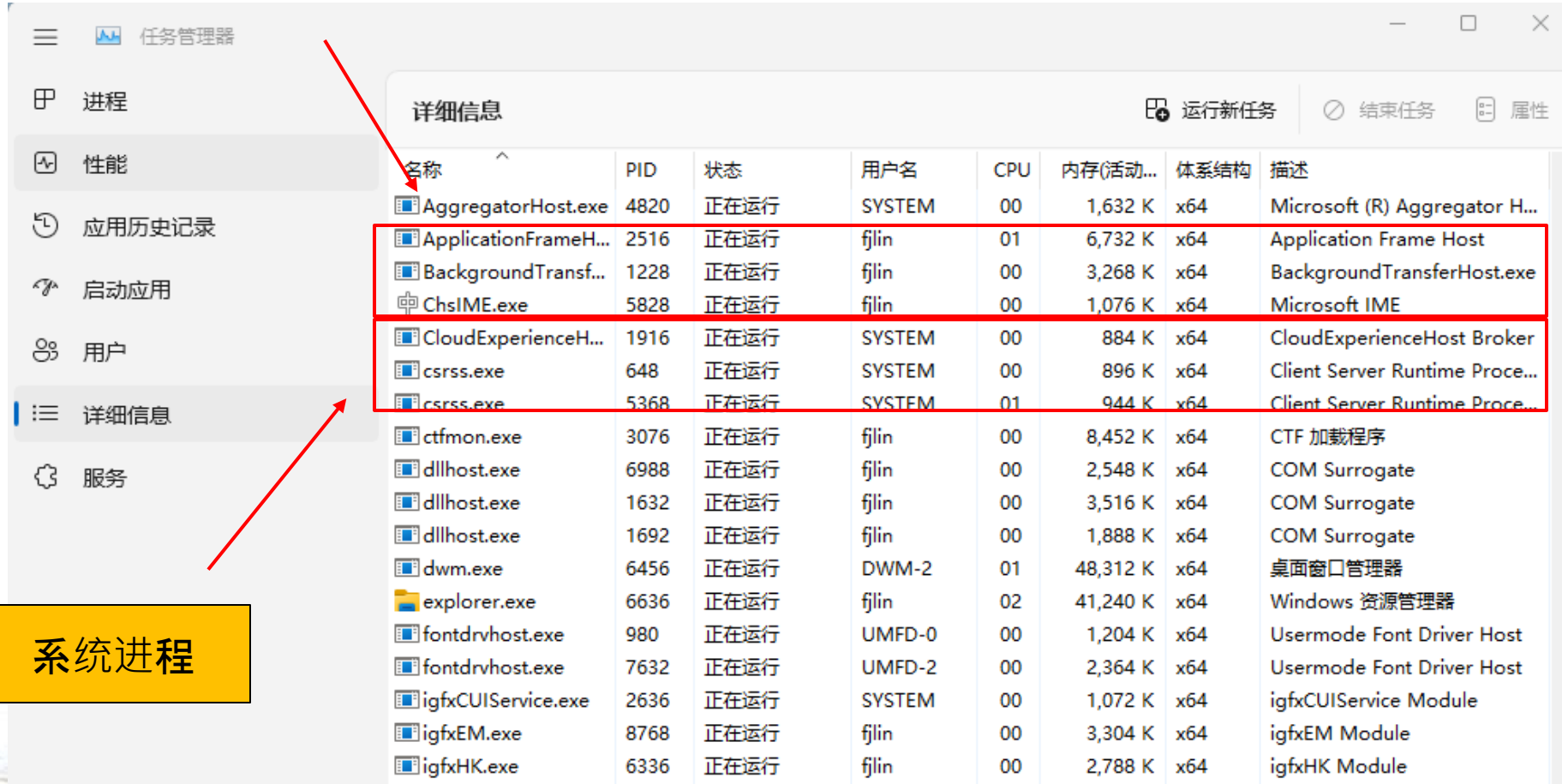
    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   1831 jie       20   0 3929480 361924 116088 S   3.0   9.0   0:05.20 gnome-shell
   1695 root      20   0  235088  67524  41128 S   1.3   1.7   0:00.97 Xorg
   2156 jie       20   0  809132  47436  36132 S   0.7   1.2   0:00.29 gnome-terminal-
   2260 jie       20   0    9196   3432   2592 R   0.7   0.1   0:00.15 top
   1942 jie       20   0  565760  27640  17580 S   0.3   0.7   0:00.13 gsd-color
      1 root      20   0  101696  10436   7208 S   0.0   0.3   0:01.01 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
      3 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 rcu_gp
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 rcu_par_gp
      5 root      20   0       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0-cgroup_destroy
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
      7 root      20   0       0      0      0 I   0.0   0.0   0:00.00 kworker/u4:0-events_freeza+
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 mm_percpu_wq
      9 root      20   0       0      0      0 S   0.0   0.0   0:00.04 ksoftirqd/0
     10 root      20   0       0      0      0 I   0.0   0.0   0:00.10 rcu_sched
     11 root      rt   0       0      0      0 S   0.0   0.0   0:00.00 migration/0
     12 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
```

进程类型

- 分为系统进程和用户进程

用户进程

Windows系统的进程类型



名称	PID	状态	用户名	CPU	内存(活动...	体系结构	描述
AggregatorHost.exe	4820	正在运行	SYSTEM	00	1,632 K	x64	Microsoft (R) Aggregator H...
ApplicationFrameH...	2516	正在运行	fjlin	01	6,732 K	x64	Application Frame Host
BackgroundTransf...	1228	正在运行	fjlin	00	3,268 K	x64	BackgroundTransferHost.exe
ChsIME.exe	5828	正在运行	fjlin	00	1,076 K	x64	Microsoft IME
CloudExperienceH...	1916	正在运行	SYSTEM	00	884 K	x64	CloudExperienceHost Broker
csrss.exe	648	正在运行	SYSTEM	00	896 K	x64	Client Server Runtime Proce...
csrss.exe	5368	正在运行	SYSTEM	01	944 K	x64	Client Server Runtime Proce...
ctfmon.exe	3076	正在运行	fjlin	00	8,452 K	x64	CTF 加载程序
dllhost.exe	6988	正在运行	fjlin	00	2,548 K	x64	COM Surrogate
dllhost.exe	1632	正在运行	fjlin	00	3,516 K	x64	COM Surrogate
dllhost.exe	1692	正在运行	fjlin	00	1,888 K	x64	COM Surrogate
dwm.exe	6456	正在运行	DWM-2	01	48,312 K	x64	桌面窗口管理器
explorer.exe	6636	正在运行	fjlin	02	41,240 K	x64	Windows 资源管理器
fontdrvhost.exe	980	正在运行	UMFD-0	00	1,204 K	x64	Usermode Font Driver Host
fontdrvhost.exe	7632	正在运行	UMFD-2	00	2,364 K	x64	Usermode Font Driver Host
igfxCUIService.exe	2636	正在运行	SYSTEM	00	1,072 K	x64	igfxCUIService Module
igfxEM.exe	8768	正在运行	fjlin	00	3,304 K	x64	igfxEM Module
igfxHK.exe	6336	正在运行	fjlin	00	2,788 K	x64	igfxHK Module

系统进程

进程的特征

进程的5个基本特征：

（1）动态性　动态特性表现在它因创建而产生，由调度而执行，因得不到资源而暂停执行，最后因完成或撤销而消亡（进程生命周期）

（2）并发性　引入进程的目的就是为了使多个程序并发执行，以提高资源利用率（主要CPU）

（3）独立性　进程是一个能独立运行的基本单位，也是系统进行资源分配和调度的基本单位

（4）异步性　进程以各自独立的、不可预知的速度向前推进

（5）交往性（交互）进程在运行过程中可能会与其它进程发生直接或间接的相互作用

Module 2-1：进程（Process）

1. 进程的引入
2. 进程模型
3. 进程表示
4. 进程类型和特性
5. 进程控制
6. 进程调度

获取进程ID（ Obtaining Process IDs ）

- Each process has a unique positive(nonzero) process *ID*(*PID*)

- `pid_t getpid(void)`
  - Returns PID of current process(返回当前进程的PID)

- `pid_t getppid(void)`
  - Returns PID of parent process(返回父进程的PID)

终止进程（Terminating Processes）

- Process becomes terminated for one of three reasons:
  - Receiving a signal whose default action is to terminate (信号参见CSAPP 8.5节)
  - Returning from the `main` routine
  - Calling the `exit` function

- `void exit(int status)`
  - Terminates with an *exit status* of `status`
  - Convention: normal return status is 0, nonzero on error
  - Another way to explicitly set the exit status is to return an integer value from the main routine(另一种显式设置退出状态的方法是从main routine返回一个整数值)

- `exit` is called once but never returns.

创建进程（ Creating Processes ）

- *Parent process*（父进程）creates a new running *child process*（子进程）by calling **`fork`** function
- `int fork(void)`
  - Returns: 0 to the child process, child's PID to parent process, -1 on error
  - Child is *almost* identical to parent:
    - Child get an identical (but separate) copy of the parent's virtual address space.（子进程得到与父进程相同但独立的虚拟地址空间的副本）
    - Child gets identical copies of the parent's open file descriptors(打开文件描述符）
    - Child has a different PID than the parent
- `Fork`: called *once* but returns *twice*
  - Calling process (parent): returns the PID of the child
  - Newly child process: returns a value of 0

fork实例

fork1.c

```
int main()
{
   pid_t pid;
   int x = 1;

   if ((pid = fork())<0){
            fprintf(stderr,“%s: %s\n”, “fork Error!”, strerror(errno));

   if (pid == 0) {  /* Child */
      printf("child : x=%d\n", ++x);
      exit(0);
   }

   /* Parent */
   printf("parent: x=%d\n", --x);
   exit(0);
}
```

- **并发执行**
  - 不能预测父子进程的执行顺序
- **相同但分离的地址空间**
  - x has a value of 1 when fork returns in parent and child
  - Subsequent changes to x are independen

```
linux> ./fork1
parent: x=0
child : x=2
```

Modeling fork with Process Graphs（进程图）

- A ***process graph***（进程图） is a useful tool for capturing the partial ordering of statements in a concurrent program:（进程图是刻画并发程序语句偏序关系的一种有效工具）
  - Each vertex is the execution of a statement
  - a -> b means `a` happens before b
  - Edges can be labeled with current value of variables
  - `printf` vertices can be labeled with output
  - Each graph begins with a vertex with no inedges（入边）

- Any ***topological sort***（拓扑排序） of the graph corresponds to a feasible total ordering.（图中的任何一个拓扑排序对应于一个可行的全序排列）
  - Total ordering of vertices where all edges point from left to right

进程图示例（ Process Graph Example ）

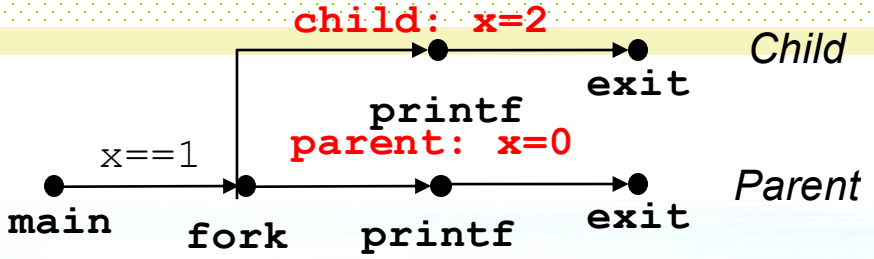
fork1.c

```
int main()
{
   pid_t pid;
   int x = 1;

   if ((pid = fork())<0){
            fprintf(stderr,"%s: %s\n", "fork Error!", strerror(errno));

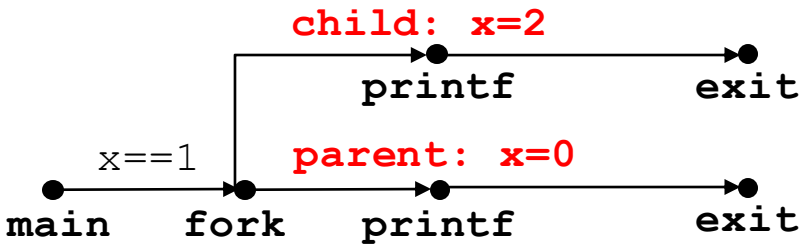
   if (pid == 0) {  /* Child */
     printf("child : x=%d\n", ++x);
     exit(0);
   }

   /* Parent */
   printf("parent: x=%d\n", --x);
   exit(0);
}
```

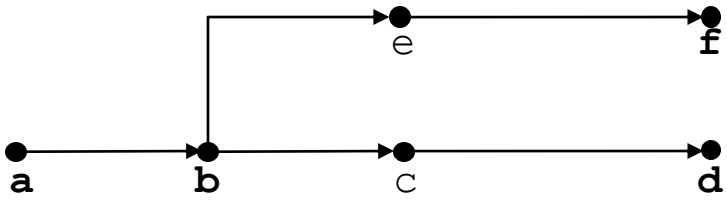


进程图解释(Interpreting Process Graphs)

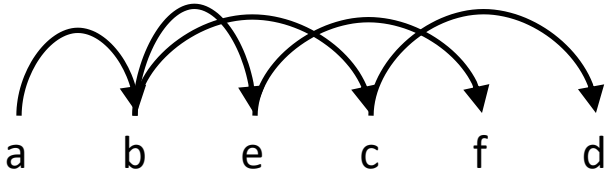
- Original graph:



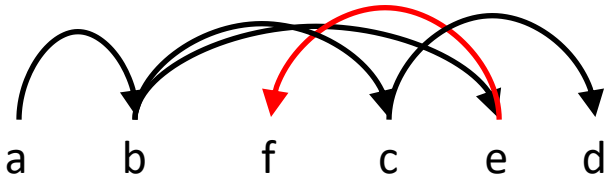
- Relabled graph:



Feasible total ordering:

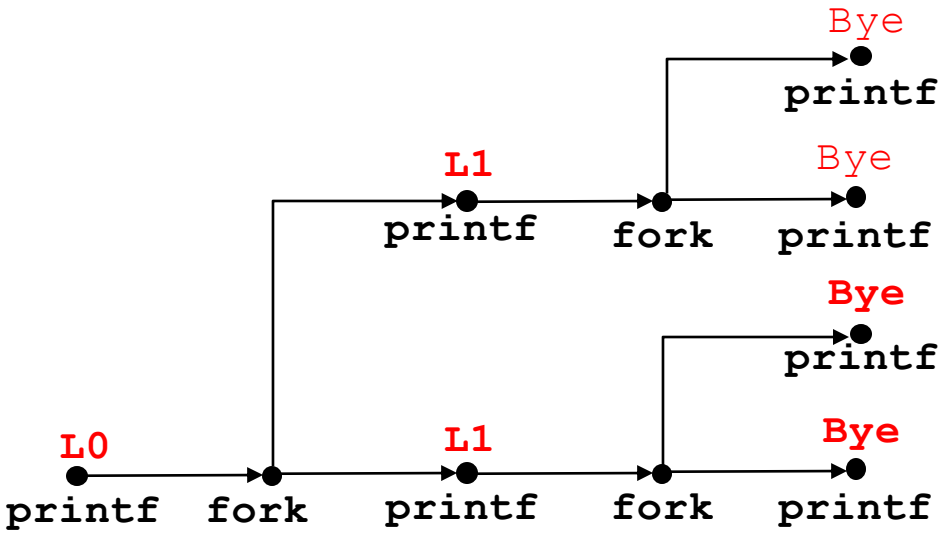


Infeasible total ordering:



fork实例: 两个连续fork调用

```
void fork2()                    fork2.c
{
    printf("L0\n");
    fork();
    printf("L1\n");
    fork();
    printf("Bye\n");
}
```



Feasible output:

L0
L1
Bye
Bye
L1
Bye
Bye

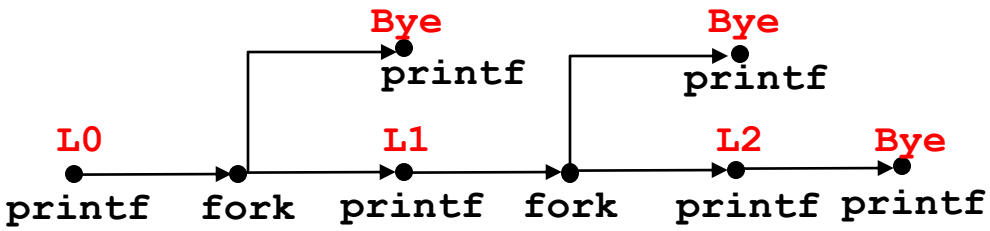
Infeasible output:

L0
Bye
L1
Bye
L1
Bye
Bye

fork实例: 父进程的嵌套fork调用

fork3.c

```
void fork3()
{
    printf("L0\n");
    if (fork() != 0) {
        printf("L1\n");
        if (fork() != 0) {
            printf("L2\n");
                }
    }
    printf("Bye\n");
}
```



Feasible output:

L0
L1
Bye
Bye
L2
Bye

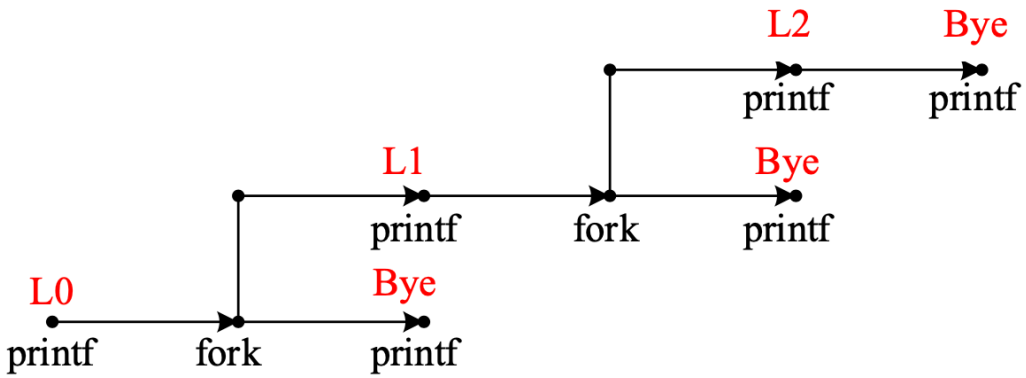
Infeasible output:

L0
Bye
L1
Bye
Bye
L2

进程图

【例题】根据进程图，以下哪个输出是不正确的( )。

A. L0, Bye, L1, Bye, L2, Bye
B. L0, L1, Bye, L2, Bye, Bye
C. L0, Bye, L1, L2, Bye, Bye
D. L0, Bye, L1, Bye, Bye, L2



答案：D
L2之后肯定有一个Bye输出

为什么这样设计API

- 进程的创建操作API为什么如此怪异？
  - fork() 调用1次，返回2次
  - fork()和exec() 组合完成一个新的进程的完整创建过程
  - 为什么不像Windows提供的 CreateProcess()函数一样，完成进程创建
- 原因
  - 调用fork() 之后创建的子进程和父进程保持一致
    - 实现父进程与子进程的通信

      如，父进程提前为子进程设置某些参数
    - 利用这种一致性可以减少创建开销、节省空间:

      在实际任务中，子进程需要和父进程保持相同，至少部分相同
  - fork()和exec()分离后，系统可以在exec()之前做更多的事情，让shell方便地实现很多有用的功能
    - shell可以利用文件描述符实现重定向
    - shell可以利用pipe()系统调用实现管道

为什么这样设计API：重定向示例


```
1.int main(int argc, char *argv[])
2.{
3.    int rc = fork();
4.    if (rc < 0) {
5.        // fork failed; exit
6.        fprintf(stderr, "fork failed\n");
7.        exit(1);
8.    } else if (rc == 0) {
9.    // child: redirect standard output to a file
10.     close(STDOUT_FILENO);
11.     open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC, S_IRWXU);
12.
13.     // now exec "wc"...
14.         char *myargs[3];
15.         myargs[0] = strdup("wc");   // program: "wc" (word count)
16.         myargs[1] = strdup("p4.c"); // argument: file to count
17.         myargs[2] = NULL;           // marks end of array
```

```
→  cpu-api git:(master) ✗ ls
Makefile  README.md  p1.c  p2.c  p3.c  p4  p4.c
→  cpu-api git:(master) ✗ ./p4
→  cpu-api git:(master) ✗ ls
Makefile  README.md  p1.c  p2.c  p3.c  p4  p4.c  p4.output
→  cpu-api git:(master) ✗ cat p4.output
 34 114 884 p4.c
→  cpu-api git:(master) ✗
```

回收子进程 （Reaping Child Processes）

- 基本想法
  - When process terminates, it still consumes（消耗）system resources
    - Examples: Exit status, various OS tables
  - Called a “zombie”（称为“僵尸进程”）
    - Living corpse（尸体）, half alive and half dead
- **Reaping（回收）**
  - Performed by parent on terminated child (using `wait` or `waitpid`)
  - Parent is given exit status information
  - Kernel then deletes zombie child process
- What if parent doesn’t reap?
  - If any parent terminates without reaping a child, then the **orphaned child**（孤儿进程）will be reaped by **`init`** process (pid == 1)
  - So, only need explicit reaping（显式回收）in long-running processes
    - e.g., shells and servers（shell 和服务器）

僵尸（zombie）进程实例

fork4.c

```
void fork4() {
   if (fork() == 0) {
      /* Child */
      printf("Terminating Child, PID = %d\n", getpid());
      exit(0);
   } else {
      printf("Running Parent, PID = %d\n", getpid());
      while (1)
         ; /* Infinite loop */
   }
}
```

```
linux> ./fork4  &
[1] 6639
Running Parent, PID = 6639
Terminating Child, PID = 6640
linux> ps
  PID TTY          TIME CMD
 6585 ttyp9    00:00:00 tcsh
 6639 ttyp9    00:00:03 forks
 6640 ttyp9    00:00:00 forks <defunct>
 6641 ttyp9    00:00:00 ps
linux> kill 6639
[1]    Terminated
linux> ps
  PID TTY          TIME CMD
 6585 ttyp9    00:00:00 tcsh
 6642 ttyp9    00:00:00 ps
```

- `ps` shows child process as “defunct” (i.e., a zombie)
- Killing parent allows child to be reaped by `init`

非终止（Non-terminating）子进程示例

fork5.c

```
void fork5()
{
   if (fork() == 0) {
      /* Child */
      printf("Running Child, PID = %d\n",
           getpid());
      while (1)
        ; /* Infinite loop */
   } else {
      printf("Terminating Parent, PID = %d\n",
           getpid());
      exit(0);
   }
}
```

```
linux> ./fork5 8
Terminating Parent, PID = 6675
Running Child, PID = 6676
linux> ps
  PID TTY          TIME CMD
 6585 ttyp9    00:00:00 tcsh
 6676 ttyp9    00:00:06 forks
 6677 ttyp9    00:00:00 ps
linux> kill 6676
linux> ps
  PID TTY          TIME CMD
 6585 ttyp9    00:00:00 tcsh
 6678 ttyp9    00:00:00 ps
```

- Child process still active even though parent has terminated
- Must kill child explicitly, or else will keep running indefinitely（必须显式杀死子进程，否则它将会永远运行）

wait: 和子进程同步（ Synchronizing ）

- Parent reaps a child by calling the **`wait`** function

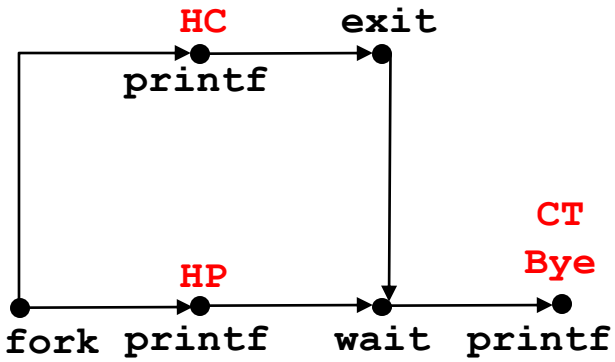
- `int wait(int *child_status)`
  - Suspends(挂起) current process until one of its children terminates
  - Return value is the **`pid`** of the child process that terminated
  - If **`child_status != NULL`**, then the integer it points to will be set to a value that indicates reason the child terminated and the exit status:

    (该值显示子进程被终止的原因及退出状态)
    - Checked using macros(宏) defined in `wait.h`
      - `WIFEXITED, WEXITSTATUS, WIFSIGNALED, WTERMSIG, WIFSTOPPED, WSTOPSIG, WIFCONTINUED`
      - See textbook for details (CSAPP 3e : 8.4.3)

wait: 和子进程同步（ Synchronizing ）

```
void fork6() {                                          wait1.c
   int child_status;

   if (fork() == 0) {
      printf("HC: hello from child\n");
               exit(0);
   } else {
      printf("HP: hello from parent\n");
      wait(&child_status);
      printf("CT: child has terminated\n");
   }
   printf("Bye\n");
}
```



Feasible output:

HC
HP
CT
Bye

Infeasible output:

HP
CT
Bye
HC

wait的另一个例子

- If multiple children completed, will take in arbitrary order(以任意顺序)
- Can use macros (宏)WIFEXITED and WEXITSTATUS to get information about exit status

wait2.c

```
void fork7() {
    pid_t pid[N];
    int i, child_status;

    for (i = 0; i < N; i++)
        if ((pid[i] = fork()) == 0) {
            exit(100+i); /* Child */
        }
    for (i = 0; i < N; i++) { /* Parent */
        pid_t wpid = wait(&child_status);
        if (WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminate abnormally\n", wpid);
    }
}
```

waitpid: 等待一个特定的进程

- `pid_t waitpid(pid_t pid, int &status, int options)`
  - Suspends current process until specific process terminates
  - Various options (see textbook)

waitpid.c

```
void fork8() {
   pid_t pid[N];
   int i;
   int child_status;

   for (i = 0; i < N; i++)
      if ((pid[i] = fork()) == 0)
         exit(100+i); /* Child */
   for (i = N-1; i >= 0; i--) {
      pid_t wpid = waitpid(pid[i], &child_status, 0);
      if (WIFEXITED(child_status))
         printf("Child %d terminated with exit status %d\n",
               wpid, WEXITSTATUS(child_status));
      else
         printf("Child %d terminate abnormally\n", wpid);
   }
}
```

让进程休眠（Putting Processes to Sleep）

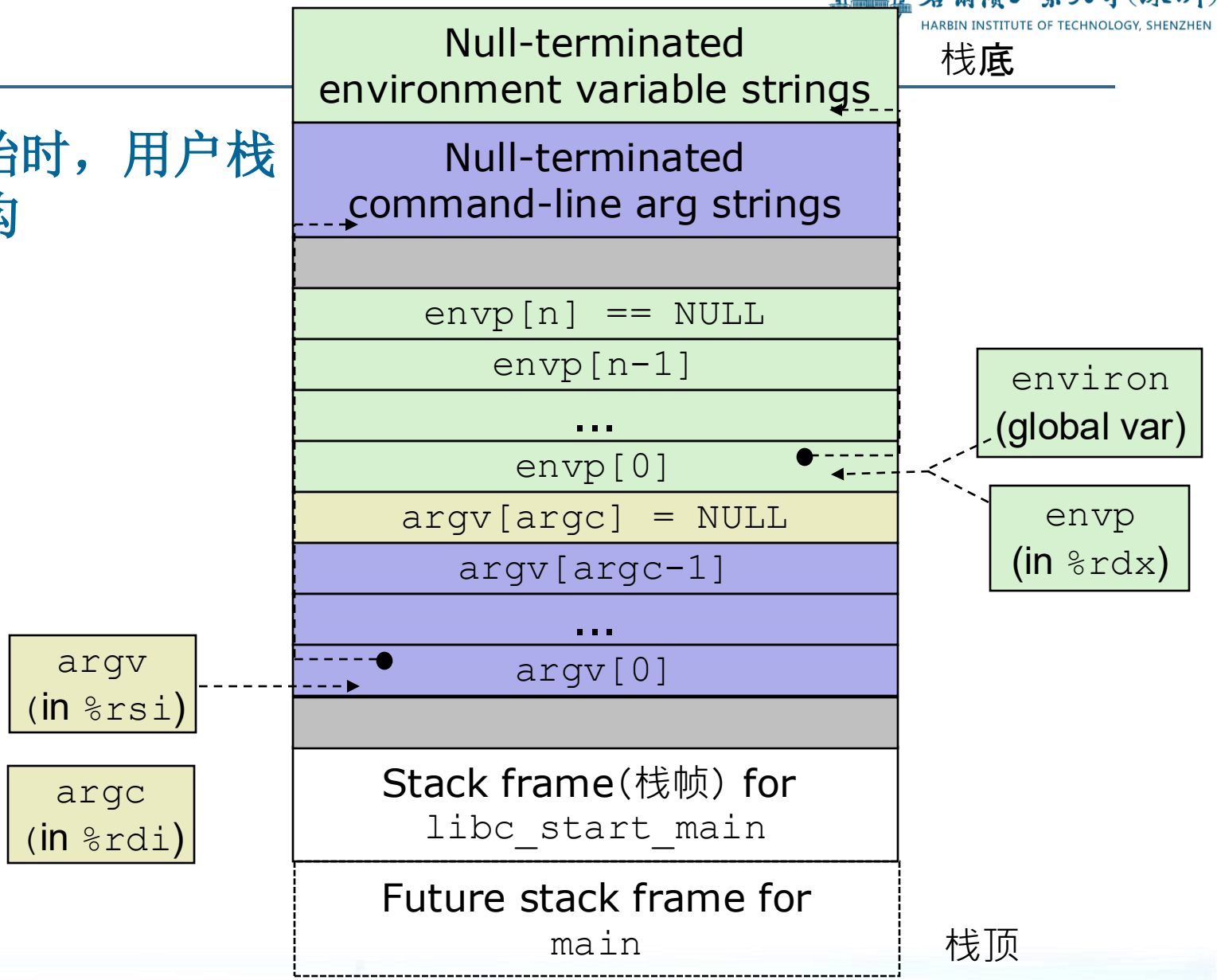
- Sleep function: suspend a process for a specified period of time
  - If the requested amount of time has elapsed(如果请求的时间量已经到了): return 0
  - Else : return the left time (when it is interrupted by a signal)(当被信号中断时，返回剩下的休眠时间秒数)

- Pause function: put the calling function to sleep until a signal is received by the process

execve: 加载并运行新程序

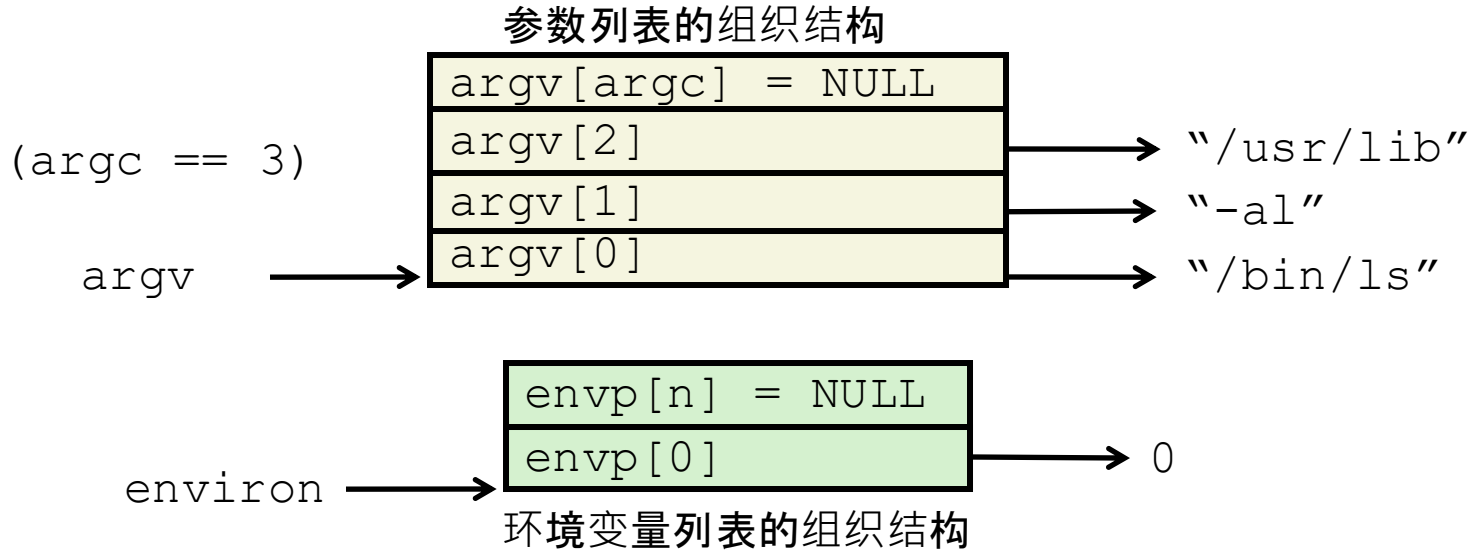
- `int execve(char *filename, char *argv[], char *envp[])`
- Loads and runs in the current process:
  - Executable file **`filename`**(可执行文件)
    - Can be object file or script file (目标文件或脚本文件)beginning with `#!interpreter` (e.g., `#!/bin/bash`)
  - …with argument list (参数列表)**`argv`**
    - By convention **`argv[0]==filename`**
  - …and environment variable list(环境变量列表) **`envp`**
    - "name=value" strings (e.g., `USER=droh`)
    - `getenv, putenv, printenv`
- Overwrites code, data, and stack
  - Retains PID, open files and signal context
- Called once and never returns
  - …except if there is an error

新程序开始时，用户栈的典型结构



execve 举例

- **Executes "/bin/ls -al /usr/lib" in child process using current environment:**



execve.c

```
int main()
{
   char *argv[]={"/bin/ls","-al","/usr/lib", NULL};
   char *envp[]={0,NULL}; /*传递给执行文件新的环境变量数组*/
   execve("/bin/ls",argv,envp);

}
```

execve 举例

```
root@ubuntu:/home/liu/Desktop/OS# ./execve
total 2536
drwxr-xr-x 130 root root   4096 Oct 13 03:01 .
drwxr-xr-x  10 root root   4096 Feb  9  2019 ..
drwxr-xr-x   4 root root   4096 Feb  9  2019 NetworkManager
drwxr-xr-x   2 root root   4096 Feb  9  2019 SYSLINUX
drwxr-xr-x   4 root root   4096 Feb  9  2019 X11
drwxr-xr-x   2 root root   4096 Feb  9  2019 accountsservice
drwxr-xr-x   2 root root   4096 Feb  9  2019 apg
drwxr-xr-x   5 root root   4096 Feb  9  2019 apt
drwxr-xr-x   3 root root   4096 Feb  9  2019 aspell
drwxr-xr-x   2 root root   4096 Feb  9  2019 at-spi2-core
drwxr-xr-x   2 root root   4096 Feb  9  2019 avahi
drwxr-xr-x   2 root root   4096 Oct  8 05:21 bfd-plugins
drwxr-xr-x   2 root root   4096 Apr 20  2018 binfmt.d
drwxr-xr-x   2 root root   4096 Feb  9  2019 bluetooth
-rwxr-xr-x   1 root root    684 May  4  2018 cnf-update-db
drwxr-xr-x   2 root root   4096 Feb  9  2019 colord
-rwxr-xr-x   1 root root   3300 May  4  2018 command-not-found
drwxr-xr-x   2 root root   4096 Oct  8 07:01 compat-ld
```

3.文件所有者

5.文件大小

7.文件名

1. 文件类型与权限

2. 链接占用的节点

4. 文件所有者的用户组

6.文件创建时间/最近修改时间

Summary: 进程控制

- Creating processes
  - Call `fork`
  - One call, two returns
- Process completion
  - Call `exit`
  - One call, no return
- Reaping and waiting for processes
  - Call `wait` or `waitpid`
- Loading and running programs
  - Call `execve` (or variant)
  - One call, (normally) no return

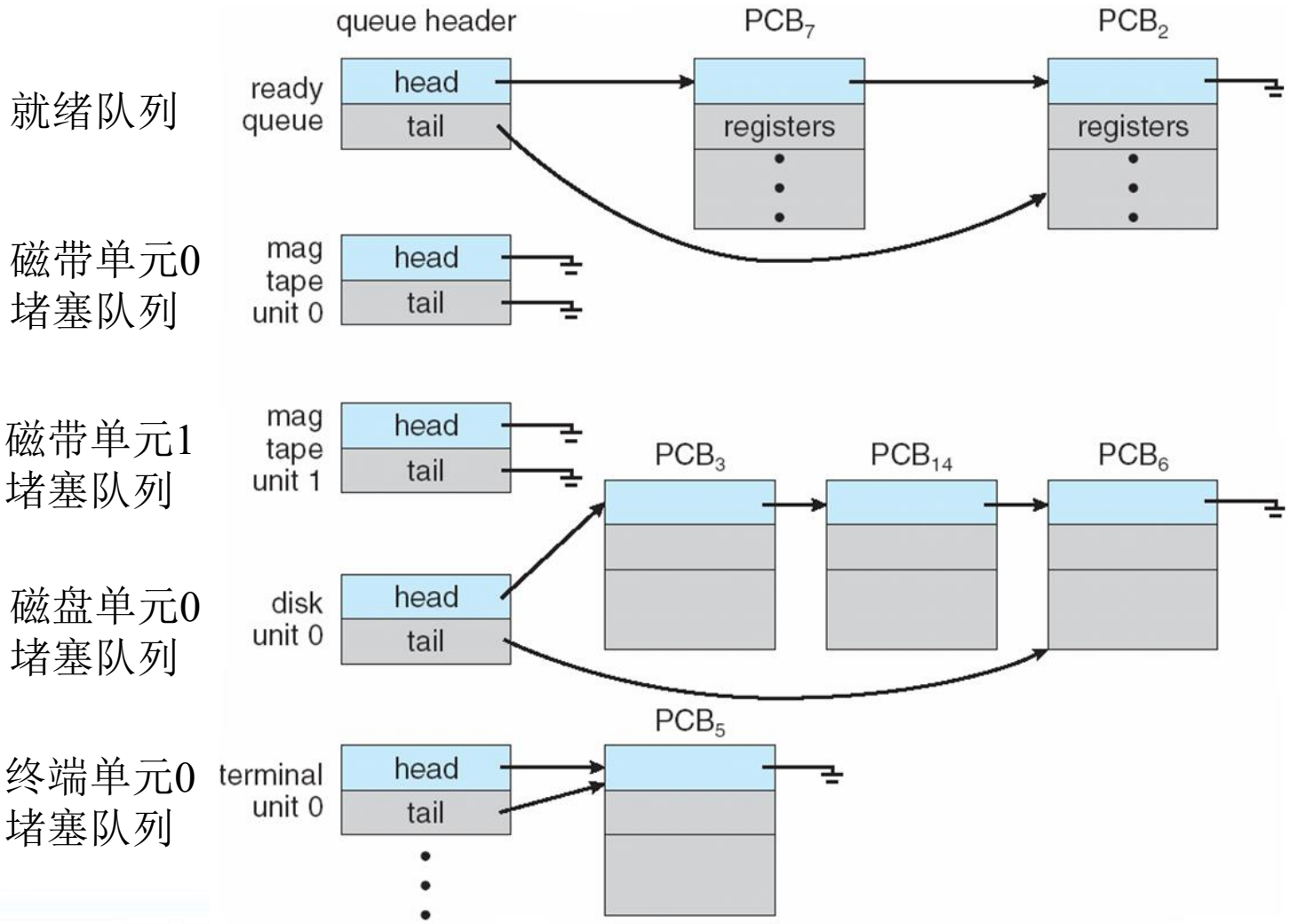
Module 2-1: 进程(Process)

1. 进程的引入
2. 进程模型
3. 进程表示
4. 进程类型和特性
5. 进程控制
6. 进程调度

进程调度

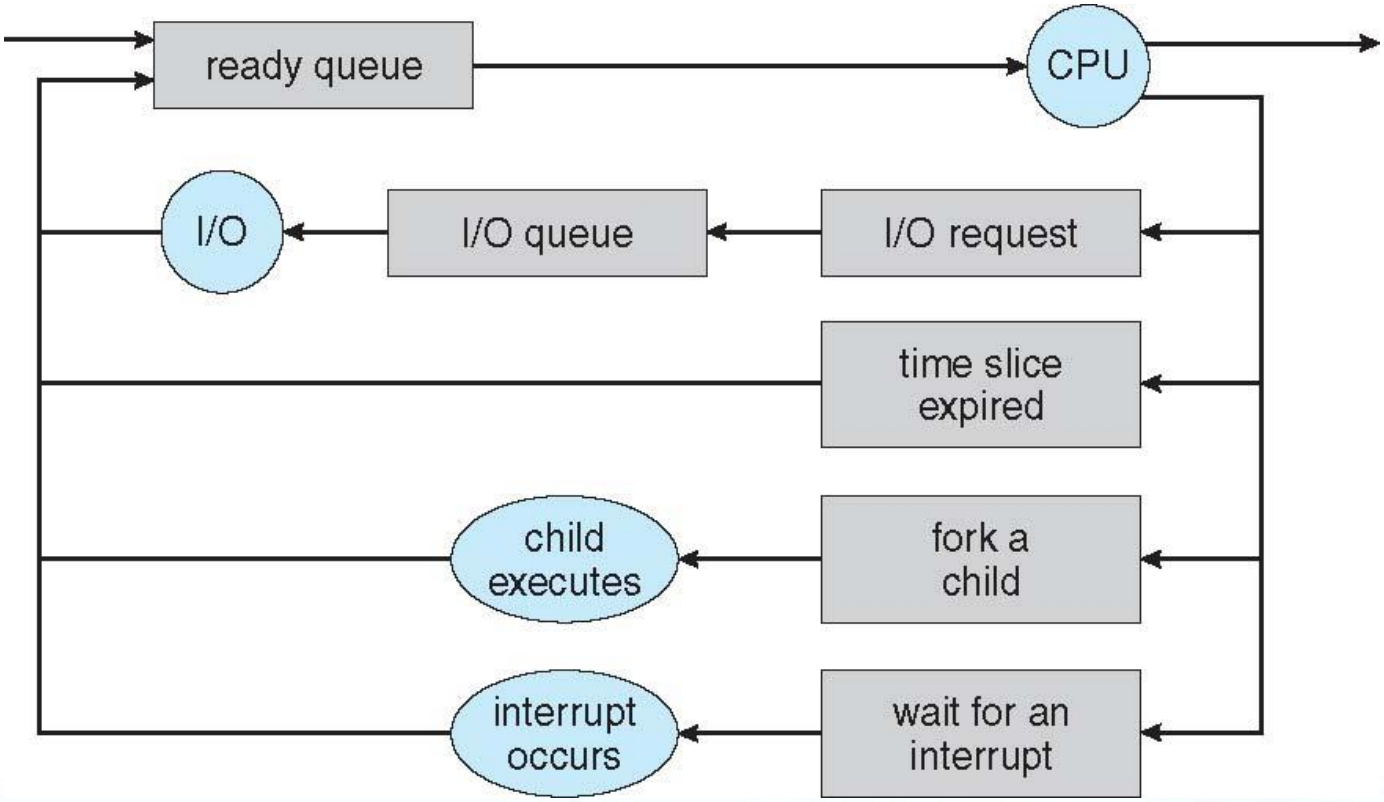
- 进程调度的目的：最大化CPU的使用，可以快速切换进程以便于实现CPU的分时共享
- **进程调度程序(Process scheduler)** 在所有可能的进程中选择出下一个将要在CPU上执行的进程
- 进程调度需要维护进程的调度队列
  - **作业队列 Job queue** – 这个队列包括系统内的所有进程
  - **就绪队列 Ready queue** – 所有驻留在内存中，已经就绪并等待执行的进程构成的队列
  - **设备队列 Device queues** – 等待某个I/O设备的所有进程构成的队列，又被称为阻塞队列
  - 进程在不同的队列间移动

就绪队列和I/O阻塞队列



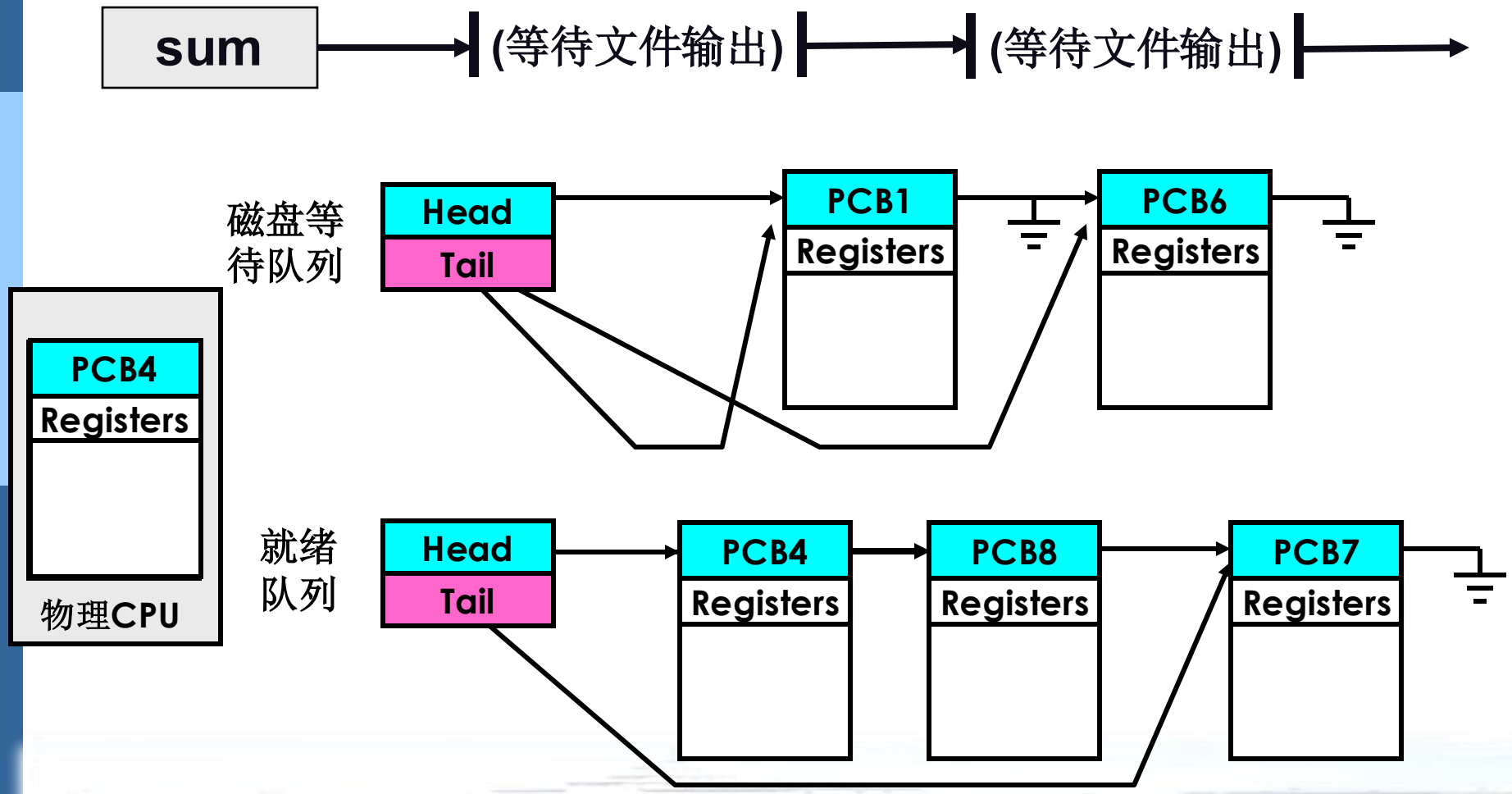
进程调度表示

- 进程调度通常用队列图(Queueing diagram)来表示。
- 每个矩形框代表1个队列；这里有两种队列：就绪队列和设备队列。圆圈表示服务队列的资源；箭头表示系统内的进程流向。



调度程序 (Scheduler)

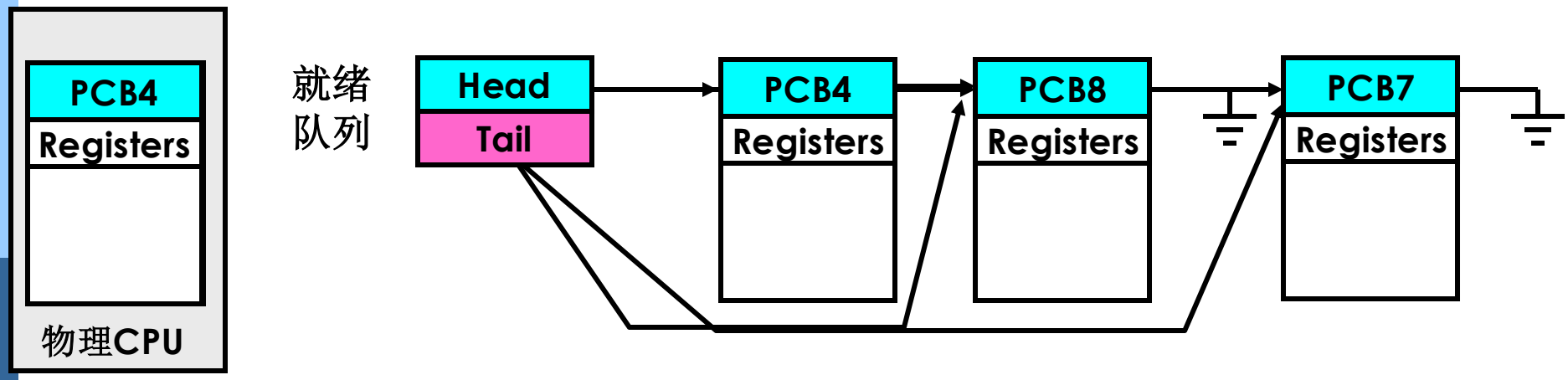
调度情况1： 因等待某些事件而让出CPU



调度程序 (Scheduler)

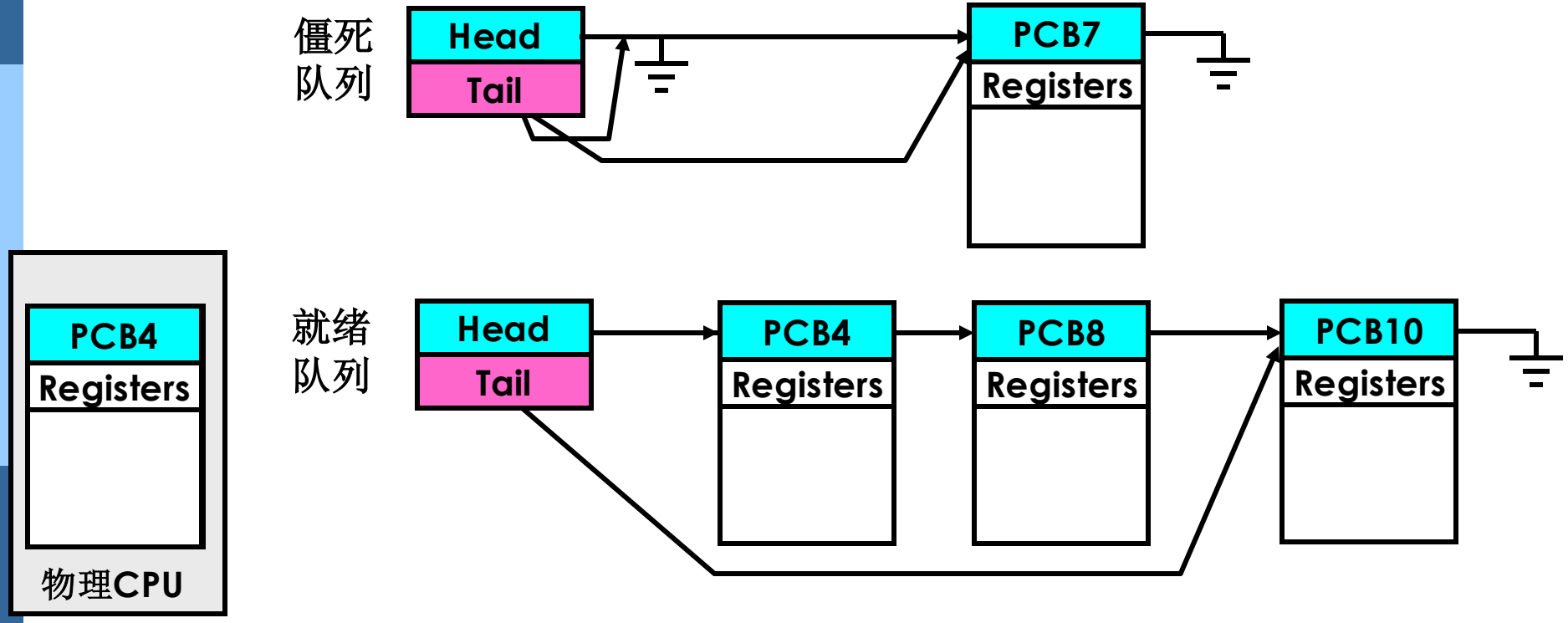
调度情况2：规定的时间片到了

调度情况3：出现了优先级更高的进程



调度程序 (Scheduler)

调度情况4：进程的任务完成了，自动终止退出



调度程序 (Scheduler)

【例题】下列选项中，降低进程优先级的合理时机是( )。

A. 进程时间片用完

B. 进程刚完成I/O操作，进入就绪队列

C. 进程长期处于就绪队列

D. 进程从就绪态转为运行态

答案：A

A. 进程时间片用完，当进程的时间片用完，系统通常会将其优先级降低，以防止占用过多的CPU时间并公平地分配资源。

B. 进程刚完成I/O操作，进入就绪队列：通常会优先运行(提高优先级)，以便利用其I/O结果而减少等待时间。因此，此时降低优先级是不合理的。

C. 进程长期处于就绪队列：通常需要提高优先级，以避免饥饿现象发生，而不是降低优先级。

D. 进程从就绪态转为运行态：此时，进程已被调度器选中运行，降低其优先级会导致不合理的资源分配。

进程调度——策略 (Policy)

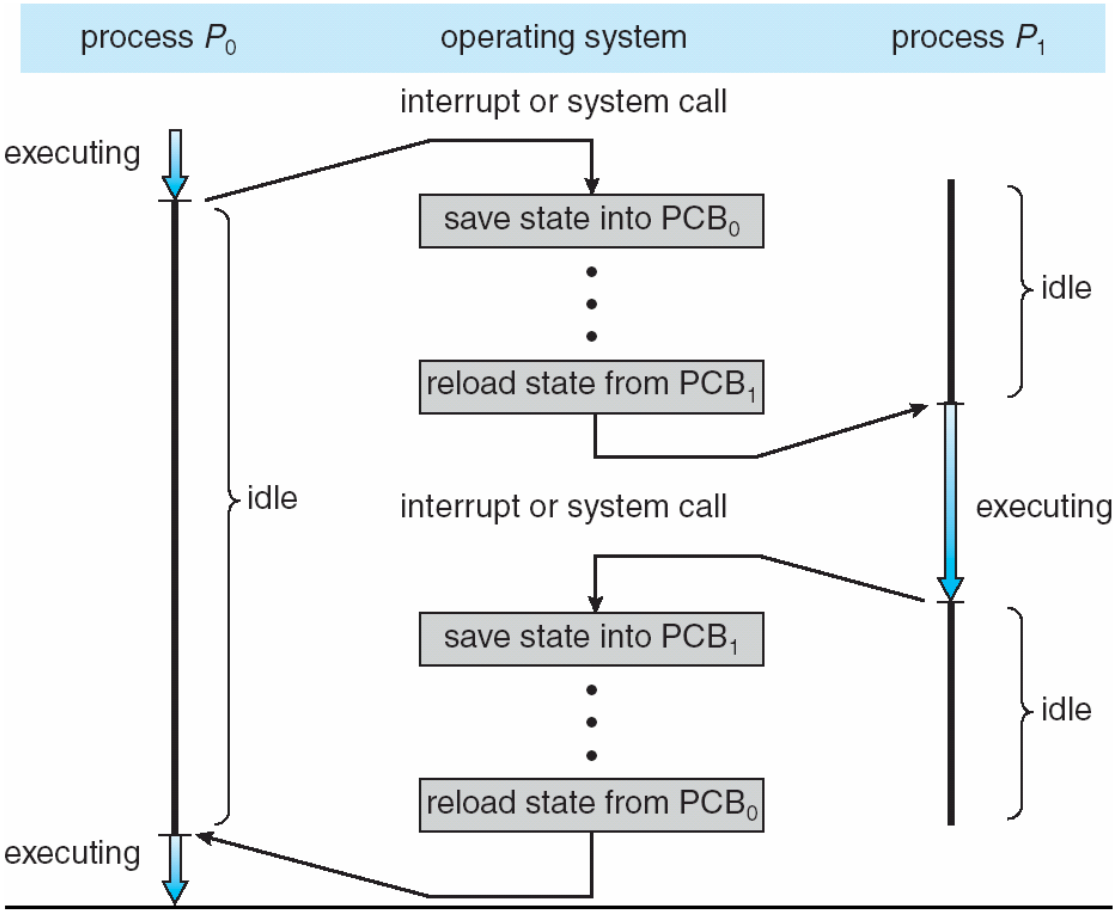
- 进程调度：（1）进程队列迁移；
  （2）下一个进程选取；
  （3）进程切换
- 想一想! 第（2）步应该有很多种选取策略
- 先到先服务 First-Come First-Served?
  - FCFS显然是公平的策略
  - 但 FCFS没有考虑进程执行的任务的区别
- 优先级 Priority?
  - 优先级该怎么设定?可能会使某些进程饥饿
- 公平 vs 效率?

详情请见专题四 CPU调度

进程调度——机制 (mechanism)

- 当CPU切换到另一个进程时，系统必须通过**上下文切换 (context switch)** 保存当前运行在CPU上的进程的状态，然后恢复下一个进程的状态
- Recall: 进程的上下文保存在PCB中
- 上下文切换的时间是纯粹的开销; 因为在切换时系统并没有做任何有用的工作
  - OS 和PCB越复杂 ➔ 上下文切换的时间越长
- 上下文切换的时间依赖于硬件支持
  - 有些处理器 (如 Sun UltraSPARC) 给每个CPU提供了多个寄存器组 ➔ 多个上下文只需加载一次 (multiple contexts loaded at once) ➔ 切换时只需切换到另一个寄存器组上，简单、快捷

进程调度：上下文切换


process P_0
operating system
process P_1
interrupt or system call
executing
save state into PCB_0
idle
reload state from PCB_1
idle
interrupt or system call
executing
save state into PCB_1
idle
reload state from PCB_0
executing

进程调度——伪代码描述

```
extern Queue ReadyQueue;
extern Queue DiskWaitQueue;
…
EnQueue(DiskWaitQueue, pCur);
Dispatch();
…
Dispatch()
{
  pNew = PickNext(ReadyQueue);
  Switch(pCur,pNew);
}
```

进程调度——伪代码描述

```
Switch(pCur，pNew){
        /* 保存现场 */
        PCB[pCur].regs.r7 = CPU.r7;  // 保存寄存器的值
                ...
        PCB[pCur].regs.r0 = CPU.r0;
        PCB[pCur].regs.sp = CPU.sp;  // 保存栈指针
        PCB[pCur].regs.retpc = CPU.retpc;  // 保存PC

        /* 恢复现场 */
        CPU.r7 = PCB[pNew].regs.r7;
                ...
        CPU.r0 = PCB[pNew].regs.r0;
        CPU.sp = PCB[pNew].regs.sp;
        CPU.retpc = PCB[pNew].regs.retpc;
        return; /* Return to CPU.retpc */
}
```

Summary: 进程调度

- 进程调度与进程队列
- 调度程序
- 调度策略
- 调度机制——上下文切换

知识点回顾

- **进程的引入**
  - 进程与程序：进程是运行中的程序
- **进程模型**
  - 异常控制流：控制流，两种改变控制流的方式，异常控制流的低级和高级机制，异常
  - 异常：异常的四种类型（中断、陷阱、故障和终止）
  - 进程：进程的定义，进程提供的两个主要抽象，并发进程与上下文切换
- **进程表示**
  - 进程状态：五个状态，以及状态切换的原因
  - 进程控制块：PCB的作用和主要内容
  - Linux的进程表示：task_struct定义、常用数据结构list_head及相关函数
  - 使用SystemTap追踪task_struct

知识点回顾

- **进程控制**
  - 创建进程: fork()， 一次调用两次返回
  - 为什么需要这样设计API？
  - 终止进程: exit()， 一次调用从不返回
  - 回收和等待: wait()和waitpid()，僵尸进程， 孤儿进程
  - 加载并运行新程序: execve()， 一次调用，通常不返回
- **进程调度**
  - 进程调度与调度队列
  - 调度程序
  - 进程调度策略
  - 进程调度机制——上下文切换

经典例题

1. 下面哪种状态的进程可以被回收？( )

A. 终止态(terminated)　　B. 就绪态(ready)

C. 阻塞态(blocked)　　D. A和C都可以

2. 列出下面程序所有可能的输出序列：

```
int main()
{
   if (fork() == 0) {
            printf("a"); fflush(stdout);
   }
   else {
      printf("b"); fflush(stdout);
      waitpid(-1, NULL, 0);
   }
   printf("c"); fflush(stdout);
   exit(0);
}
```

经典例题

3. 判断下面说法正确的是( )

A. 中断产生的原因可能是来自于CPU内部

B. 系统调用是异步的

C. 故障处理完成会返回到当前指令，重新执行

D. 发生终止，可以尝试恢复并返回到下一条指令

经典例题

4. 编写一个你自己的 Unix system函数，该函数原型如下：

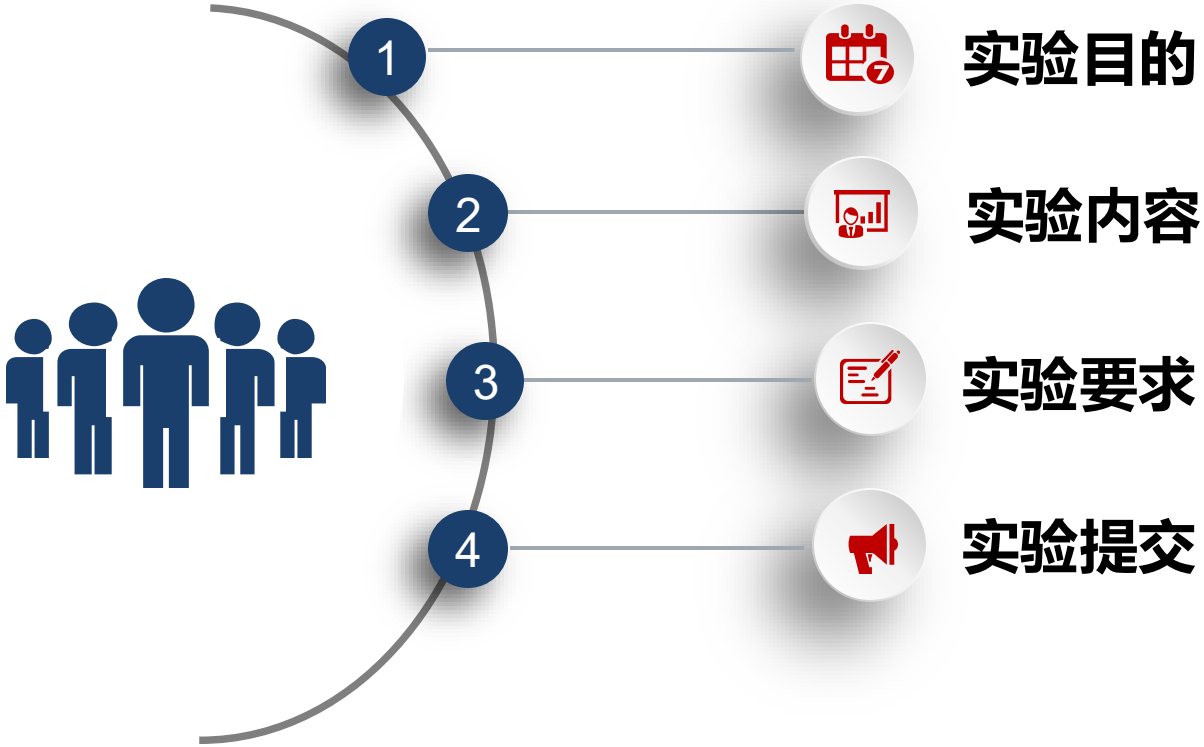
int mysystem(char *command);

mysystem函数通过调用 “/bin/sh –c command” 来执行command，然后在command完成后返回，其中command是你想要让mysystem函数执行的命令。

如果command正常退出 (通过调用exit函数或者执行一条return语句)，那么mysystem返回command退出状态。例如，如果command通过调用exit(8)终止，那么mysystem返回8。

否则，如果command是异常终止的，则mysystem就返回command执行完shell返回的状态。

实验二：进程与系统调用



实验目的

- 了解xv6系统调用的工作原理；
- 熟悉xv6通过系统调用给用户程序提供服务的机制；
- 理解进程和进程切换的概念；
- 了解进程的完整生命周期。

实验内容 | 3个实验任务

① 进程信息收集

进程退出时打印父子进程信息：

- 阅读exit源码；
- 在合适的位置使用exit_info输出父子进程信息。

② wait非阻塞选项

实现非阻塞方式的wait：

- 阅读wait源码；
- 增加wait系统调用的非阻塞选项参数int flags；
- 当flags=1时，不进行sleep，而是直接返回-1。

③ yield系统调用

实现yield系统调用：

- 理解系统调用的流程；
- 新增sys_yield函数；
- 在该函数中打印用户上下文中保存的PC值；并将当前进程挂起。

实验内容 | 获取syscall分支

- 本实验的**内容及要求与MIT xv6 syscall不一样**，请大家以指导书为准。

1. 在做实验前，需保存上一个实验的更改。

- 使用git命令完成commit

```
$ git add --all
$ git commit -m “本次commit的注释”
```

建议：

- 每次修改代码前先进行commit
- commit注释尽可能详细，便于后续跟踪代码进度

2. 切换到 syscall 分支，在该分支的代码上进行实验

```
$ git checkout syscall
$ git fetch
```

```
os@os:~/hitsz/test/xv6-oslab23-hitsz$ git checkout syscall
Branch 'syscall' set up to track remote branch 'syscall' from 'origin'.
Switched to a new branch 'syscall'
os@os:~/hitsz/test/xv6-oslab23-hitsz$ git fetch
os@os:~/hitsz/test/xv6-oslab23-hitsz$
```

如果没有代码库，也可以直接获取

```
$ git clone -b syscall https://gitee.com/greenhandzpx/xv6-oslab23-hitsz.git
```

实验任务一：进程信息收集

实验要求

- 在exit系统调用当中寻找合适的输出时间点打印出父、子进程信息，格式如下。
  - 当前进程的*父进程*: *proc PID exit, parent pid PID, name NAME, state STATE*
  - 当前进程的*子进程*: *proc PID exit, child CHILD_NUM, pid PID, name NAME, state STATE*

- *PID:* 进程的进程号
- *CHILD_NUM*: 该信息是第几个子进程的信息
- *NAME:* 父进程或者子进程名
- *STATE:* 父进程或者子进程的进程状态

不要使用printf输出， 请使用exit_info函数输出

```
xv6 kernel is booting

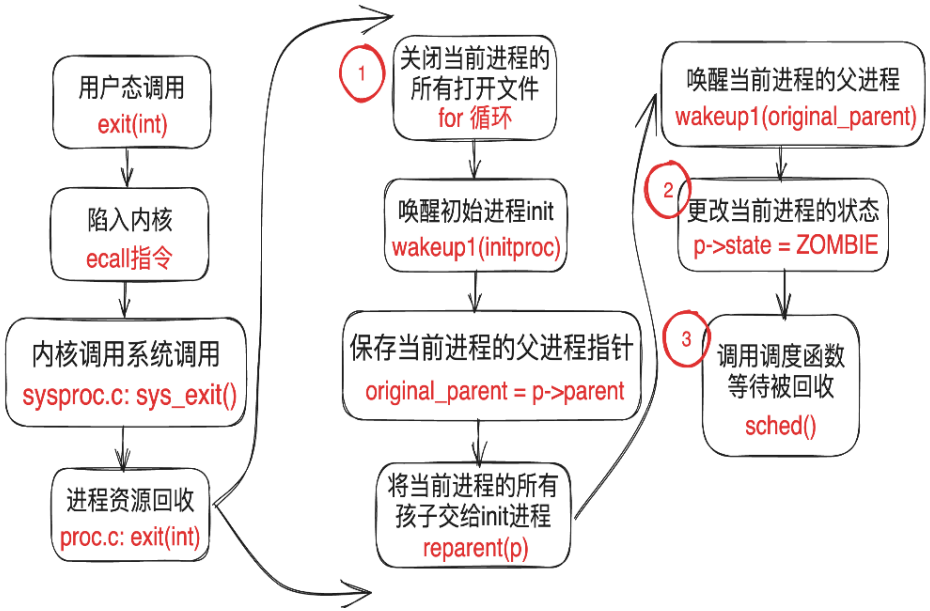
hart 2 starting
hart 1 starting
init: starting sh
$ exittest
exit test
proc 3 exit, parent pid 2, name sh, state SLEEPING
proc 3 exit, child 0, pid 4, name child0, state SLEEPING
proc 3 exit, child 1, pid 5, name child1, state SLEEPING
proc 3 exit, child 2, pid 6, name child2, state SLEEPING
$ proc 4 exit, parent pid 1, name init, state SLEEPING
proc 5 exit, parent pid 1, name init, state SLEEPING
proc 6 exit, parent pid 1, name init, state RUNNABLE
```

实验任务一：进程信息收集

实验提示

- 在理解*exit系统调用*工作流程的基础上，再实现打印信息这一功能。

1. 如何获取父/子进程信息？
   - 进程控制块（PCB）
2. 如何表征进程的状态信息？
   - 在*proc.h: enum procstate* 中选取




```
enum procstate { UNUSED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
```

实验任务二： wait系统调用的非阻塞选项实现

实验要求

- 在wait系统调用增加*int flags*，当*flags=1*时不需要阻塞等待，而是直接返回-1
  - 非阻塞的wait： *int wait(uint64 addr)* → *int wait(uint64 addr, int flags)*
  - 需要改动：

- 在*sys_wait*函数中获取新添加的参数（sysproc.c）；
- 更改头文件的wait的定义（defs.h）；
- 更改*wait*函数以满足当前的语义（proc.c）；
- 通过*waittest*测试。

```
xv6 kernel is booting

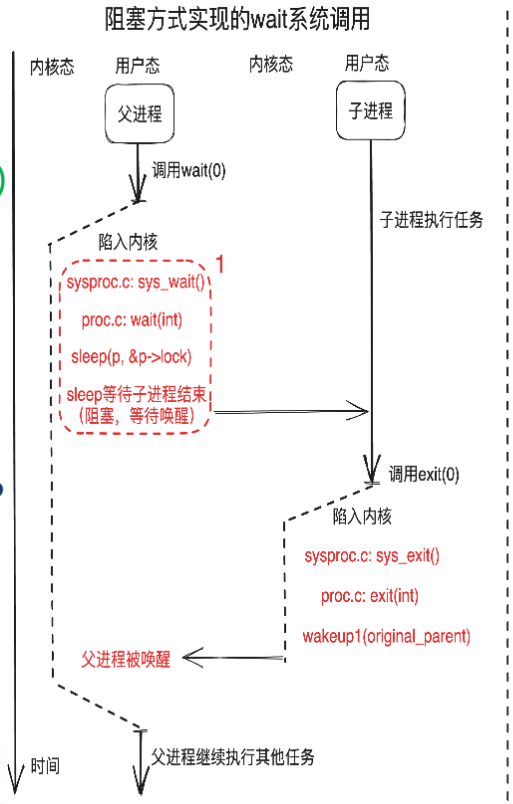
hart 2 starting
hart 1 starting
init: starting sh
$ waittest
wait test
no child exited yet, round 0
no child exited yet, round 1
no child exited yet, round 2
proc 4 exit, parent pid 3, name waittest, state SLEEPING
child exited, pid 4
wait test OK
proc 3 exit, parent pid 2, name sh, state SLEEPING
$
```

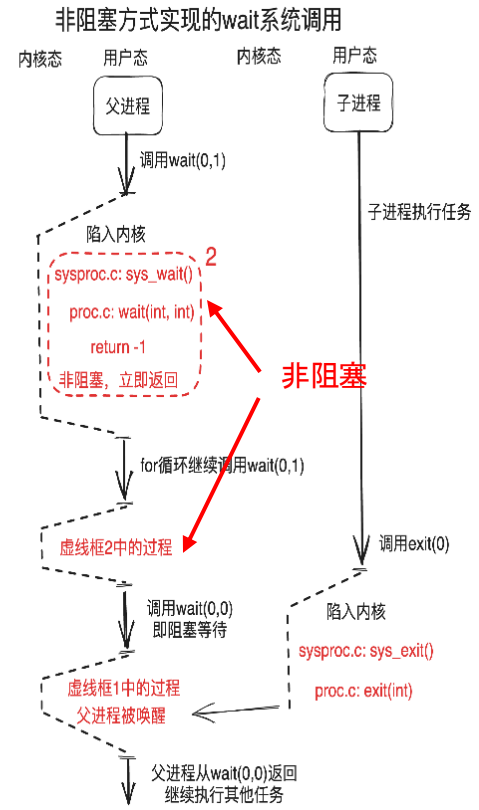
实验任务二： wait系统调用的非阻塞选项实现

实验提示

- 在理解*wait系统调用*工作流程的基础上，再实现非阻塞这一功能。

1. 阻塞与非阻塞实现的区别？ (*sleep*)
2. 如何获取用户态传入的参数?
   - 寄存器传参
3. 如何保证进程PCB的操作的互斥性?
   - wait函数会通过freeproc改写PCB
   - 上锁：*acquire(&p->lock)*
   - 解锁：*release(&p->lock)*





实验任务三：实现yield系统调用

实验要求

- 实现一个新的系统调用yield：*int yield();*
  - 需要打印用户态陷入内核时对应的PC值：*start to yield, user pc 0x???????*
  - 还可以使当前进程让出CPU，从而使CPU调度到别的进程；
  - 需要注意：
    - 修改Makefile，添加_yieldtest;

```
162         $U/_waittest\
163         $U/_exittest\
164         $U/_yieldtest\
165
```

```
xv6 kernel is booting

init: starting sh
$ yieldtest
yield test
parent yield
start to yield, user pc 0x00000000000003e0
switch to child 0
proc 4 exit, parent pid 3, name yieldtest, state RUNNABLE
switch to child 1
proc 5 exit, parent pid 3, name yieldt
switch to child 2
proc 6 exit, parent pid 3, name yieldtest, state RUNNABLE
parent yield finished
proc 3 exit, parent pid 2, name sh, state SLEEPING
```

打印用户态的PC值

    - 测试时，需将CPU的数量设为1。

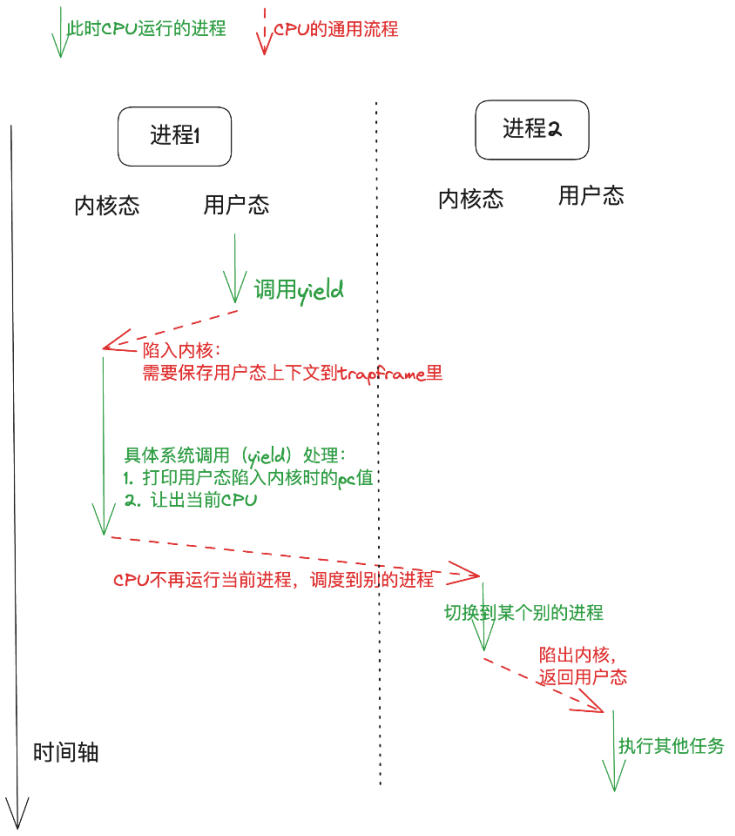
```
$ make qemu CPUS=1
```

实验任务三：实现yield系统调用

实验提示

- 在理解*系统调用*整体流程的基础上，再实现新的系统调用yield。

1. 如何获取用户态陷入内核的PC值，即ecall指令的地址？
   - 进程控制块（PCB）
2. 多核心调度与单核心调度区别？
   - 指定CPU的数量为1



实验提交

- 当完成上述的三个任务后，在xv6-oslab23-hitsz目录下，新建time.txt，写入你做完这个实验所花费的时间（估算一下就行，单位是小时），只需要写一个整数即可。

```
$ echo 3 > time.txt
```

- 在命令行输入 *make grade* 进行测试。如果通过测试，会显示如下内容：

```
$ make grade
```

```
$ make qemu-gdb LAB_SYSCALL_TEST=on
exit test: OK (4.0s)
== Test wait test ==
$ make qemu-gdb LAB_SYSCALL_TEST=on
wait test: OK (1.9s)
== Test yield test ==
$ make qemu-gdb CPUS=1 LAB_SYSCALL_TEST=on
yield test: OK (0.6s)
== Test time ==
time: OK
Score: 100/100
```

实验提交

- 请务必注意查看作业提交平台(地址见实验指导书)，按时提交代码及报告
- 实验二提交截止时间：两个星期内
- 实验提交内容（将以下内容打包提交）
  - 实验报告
  - **commit.patch文件(使用make diff命令导出，无需提交整个实验代码库)**

实验步骤：任务一

在kernel/proc.c的exit函数添加以下代码

```
#define NSTATE 5
// 在 proc.c 中定义进程状态字符串
char *states[NSTATE] = { "UNUSED", "SLEEPING", "RUNNABLE", "RUNNING", "ZOMBIE" };

// 在 proc.c 文件的exit函数添加如下代码:
void
exit(int status)
{
   struct proc *p = myproc();
   struct proc *pp = p->parent;
   struct proc *child;
   int i;

   // 打印当前进程退出时父进程的信息
   if (pp) {
      printf("proc %d exit, parent pid %d, name %s, state %s\n",
            p->pid, pp->pid, pp->name, states[p->state]);
   } else {
      printf("proc %d exit, parent pid -1, name %s, state %s\n",
            p->pid, p->name, states[p->state]);
   }

   // 打印当前进程的子进程信息
   int child_num = 0;
   for (i = 0; i < NPROC; i++) {
      child = &proc[i];
      if (child->parent == p) {
         printf("proc %d exit, child %d, pid %d, name %s, state %s\n",
               p->pid, child_num++, child->pid, child->name, states[child->state]);
      }
   }

   // 其他现有的exit逻辑
   // ...
}
```

实验步骤：任务一

实验结果

编译
$ make qemu

在xv6中运行exittest输出如下结果

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ exittest
exit test
proc 3 exit, parent pid 2, name sh, state RUNNING
proc 3 exit, child 4, pid 4, name child0, state SLEEPING
proc 3 exit, child 5, pid 5, name child1, state SLEEPING
proc 3 exit, child 6, pid 6, name child2, state SLEEPING
$ proc 5 exit, parent pid 1, name init, state RUNNING
proc 4 exit, parent pid 1, name init, state RUNNING
proc 6 exit, parent pid 1, name init, state RUNNING
```

实验步骤：任务二

1、在kernel/sysproc.c的sys_wait函数中，修改为以下代码

```
uint64 sys_wait(void) {
  uint64 p;
  int flags;
  if (argaddr(0, &p) < 0) return -1;
  if (argint(1, &flags) < 0) return -1;
  return wait(p, flags);
}
```

2、更改kernel/defs.h头文件中的wait的定义；

```
int          wait(uint64, int);
```

3、更改kernel/proc.c中的wait实现；

```
int wait(uint64 addr, int flags) {
  if(flags == 1) return -1;
```

实验步骤：任务二

实验结果

编译

$ make qemu

在xv6中运行waittest输出如下结果

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ waittest
wait test
no child exited yet, round 0
no child exited yet, round 1
no child exited yet, round 2
proc 4 exit, parent pid 3, name waittest, state RUNNING
child exited, pid 4
wait test OK
proc 3 exit, parent pid 2, name sh, state RUNNING
$
```

实验步骤：任务三

1、在kernel/proc.c的yield函数中加入打印语句

```
printf("start to yield, user pc %p\n", p->trapframe->epc);
```

2、在kernel/sysproc.c中加入sys_yield函数的实现

```
uint64 sys_yield(void) {
  yield();
  return 0;
}
```

3、在kernel/syscall.c添加系统调用

```
extern uint64 sys_yield(void);
```

4、在kernel/syscall.h添加系统调用号

```
#define SYS_unlink 18
#define SYS_link   19
#define SYS_mkdir  20
#define SYS_close  21
#define SYS_rename 22
#define SYS_yield 23
~
```

必须连续

```
    [SYS_mknod] sys_mknod, [SYS_unlink] sys_unlink, [SYS_link] sys_link,
S_mkdir] sys_mkdir,
    [SYS_close] sys_close, [SYS_rename] sys_rename, [SYS_yield] sys_yield,
};
```

实验步骤：任务三

5、在user/user.h添加yield函数声明

```
void yield(void);
```

6、在user/usys.pl添加函数入口定义

```
entry("yield");
```

7、在Makefile中加入一行

```
$U/_wc\
$U/_zombie\
$U/_waittest\
$U/_exittest\
$U/_yieldtest\
```

实验步骤：任务三

实验结果

编译

$ make qemu CPUS=1

在xv6中运行yieldtest输出如下结果

```
xv6 kernel is booting

init: starting sh
$ yieldtest
yield test
parent yield
start to yield, user pc 0x00000000000003e0
switch to child 0
proc 4 exit, parent pid 3, name yieldtest, state RUNNING
switch to child 1
proc 5 exit, parent pid 3, name yieldtest, state RUNNING
switch to child 2
proc 6 exit, parent pid 3, name yieldtest, state RUNNING
parent yield finished
proc 3 exit, parent pid 2, name sh, state RUNNING
```

实验步骤：

最终评估

```
$ echo 3 > time.txt
$ make grade
```

```
$ make qemu-gdb LAB_SYSCALL_TEST=on
exit test: OK (3.0s)
== Test wait test ==
$ make qemu-gdb LAB_SYSCALL_TEST=on
wait test: OK (1.8s)
== Test yield test ==
$ make qemu-gdb CPUS=1 LAB_SYSCALL_TEST=on
yield test: OK (0.6s)
== Test time ==
time: OK
Score: 100/100
```


obrigado
ありがとうございます
bayarll
죄송해요
СПАСИБО
ভালো আছি
sağol
Köszi
شکریه (بهت)
Kiitos
merci bien
Thank You
谢 谢
Dank
хвала
شکرا
ขอบคุณ
Gracias
ευχαριστώ
Dank u
Tak
CẢM ƠN CÔ
Děkuji
תודה
terima kasih